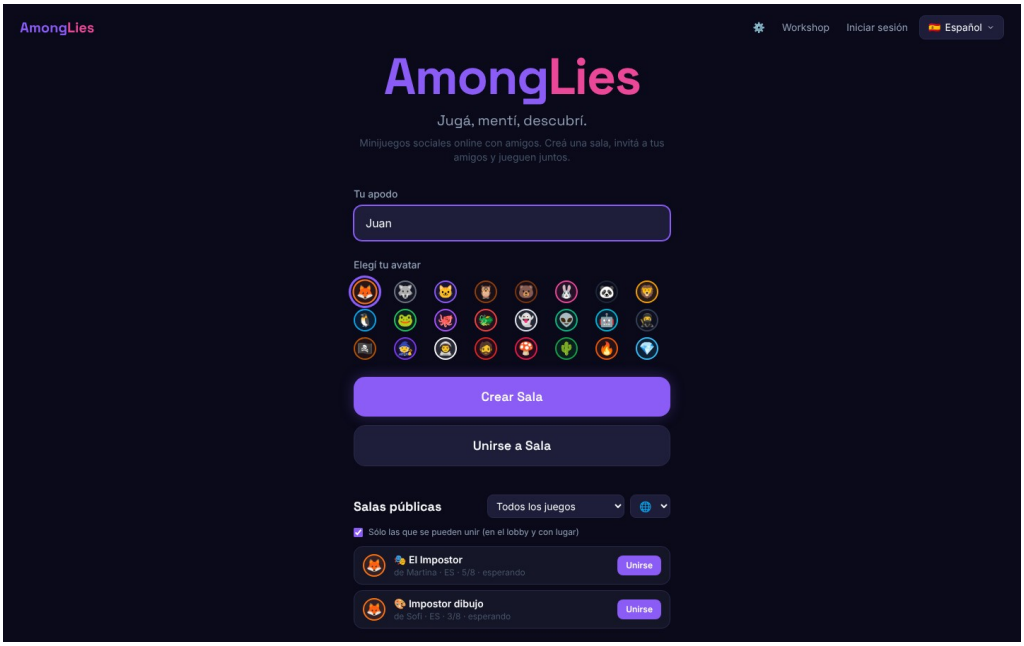
AmongLies

Jugá, mentí, descubrí.

Documentación de Arquitectura y Manual de Usuario

Versión final del sistema (v3.0)



Trabajo Práctico N.° 3 — Documentación

Materia: Desarrollo de Sistemas

Integrantes: _____

Octubre de 2026

Control del documento

Versión	Fecha	Descripción
1.0	16/09/2026	Primera entrega: arquitectura y manual del MVP.
2.0	Septiembre 2026	Reescritura completa para la **versión final** planificada, con plantilla arc42, modelo C4, decisiones con criterios, manual con capturas anotadas y corrección de datos inconsistentes con el código.
3.0	24/10/2026	Documenta el **producto terminado**: los tres juegos que se construyeron (El Impostor, Impostor por tiempo e Impostor dibujo), amigos, workshop, moderación, ajustes y sonido; arquitectura real (estado en memoria, sin Redis; despliegue en Render); pruebas automáticas y de punta a punta. Todas las capturas del manual son del sistema real.

Tabla 1 — Historial de versiones

Respuesta a la devolución de la entrega anterior

La siguiente tabla indica dónde se atiende cada observación recibida en la corrección y en la defensa oral:

Observación	Cómo se resolvió	Dónde
La documentación describía el MVP y no la versión final de la página (defensa oral).	Todo el documento describe la **versión final terminada (v3.0)**: cada requerimiento está implementado y probado, y se explica qué cambió respecto de lo planificado.	Todo el documento; §1.3, §3.3
No incluyeron diagramas ni ayuda para todas las funcionalidades planteadas.	17 diagramas generados como código y un manual que cubre todas las funcionalidades: los tres juegos, amigos, workshop, ajustes y moderación.	§3–§8; M1–M17
Falta de objetivos del proyecto.	Objetivos medibles (con su verificación), objetivos de calidad priorizados y partes interesadas.	§1.2, §1.4, §1.5
No incluyen restricciones, contexto ni alcance.	Restricciones técnicas, organizativas y legales; contexto de negocio y técnico; alcance dentro/fuera.	§2, §3
Falta de gráficos para explicar la arquitectura.	Modelo C4 completo (contexto, contenedores, componentes, código), secuencias, estados, datos y despliegue.	§3–§8
Falta de detalle en el criterio de las decisiones; foco en tecnologías.	Criterios ponderados derivados de los objetivos, 17 ADRs con alternativas, matrices de puntaje y consecuencias negativas.	§9
Falta de índice.	Índice general al inicio.	Índice
Manual de usuario sin ayudas visuales.	Más de 60 capturas **del sistema real** (partidas jugadas con 4 navegadores, en escritorio y celular); las capturas clave están anotadas con números y leyenda.	Parte II
Manual sin contexto previo al flujo de juego.	Secciones de bienvenida, conceptos clave, mapa de navegación, matriz de permisos y mapa de pantallas antes de los flujos.	M1–M4

Tabla 2 — Trazabilidad de la devolución

> **Correcciones respecto de la versión 1.0**
>
> Al contrastar la entrega anterior con el código se corrigieron datos que no coincidían: el código de sala tiene **6** caracteres (no 4); hay **24** avatares (no 8); el apodo admite de **2 a 16** caracteres (no 20); El Impostor requiere **4** jugadores (no 3); el frontend usa **Next.js 16** (no 15); la sala no tiene un máximo fijo de 8 jugadores; y la revelación de palabra avanza por **temporizador**, no con un botón “Listo”.

Qué cambió entre lo planificado (v2.0) y lo construido (v3.0)

La versión 2.0 describía el producto que el equipo **planeaba** construir. Durante el desarrollo se tomaron decisiones que cambiaron el plan; este documento describe lo que **realmente quedó** y, donde corresponde, explica por qué:

- **Juegos:** en lugar de *Dibujá y Adiviná* y *Teléfono Descompuesto* se construyeron dos variantes de El Impostor — **Impostor por tiempo** e **Impostor dibujo** — que reutilizan toda la mecánica de deducción (roles ocultos, discusión, votación, puntaje). Ver ADR-08.
- **Datos:** el estado de las partidas vive **en memoria** de una sola instancia; se descartó Redis (ADR-05). Las estadísticas, amigos, workshop y reportes se guardan en **Supabase**.
- **Despliegue: Render** en lugar de Railway (ADR-13).
- **Idiomas:** español e inglés; se quitó el portugués. **Cuentas:** email y contraseña (sin Google).
- **Nuevo, no planificado en la v2.0:** amigos con presencia e invitaciones, workshop de listas de palabras y presets, moderación con panel de administración, ajustes de usuario, sonido, puntaje por partida y pruebas de punta a punta en el CI.

Índice

Control del documento 2
Respuesta a la devolución de la entrega anterior 2
Índice 4
Documentación de Arquitectura 8
Cómo leer este documento 9
Estructura: plantilla arc42 + modelo C4 9
1. Introducción y objetivos 10
1.1 Propósito del sistema 10
1.2 Objetivos del proyecto 10
1.3 Requerimientos funcionales principales 10
1.4 Objetivos de calidad 11
1.5 Partes interesadas 12
2. Restricciones 13
2.1 Restricciones técnicas 13
2.2 Restricciones organizativas 13
2.3 Convenciones 13
2.4 Restricciones legales y éticas 13
3. Contexto y alcance 15
3.1 Contexto de negocio (C4 — Nivel 1) 15
3.2 Contexto técnico 15
3.3 Alcance 16
4. Estrategia de solución 17
4.1 Estilo arquitectónico 17
4.2 Enfoques por objetivo 17
4.3 De la tecnología al impacto para el usuario 17
5. Vista de bloques de construcción 19
5.1 Nivel 1 — Contenedores (C4 — Nivel 2) 19
5.2 Nivel 2 — Componentes (C4 — Nivel 3) 20
5.3 Nivel 3 — Código (C4 — Nivel 4): el contrato de los motores de juego 21
5.4 Estructura del repositorio 22
6. Vista de ejecución 23
6.1 Escenario 1 — Crear una sala y unirse 23
6.2 Escenario 2 — Una ronda de El Impostor 24
6.3 Máquina de estados del motor El Impostor 25
6.4 Escenario 3 — Fin del juego y registro de estadísticas 26
6.5 Escenario 4 — Inicio de sesión y conexión autenticada 26
6.6 Escenario 5 — Desconexión y reconexión 27
6.7 Catálogo de eventos 27
7. Vista de despliegue 29

Entornos....30
8. Conceptos transversales....31
8.1 Modelo de dominio....31
8.2 Diseño de datos....31
8.3 Contrato de comunicación....33
8.4 Seguridad e integridad del juego....33
8.5 Manejo del tiempo....34
8.6 Identidad y sesiones....34
8.7 Internacionalización....34
8.8 Manejo de errores y retroalimentación....34
8.9 Escalabilidad....34
8.10 Observabilidad....35
8.11 Estrategia de pruebas....35
8.12 Diseño visual y experiencia....35
9. Decisiones de arquitectura....36
9.1 Criterios de evaluación....36
ADR-01 — Comunicación en tiempo real con Socket.io....36
ADR-02 — Servidor autoritativo....37
ADR-03 — Monolito modular con NestJS....37
ADR-04 — Next.js (App Router) para la aplicación web....37
ADR-05 — Estado vivo en memoria (sin Redis)....37
ADR-06 — PostgreSQL (Supabase) para datos persistentes....38
ADR-07 — Cuentas opcionales (invitado primero) con Supabase Auth....38
ADR-08 — Motores de juego enchufables....38
ADR-09 — Vistas por jugador calculadas en el servidor....39
ADR-10 — Modo voz sin audio propio....39
ADR-11 — Zustand para el estado del cliente....39
ADR-12 — Monorepo con tipos compartidos....39
ADR-13 — Despliegue en Render....40
ADR-14 — Listado de salas públicas por REST....40
ADR-15 — Resultados escritos sólo por el servidor....40
ADR-16 — Amigos y workshop por el socket del juego....40
ADR-17 — Moderación: filtro compartido, reportes y panel de admin....41
10. Requerimientos de calidad....42
10.1 Árbol de calidad....42
10.2 Escenarios de calidad....42
11. Riesgos y deuda técnica....44
11.1 Riesgos....44
11.2 Deuda técnica del MVP: estado....44
12. Glosario....46
Manual de Usuario....48

M1. Bienvenida..... 49
Los tres minijuegos..... 49
Qué necesitás..... 49
M2. Antes de jugar: conceptos clave..... 50
El recorrido general..... 50
M3. Tipos de usuario y permisos..... 52
M4. Mapa de pantallas..... 53
M5. Primeros pasos..... 54
M5.1 Cómo elegir tu apodo y avatar..... 54
M5.2 Cómo crear una cuenta o iniciar sesión (opcional)..... 55
M5.3 Cómo cambiar el idioma..... 55
M6. Salas..... 56
M6.1 Cómo crear una sala..... 56
M6.2 Cómo invitar a tus amigos..... 56
M6.3 Cómo unirse con un código..... 57
M6.4 Cómo unirse con un link..... 57
M6.5 Cómo encontrar una sala pública..... 58
M6.6 Qué ve un jugador en el lobby..... 59
M6.7 Cómo configurar el juego (anfitrión)..... 59
M6.8 Cómo gestionar jugadores (anfitrión)..... 61
M6.9 Cómo usar el chat..... 61
M6.10 Cómo salir de una sala..... 61
M7. Cómo jugar El Impostor..... 62
Objetivo y reglas..... 62
M7.1 Ver tu palabra..... 62
M7.2 Dar tu pista (modo chat)..... 63
M7.3 Dar tu pista (modo voz)..... 64
M7.4 Discutir..... 65
M7.5 Votar..... 66
M7.6 Ver los resultados..... 67
M7.7 Si te eliminan..... 68
M7.8 Fin de partida..... 69
M7.9 Fin del juego..... 69
M8. Cómo jugar Impostor dibujo..... 71
Reglas..... 71
Cómo dibujar..... 71
Ajustes propios..... 72
M9. Cómo jugar Impostor por tiempo..... 73
Reglas..... 73
Tu turno..... 73
Ajustes propios..... 74

M10. Cómo ver tu perfil y estadísticas....75
M11. Amigos e invitaciones....76
M11.1 Cómo invitar a un amigo a tu sala....76
M12. Workshop: listas de palabras y presets....78
M12.1 Cómo crear una lista de palabras....78
M12.2 Cómo usarlo en el lobby....79
M13. Ajustes y sonido....80
M14. Reportar y moderación....82
M14.1 Cómo reportar....82
M14.2 Panel de admin (sólo administradores)....82
M15. Uso desde el celular....84
M16. Problemas comunes....85
M17. Preguntas frecuentes....86
Anexo A — Fuentes de los diagramas (Mermaid)....87
Anexo B — Referencias....96

PARTE I

Documentación de Arquitectura

Cómo está construido AmongLies y por qué — arc42 + C4

Cómo leer este documento

Este documento describe la arquitectura y el uso de **AmongLies en su versión final (v3.0)**, tal como está construida y desplegada en https://amonglies-web.onrender.com. A diferencia de la versión anterior, ya no hay funcionalidades "planificadas": **todo lo que se describe está implementado**, y todas las capturas del manual se tomaron del sistema real, jugando partidas con cuatro navegadores.

El documento tiene dos partes con públicos distintos, siguiendo el principio de separar la ingeniería del uso:

	Parte I — Arquitectura	Parte II — Manual de usuario
Dirigida a	Desarrolladores, docentes evaluadores, quien opere el despliegue.	Jugadores y anfitriones de sala (usuarios no técnicos).
Responde a	¿Cómo está construido el sistema y **por qué** se construyó así?	¿Cómo uso la aplicación para jugar con mis amigos?
Enfoque	Contexto, decisiones, criterios, calidad, riesgos.	Pantallas, pasos, permisos, problemas comunes.

Tabla 3 — Públicos y alcance de cada parte del documento

Estructura: plantilla arc42 + modelo C4

La Parte I se organiza con la plantilla **arc42** (doce secciones con un lugar predecible para cada tipo de información) y los diagramas siguen el **modelo C4** (cuatro niveles de zoom: contexto, contenedores, componentes y código). La correspondencia entre ambos es la siguiente:

Diagrama C4	Sección arc42 donde se ubica
Nivel 1 — Contexto del sistema	§3 Contexto y alcance
Nivel 2 — Contenedores	§5.1 Vista de bloques, nivel 1
Nivel 3 — Componentes	§5.2 Vista de bloques, nivel 2
Nivel 4 — Código	§5.3 Vista de bloques, nivel 3
Diagramas dinámicos (secuencia, estados)	§6 Vista de ejecución
Diagrama de despliegue	§7 Vista de despliegue

Tabla 4 — Mapeo entre niveles C4 y secciones arc42

> **Documentación como código**
> Todos los diagramas de este documento se generaron a partir de **código Mermaid** (diagrams-as-code) versionado junto al proyecto; las fuentes completas están en el **Anexo A**. Si cambia la arquitectura, se edita el texto del diagrama y se vuelve a generar la imagen, evitando imágenes “dibujadas a mano” que quedan desactualizadas al primer cambio.

1. Introducción y objetivos

1.1 Propósito del sistema

AmongLies es una **plataforma web de minijuegos sociales multijugador en tiempo real**, pensada para que un grupo de amigos pueda juntarse a jugar desde el navegador —en la computadora o en el celular— sin instalar nada y sin la obligación de registrarse. Se inspira en juegos como *Among Us*, *Spyfall* y *Gartic Phone*: juegos cortos de deducción, donde lo divertido es la interacción entre las personas. Los tres juegos de la plataforma comparten la misma idea: hay uno o más **impostores** escondidos y el grupo tiene que descubrirlos votando.

El problema que resuelve es concreto: organizar una partida de este tipo de juegos entre amigos suele requerir papel, un moderador que no juega, o aplicaciones que obligan a todos a crear una cuenta antes de empezar. AmongLies automatiza el rol de moderador (reparte roles secretos, controla turnos y tiempos, cuenta votos) y reduce la entrada a **elegir un apodo y compartir un código**.

1.2 Objetivos del proyecto

Los objetivos están formulados de forma verificable, para poder evaluar si la arquitectura los cumple. La última columna indica cómo se verificó cada uno en el sistema construido:

ID	Objetivo	Cómo se mide / verificó
O1	**Jugar sin fricción**: cualquier persona puede pasar de abrir el link a estar dentro de una sala en menos de 30 segundos, sin registrarse.	Escenario QS-5. Se entra con apodo, avatar y código o link; la cuenta es opcional.
O2	**Experiencia en tiempo real**: todos los jugadores de una sala ven el mismo estado del juego prácticamente al mismo tiempo.	QS-1. Partidas de 4 navegadores contra producción; el dibujo se transmite trazo a trazo.
O3	**Juego justo**: ningún jugador puede ver información que no le corresponde ni hacer trampa manipulando el cliente.	QS-3 y QS-4. Tests unitarios ("el impostor nunca recibe la palabra") y suite e2e `servidor`.
O4	**Catálogo ampliable**: tres minijuegos y la posibilidad de sumar otros sin reescribir salas, chat ni conexión.	QS-7. Impostor por tiempo e Impostor dibujo se agregaron sin tocar salas, chat ni transporte.
O5	**Progreso persistente opcional**: quien cree una cuenta conserva perfil, estadísticas, amigos y su colección del workshop.	RF-14 a RF-16 y RF-18 a RF-20 implementados y probados de punta a punta con cuentas temporales.
O6	**Accesible y multi-idioma**: interfaz usable desde 360 px de ancho, en español e inglés.	QS-6. Pruebas e2e en un celular táctil; ninguna página tiene scroll horizontal a 360 px.
O7	**Costo operativo mínimo**: operar dentro de planes gratuitos durante la etapa académica.	Render (plan gratis) + Supabase (plan gratis) + GitHub Actions: USD 0 por mes.

Tabla 5 — Objetivos del proyecto

1.3 Requerimientos funcionales principales

Todos los requerimientos están **implementados** en la versión final. Los marcados como *nuevo* no estaban en el plan de la versión 2.0.

ID	Requerimiento	Actor	Estado
RF-01	Ingresar con apodo (2–16 caracteres) y avatar (24 opciones), recordados en el navegador.	Todos	Hecho
RF-02	Crear una sala con código único de 6 caracteres y link para compartir; privada por defecto.	Todos	Hecho
RF-03	Unirse a una sala por código, abriendo el link directo o desde una invitación.	Todos	Hecho

ID	Requerimiento	Actor	Estado
RF-04	Chat de sala en tiempo real (lobby y juego), con panel flotante en el celular y censura de insultos.	Todos	Hecho
RF-05	Elegir minijuego y configurar sus reglas (partidas, impostores, tiempos, voto secreto, desempate, listas de palabras, modo chat/voz…).	Anfitrión	Hecho
RF-06	Expulsar jugadores y transferir el rol de anfitrión; si el anfitrión se desconecta, decide el jugador conectado más antiguo.	Anfitrión	Hecho
RF-07	Jugar **El Impostor**: roles secretos, pistas por turnos, discusión, votación, resultados, partidas y rondas, puntaje.	Jugadores	Hecho
RF-08	Modo voz: la app controla turnos y tiempos; la conversación ocurre fuera (presencial o por llamada).	Jugadores	Hecho
RF-09	Interfaz en español e inglés; listas de palabras según el idioma de la sala.	Todos	Hecho
RF-10	Jugar **Impostor por tiempo**: parar un reloj oculto lo más cerca posible de un tiempo que el impostor no conoce.	Jugadores	Hecho (nuevo)
RF-11	Jugar **Impostor dibujo**: todos dibujan la palabra secreta en un lienzo compartido con tinta limitada; el impostor no la conoce.	Jugadores	Hecho (nuevo)
RF-12	Marcar una sala como **pública**, definir cupo e idioma, y listarla en la página de inicio.	Anfitrión	Hecho
RF-13	Ver salas públicas filtrando por juego e idioma, y unirse con un clic.	Todos	Hecho
RF-14	Registrarse e iniciar sesión con email y contraseña; recuperar la contraseña. La cuenta es **opcional**.	Invitado	Hecho
RF-15	Perfil con nombre de usuario único y avatar.	Registrado	Hecho
RF-16	Estadísticas por juego y por rol (partidas, victorias como impostor e inocente, votos acertados, puntos).	Registrado	Hecho
RF-17	Reconexión: si se corta la conexión o se recarga la página, volver a la misma sala y partida; la partida se pausa mientras tanto.	Todos	Hecho
RF-18	**Amigos**: solicitudes, presencia en vivo (en línea, en una sala, jugando) e invitaciones a la sala.	Registrado	Hecho (nuevo)
RF-19	**Workshop**: crear, publicar, buscar, dar like y guardar listas de palabras y presets de reglas; usarlos en el lobby.	Registrado	Hecho (nuevo)
RF-20	**Moderación**: reportar jugadores y contenido; panel de admin para resolver reportes, ocultar contenido y suspender cuentas.	Todos / Admin	Hecho (nuevo)
RF-21	**Ajustes**: idioma, efectos y música, avisos, privacidad, cambio de email y contraseña, borrar la cuenta.	Todos	Hecho (nuevo)

Tabla 6 — Requerimientos funcionales y estado de implementación

1.4 Objetivos de calidad

De todos los atributos de calidad posibles, el equipo priorizó cinco. El orden importa: cuando dos objetivos entran en conflicto, gana el de mayor prioridad. Cada uno se detalla con escenarios medibles en la sección 10.

Prioridad	Atributo	Motivación
1	**Rendimiento en tiempo real**	Un juego social se arruina si un jugador ve el turno o el voto con segundos de retraso respecto de los demás.
2	**Integridad del juego** (seguridad)	Todo el juego depende de información oculta. Si el impostor puede leer la palabra desde las herramientas del navegador, el juego deja de tener sentido.
3	**Usabilidad / baja fricción**	El público es casual. Cada paso extra antes de jugar (registro, instalación) reduce cuántas personas llegan a jugar.
4	**Extensibilidad**	El producto es una plataforma de varios minijuegos; agregar uno nuevo debe ser barato.
5	**Disponibilidad y escalabilidad**	Soportar el crecimiento de salas simultáneas y reinicios del servidor sin cortar partidas.

Tabla 7 — Objetivos de calidad priorizados

1.5 Partes interesadas

Rol	Expectativa sobre el sistema	Qué busca en este documento
Jugador casual	Entrar rápido, que funcione en el celular, que sea justo y divertido.	Parte II: cómo se juega.
Anfitrión de sala	Controlar la sala: configurar, moderar, invitar.	Parte II: permisos y configuración.
Usuario registrado	Conservar su progreso, sus amigos y lo que crea en el workshop.	Parte II: perfil, amigos y workshop.
Administrador	Mantener la comunidad sana: revisar reportes y suspender a quien abusa.	Parte II: moderación (M14).
Equipo de desarrollo	Entender la estructura para mantenerla y extenderla; incorporar integrantes.	Secciones 4 a 9 y 11.
Docente / evaluador	Verificar que las decisiones estén justificadas con criterios.	Secciones 1, 3, 4, 9 y 10.
Operador del despliegue	Saber qué se despliega, dónde y con qué configuración.	Sección 7.

Tabla 8 — Partes interesadas

2. Restricciones

Las restricciones son condiciones que el equipo no eligió libremente y que limitan el espacio de soluciones. Muchas de las decisiones de la sección 9 se explican por ellas.

2.1 Restricciones técnicas

ID	Restricción	Consecuencia en la arquitectura
RT-1	Debe funcionar en el **navegador**, en desktop y móvil, sin instalar aplicaciones.	Aplicación web responsive; nada de clientes nativos ni plugins.
RT-2	La comunicación en tiempo real requiere **conexiones persistentes** (WebSockets).	El servidor de juego debe ser un proceso de larga duración: no puede desplegarse en plataformas serverless como Vercel Functions.
RT-3	**TypeScript** en todo el stack (conocimiento del equipo).	Node.js en el servidor; tipos compartidos entre cliente y servidor.
RT-4	Infraestructura dentro de **planes gratuitos**.	Servicios gestionados (Render, Supabase). En el plan gratis de Render el servidor se duerme tras 15 minutos sin tráfico: la aplicación lo tiene en cuenta (§8.8).
RT-5	Sin servidor de audio/video propio.	El modo voz sólo coordina turnos; la conversación ocurre en otra app o en persona.

Tabla 9 — Restricciones técnicas

2.2 Restricciones organizativas

ID	Restricción	Consecuencia
RO-1	Equipo reducido de estudiantes con dedicación parcial.	Preferir frameworks con convenciones fuertes (NestJS, Next.js) y un único repositorio.
RO-2	Plazos fijados por el calendario de la materia; entregas incrementales (MVP → versión final).	Arquitectura que crezca sin reescrituras: motores enchufables y tipos compartidos desde el MVP. La versión final se construyó en 9 fases, cada una verificada antes de la siguiente.
RO-3	Control de versiones en GitHub; nada se integra a `main` sin pasar los controles.	Hook pre-commit (secretos, lint, tests) y CI en cada push; despliegue automático desde `main`.

Tabla 10 — Restricciones organizativas

2.3 Convenciones

- Idioma de la interfaz por defecto: español rioplatense; código y nombres técnicos en inglés.
- Eventos de tiempo real nombrados `dominio:acción` (por ejemplo `room:join`, `game:action`), definidos una sola vez en el paquete compartido.
- Estilo visual “Neon Noir” (fondo azul muy oscuro, violeta como color primario, rosa para el impostor) definido como variables de diseño.

2.4 Restricciones legales y éticas

- **Ley 25.326 de Protección de Datos Personales (Argentina)**: se minimizan los datos recolectados. Los invitados no dejan datos personales; los usuarios registrados sólo entregan un email (gestionado por Supabase Auth) y un nombre de usuario público. Cada usuario puede **borrar su cuenta** y todo lo asociado desde Ajustes.

- El chat no se guarda: vive en memoria mientras exista la sala. La única excepción es un **reporte**: se adjuntan como evidencia los últimos 20 mensajes de la sala, que sólo ven los administradores.
- Contenido apto para todo público: las listas de palabras propias son revisadas por el filtro de insultos, y el contenido del workshop se puede reportar y ocultar.

3. Contexto y alcance

3.1 Contexto de negocio (C4 — Nivel 1)

El diagrama de contexto muestra el sistema como una caja negra: quiénes lo usan y con qué sistemas externos se relaciona. Es la vista pensada para personas no técnicas.

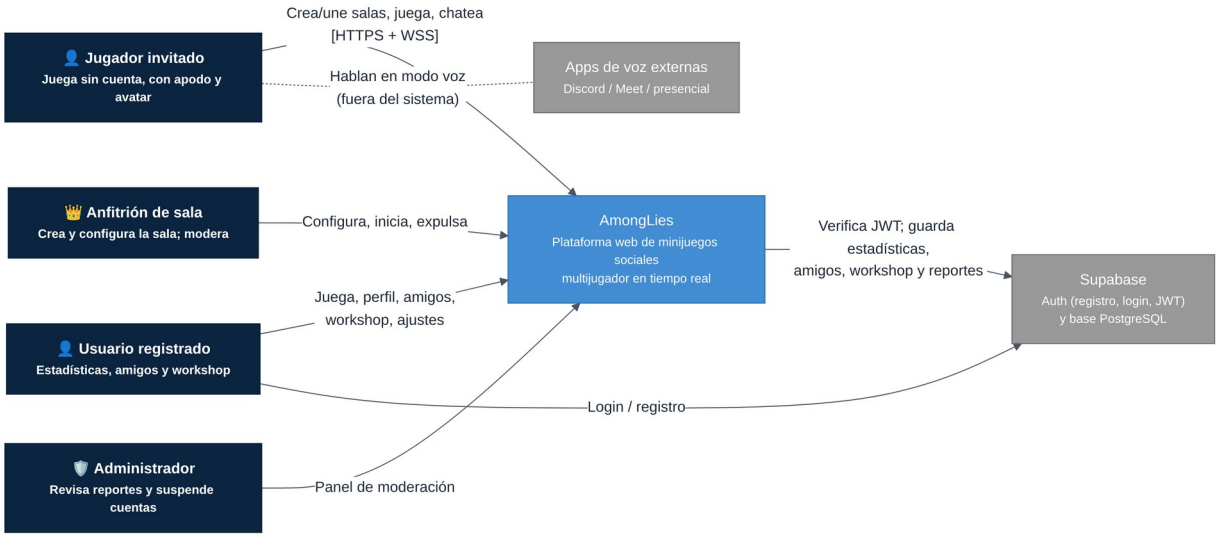


Figura 1 — C4 Nivel 1: contexto del sistema AmongLies

Actor / sistema	Tipo	Qué intercambia con AmongLies
Jugador invitado	Persona	Apodo, avatar, código de sala, mensajes de chat, pistas, trazos, votos. Recibe el estado del juego.
Usuario registrado	Persona	Lo mismo que el invitado, más credenciales; recibe perfil, estadísticas, amigos y su colección del workshop.
Anfitrión de sala	Rol de persona	Configuración de la partida, órdenes de iniciar, expulsar y transferir el rol.
Administrador	Rol de persona	Revisa reportes, oculta contenido del workshop y suspende cuentas desde el panel de admin.
Supabase	Sistema externo	Auth: registro, login, recuperación de contraseña y emisión de JWT. PostgreSQL: perfiles, estadísticas, amistades, workshop y reportes, con RLS.
Apps de voz externas	Fuera del sistema	En modo voz los jugadores hablan por Discord, Meet o en persona; AmongLies no transmite audio.

Tabla 11 — Interfaces del contexto de negocio

3.2 Contexto técnico

Canal	Protocolo	Formato	Uso
Navegador → Aplicación Web	HTTPS	HTML/JS	Carga de la aplicación.
Navegador ↔ Game Server	Socket.io (HTTP long-polling que sube a WebSocket)	Eventos JSON tipados	Salas, chat, juego, amigos, workshop y moderación.
Navegador → Game Server	HTTPS REST	JSON	`GET /rooms/public` (salas públicas) y `GET /health` (mantener despierto el servidor).

Canal	Protocolo	Formato	Uso
Navegador → Supabase	HTTPS	JSON (supabase-js)	Login y registro; lectura de perfiles, estadísticas y workshop (protegida por RLS).
Game Server → Supabase Auth	HTTPS	JWKS	Claves públicas para verificar los JWT de las cuentas.
Game Server → Supabase PostgreSQL	HTTPS (API REST)	JSON	Escritura de estadísticas, amistades, workshop y reportes con la clave secreta del servidor.

Tabla 12 — Contexto técnico: canales y protocolos

3.3 Alcance

Definir qué queda **fuera** es tan importante como definir qué entra: evita que el proyecto crezca sin control y explica por qué ciertas funciones no existen.

Dentro del alcance (versión final)	Fuera del alcance
Salas privadas (código, link, invitación) y públicas (listado en la página de inicio) con cupo configurable.	Matchmaking automático con desconocidos por nivel o ranking.
Tres minijuegos de deducción: El Impostor, Impostor por tiempo e Impostor dibujo.	Juegos de otro tipo (por ejemplo, de dibujo y adivinanza sin impostor).
Cuentas opcionales, perfil y estadísticas por juego.	Historial detallado partida por partida; rankings globales.
Amigos con presencia e invitaciones a salas.	Mensajes privados entre usuarios; notificaciones push.
Workshop de listas de palabras y presets de reglas hechos por los usuarios.	Editor de minijuegos o de estética (temas, avatares propios).
Moderación: filtro de insultos, reportes, panel de admin y suspensiones.	Moderación automática con inteligencia artificial.
Chat de sala en tiempo real; sonido sintetizado en el navegador.	Chat de voz o video integrado (se usa una app externa).
Español e inglés. Aplicación web responsive.	Otros idiomas; aplicaciones nativas; monetización.

Tabla 13 — Alcance de la versión final

Evolución del producto

Versión	Contenido	Arquitectura de datos
v1.0 — MVP (entregado)	Salas privadas, lobby, chat, El Impostor completo (chat y voz), i18n base, responsive.	Todo en memoria del proceso servidor; una sola instancia.
v2.0 — Plan de la versión final	Dos minijuegos nuevos, salas públicas, cuentas, estadísticas e historial, reconexión.	Planeado: Redis para el estado vivo y varias réplicas; PostgreSQL para lo persistente.
v3.0 — Versión final (este documento)	Tres juegos de la familia Impostor, reconexión con pausa, reglas configurables y puntaje, cuentas y estadísticas, amigos, workshop, moderación, ajustes, sonido, salas públicas, pruebas e2e.	Estado vivo en memoria de una instancia (sin Redis, ADR-05). Supabase (PostgreSQL + Auth) para lo persistente.

Tabla 14 — Del MVP a la versión final

4. Estrategia de solución

4.1 Estilo arquitectónico

AmongLies combina tres estilos, cada uno elegido por un objetivo concreto:

- **Cliente–servidor con servidor autoritativo.** El servidor es la única fuente de verdad del estado del juego. Los clientes nunca modifican el estado: sólo envían *intenciones* ("quiero votar a Sofi"), que el servidor valida contra las reglas antes de aplicarlas. Esto responde al objetivo de integridad (O3).
- **Arquitectura orientada a eventos.** Cliente y servidor se comunican con eventos con nombre sobre una conexión persistente. El servidor puede *empujar* cambios a todos los jugadores de una sala en el instante en que ocurren. Responde al objetivo de tiempo real (O2).
- **Monolito modular.** El servidor es una sola aplicación desplegable, dividida en módulos por dominio (salas, chat, juego, jugadores, estadísticas, amigos, workshop, moderación) con límites claros. Para un equipo chico da la organización de los microservicios sin su costo operativo (ADR-03).

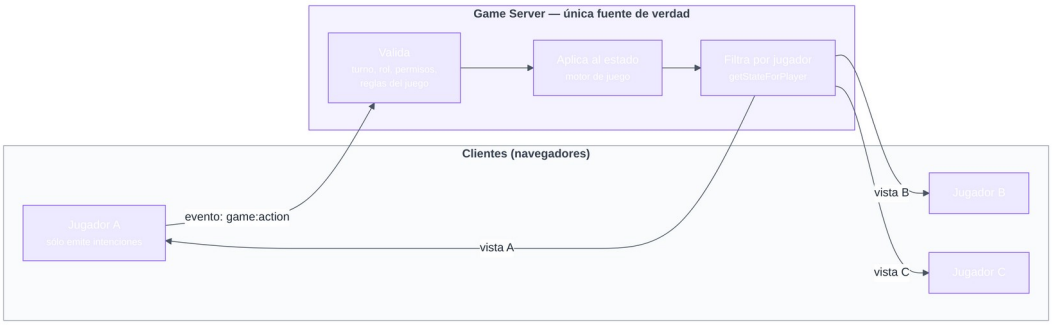


Figura 2 — Servidor autoritativo: los clientes emiten intenciones y reciben vistas filtradas

4.2 Enfoques por objetivo

Objetivo	Enfoque de solución	Detalle
O1 Sin fricción	Identidad de invitado por defecto; cuenta opcional. Salas por código de 6 caracteres sin letras ambiguas, link e invitaciones de amigos.	§8.6, ADR-07
O2 Tiempo real	Socket.io; difusión por sala; estado vivo en memoria, nunca en la base relacional; trazos del dibujo como eventos livianos.	ADR-01, ADR-05
O3 Juego justo	Servidor autoritativo; vista por jugador (`getStateForPlayer`); validación de turno, rol, permisos y tinta en el servidor.	§8.4, ADR-02, ADR-09
O4 Catálogo ampliable	Interfaz `GameEngine` + registro de juegos; `DeductionEngine` con lo común de la familia Impostor.	§5.3, ADR-08
O5 Progreso persistente	Supabase (Auth + PostgreSQL) con Row Level Security; todo lo que cuenta lo escribe sólo el servidor.	§8.2, ADR-06, ADR-15
O6 Accesible y multi-idioma	Diseño mobile-first con Tailwind; diccionarios es/en y listas de palabras por idioma.	§8.7, §8.12
O7 Costo mínimo	Planes gratuitos de Render y Supabase; un monolito en lugar de muchos servicios.	§7, ADR-13

Tabla 15 — Estrategia de solución por objetivo

4.3 De la tecnología al impacto para el usuario

Cada elección técnica tiene sentido sólo si produce un beneficio observable. La siguiente tabla traduce las decisiones principales a lo que percibe quien juega:

Decisión técnica	Impacto para el usuario
WebSockets con difusión por sala.	Cuando alguien vota o dibuja, todos lo ven al instante; no hay que recargar la página.
Servidor autoritativo con vistas filtradas por jugador.	Nadie puede descubrir al impostor ni la palabra "espiando" el navegador: el dato nunca llega a su dispositivo.
Identidad de invitado + código de sala.	Entrar a jugar toma segundos: apodo, avatar y código.
Reconexión con pausa de la partida.	Si a alguien se le corta internet o recarga la página, vuelve a su lugar y la partida lo espera.
Motores de juego enchufables con una base común.	Los tres juegos se juegan igual (roles, discusión, votación, puntaje): aprender uno es aprender los tres.
Supabase Auth + RLS.	Tus estadísticas, tu colección y tus amigos son tuyos; nadie puede modificar lo de otro.

Tabla 16 — Traducción de decisiones técnicas a impacto

5. Vista de bloques de construcción

Esta sección descompone el sistema de lo general a lo particular. Cada nivel “abre” una caja del nivel anterior.

5.1 Nivel 1 — Contenedores (C4 — Nivel 2)

Un contenedor es una unidad que se ejecuta o almacena datos por separado. AmongLies tiene tres contenedores propios y un paquete compartido que no se ejecuta solo, sino que se compila dentro de los otros.

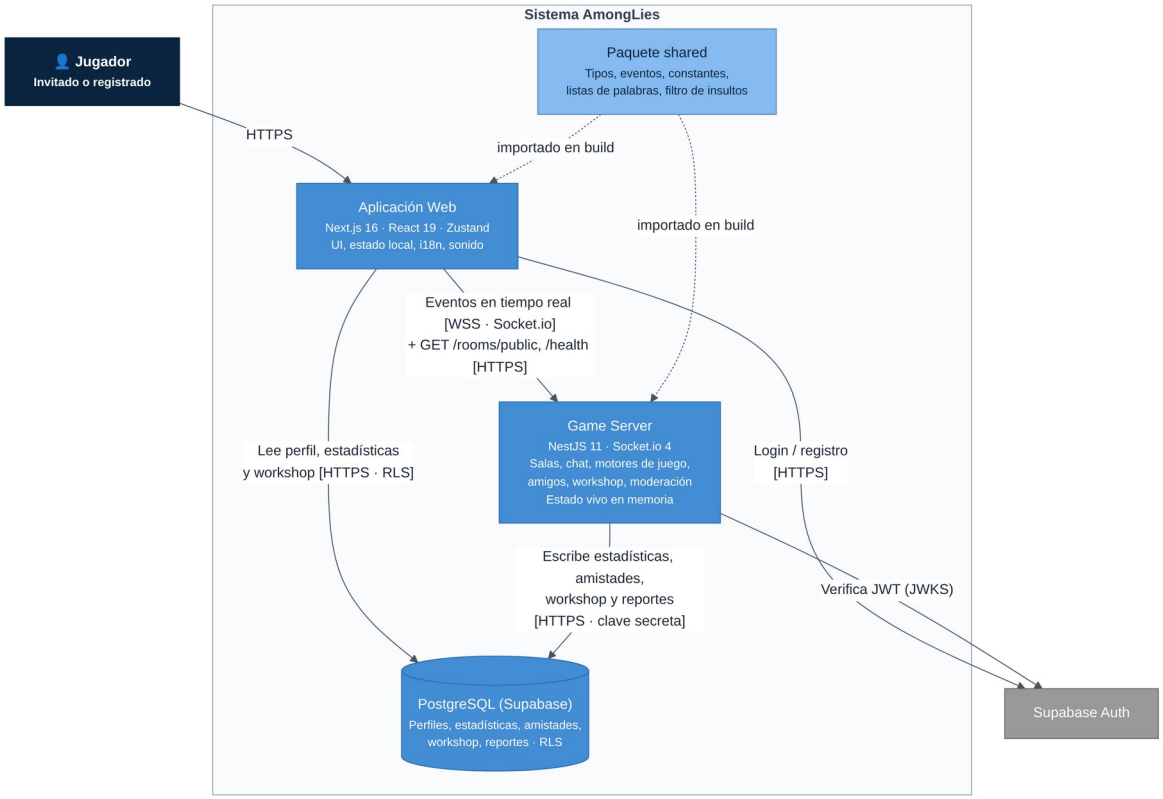


Figura 3 — C4 Nivel 2: contenedores de AmongLies

Contenedor	Tecnología	Responsabilidad	Por qué existe separado
Aplicación Web	Next.js 16 (App Router), React 19, Tailwind 4, Framer Motion, Zustand	Interfaz, animaciones, estado local, idioma, sonido sintetizado, conexión con el servidor y lecturas de Supabase.	Ciclo de despliegue distinto del servidor de juego; sirve contenido estático cacheable.
Game Server	NestJS 11, Socket.io 4, Node 22	Salas, chat, sesiones y reconexión, motores de juego y temporizadores, amigos, workshop, moderación, salas públicas, registro de estadísticas. **El estado vivo está en su memoria.**	Proceso de larga duración que mantiene conexiones persistentes (RT-2).
PostgreSQL + Auth	Supabase (PostgreSQL 17 + Auth)	Cuentas, perfiles, estadísticas, amistades, workshop y reportes.	Datos que deben sobrevivir y consultarse con relaciones y agregaciones.
Paquete shared	TypeScript	Tipos de dominio, contrato de eventos, definiciones de juegos, puntaje, listas de palabras, filtro de insultos y cálculo de tinta.	Una única fuente de verdad del contrato cliente–servidor y de la lógica que corren los dos lados.

Tabla 17 — Contenedores y responsabilidades

5.2 Nivel 2 — Componentes (C4 — Nivel 3)

Componentes del Game Server

El servidor se organiza en módulos de NestJS. La **capa de transporte** (gateways y controladores) traduce eventos de red en llamadas y verifica permisos; la **capa de dominio** contiene las reglas. Las reglas del juego nunca dependen de Socket.io, lo que permite probarlas de forma aislada con timers simulados.

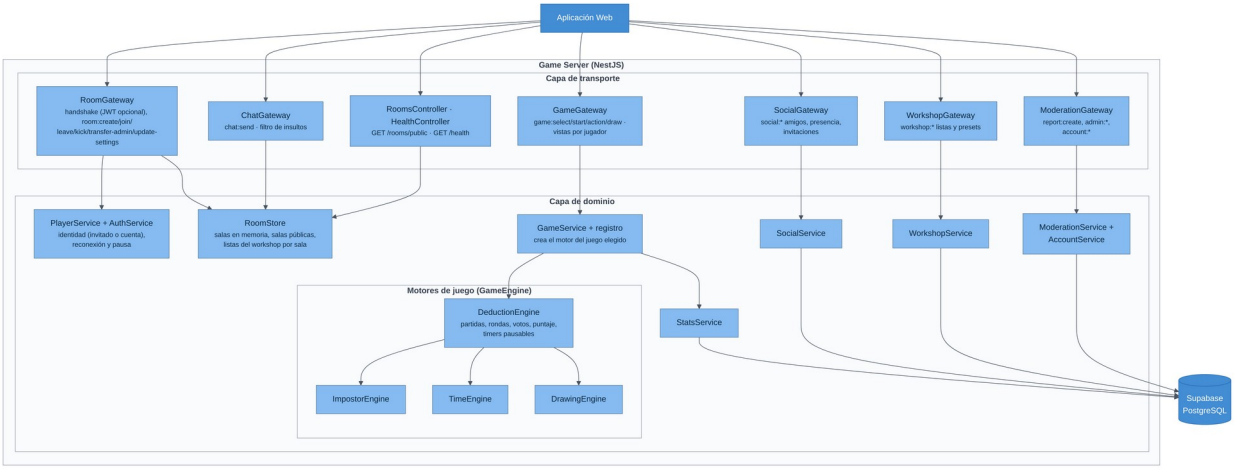


Figura 4 — C4 Nivel 3: componentes del Game Server

Componente	Responsabilidad
RoomGateway	Conexión: identidad de invitado (sessionToken) o de cuenta (JWT verificado con el JWKS de Supabase), reconexión y pausa. Eventos de sala: crear, unirse, salir, expulsar, transferir anfitrión, cambiar ajustes de la sala. Rechaza apodos con insultos y cuentas suspendidas.
ChatGateway	Recibe mensajes, recorta a 200 caracteres, censura insultos con `***` y los difunde a la sala. Rechaza mensajes de eliminados durante la partida.
GameGateway + GameService	Selección de juego, ajustes, inicio, acciones y trazos del dibujo; crea el motor del juego elegido y **difunde una vista distinta a cada jugador**. Resuelve las listas de palabras (del juego o del workshop).
Motores de juego	`DeductionEngine` (partidas, rondas, turnos, votación, desempates, puntaje, pausa) y sus tres variantes: `ImpostorEngine`, `TimeEngine`, `DrawingEngine`.
PlayerService / AuthService	Vínculo socket ↔ jugador ↔ sala; identidad estable entre reconexiones; una segunda pestaña de la misma cuenta reemplaza a la primera.
RoomStore	Salas en memoria, listado de salas públicas y listas del workshop cargadas en cada sala. Limpia las salas abandonadas.
RoomsController / HealthController	`GET /rooms/public` (salas públicas) y `GET /health` (los clientes en una sala lo consultan para que el servidor no se duerma).
SocialGateway + SocialService	Solicitudes de amistad, presencia (cada 2 s avisa los cambios) e invitaciones a la sala.
WorkshopGateway + WorkshopService	Crear, editar, publicar, copiar, actualizar copias y dar like; valida listas (10 a 300 palabras) y presets.
ModerationGateway + ModerationService + AccountService	Reportes con evidencia del chat, acciones del panel de admin (suspender, ocultar, resolver), marcas de la cuenta (admin, suspensión, privacidad) y borrado de cuenta.
StatsService	Al terminar un juego, suma las estadísticas de los jugadores con cuenta (función `record_game_stats` de la base).

Tabla 18 — Componentes del Game Server

Componentes de la Aplicación Web

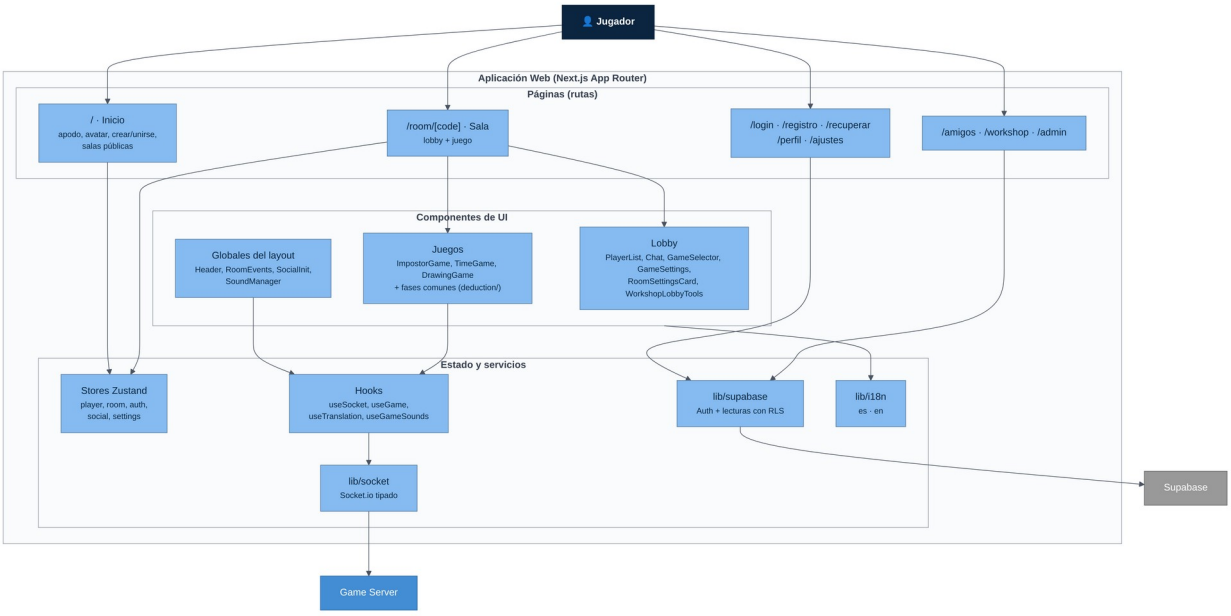


Figura 5 — C4 Nivel 3: componentes de la Aplicación Web

La aplicación web mantiene muy poco estado propio: el estado del juego llega del servidor y se guarda tal cual en los stores. `playerStore` guarda la identidad local (apodo, avatar, idioma); `roomStore`, la sala y el chat; `authStore`, la sesión de Supabase y el perfil; `socialStore`, amigos y solicitudes; `settingsStore`, sonido y avisos. Los eventos de la sala se escuchan en un componente global del layout (`RoomEvents`) para no perder mensajes mientras se navega hacia la sala. El socket es un *singleton* tipado con los mismos tipos que usa el servidor, de modo que un evento mal escrito no compila.

5.3 Nivel 3 — Código (C4 — Nivel 4): el contrato de los motores de juego

El punto de extensión más importante de la arquitectura es la interfaz `GameEngine`. Cada minijuego la implementa; el resto del sistema (salas, chat, transporte, estadísticas) sólo conoce la interfaz. Como los tres juegos son de deducción, lo común vive en la clase abstracta `DeductionEngine`, y cada juego sólo define la **actividad de la ronda**: dar pistas, medir un tiempo o dibujar.

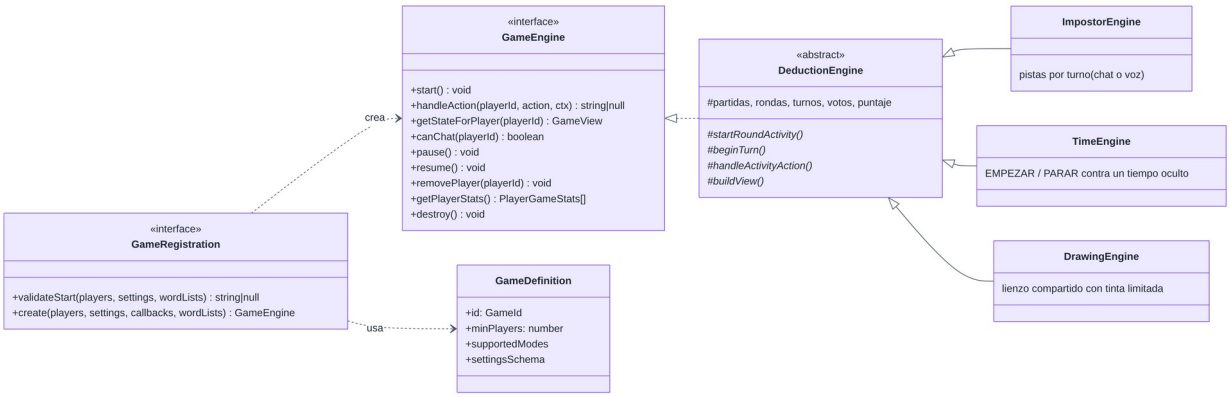


Figura 6 — C4 Nivel 4: interfaz GameEngine, motor base de deducción y juegos concretos

Con este contrato, agregar un minijuego nuevo requiere solamente:

1. Definir su `GameDefinition` (nombre, jugadores mínimos, modos y esquema de ajustes) y sus tipos de vista en `packages/shared`.

2. Implementar su motor: si es de deducción, extender `DeductionEngine` con los métodos de la actividad (`startRoundActivity`, `beginTurn`, `handleActivityAction`, `buildView`).
3. Registrarlo en `game.registry.ts`.
4. Crear sus componentes de interfaz en `apps/web/src/components/<juego>/` (las fases comunes ya están en `components/deduction/`).

El lobby lista automáticamente los juegos registrados y genera los controles de configuración a partir del `settingsSchema`; el servidor valida los ajustes contra el mismo esquema. Así se agregaron Impostor por tiempo e Impostor dibujo sin modificar salas, chat ni transporte (QS-7).

5.4 Estructura del repositorio

```
AmongLies/                        monorepo (npm workspaces + Turborepo)
├── apps/
│   ├── web/                      Aplicación Web (Next.js)
│   │   └── src/
│   │       ├── app/              /, /room/[code], /login, /registro, /recuperar, /perfil,
│   │       │                     /amigos, /workshop, /ajustes, /admin
│   │       ├── components/       lobby/, deduction/, impostor/, time/, drawing/, social/,
│   │       │                     workshop/, moderation/, sound/, layout/, ui/
│   │       ├── hooks/            useSocket, useGame, useTranslation, useGameSounds
│   │       ├── stores/           player, room, auth, social, settings
│   │       └── lib/              socket, supabase, i18n, sound, keepAlive, workshop, admin
│   └── server/                   Game Server (NestJS)
│       └── src/
│           ├── room/             gateway, store, validación, salas públicas
│           ├── chat/  player/  auth/  stats/
│           ├── game/             gateway, service, registry, settings, engines/
│           │                     (core, deduction, impostor, time, drawing)
│           ├── social/  workshop/  moderation/
│           └── health.controller.ts
├── packages/shared/              tipos, eventos, juegos, puntaje, listas de palabras,
│                                 filtro de insultos, tinta
├── supabase/migrations/          esquema de la base (con RLS)
├── e2e/                          pruebas de punta a punta (Playwright + bots)
├── docs/                         progreso, testing y esta documentación
└── turbo.json                    orden de build: shared → server, web
```

6. Vista de ejecución

Esta sección muestra cómo colaboran los bloques en los escenarios más importantes. Los diagramas de secuencia se leen de arriba hacia abajo; las flechas punteadas son respuestas.

6.1 Escenario 1 — Crear una sala y unirse

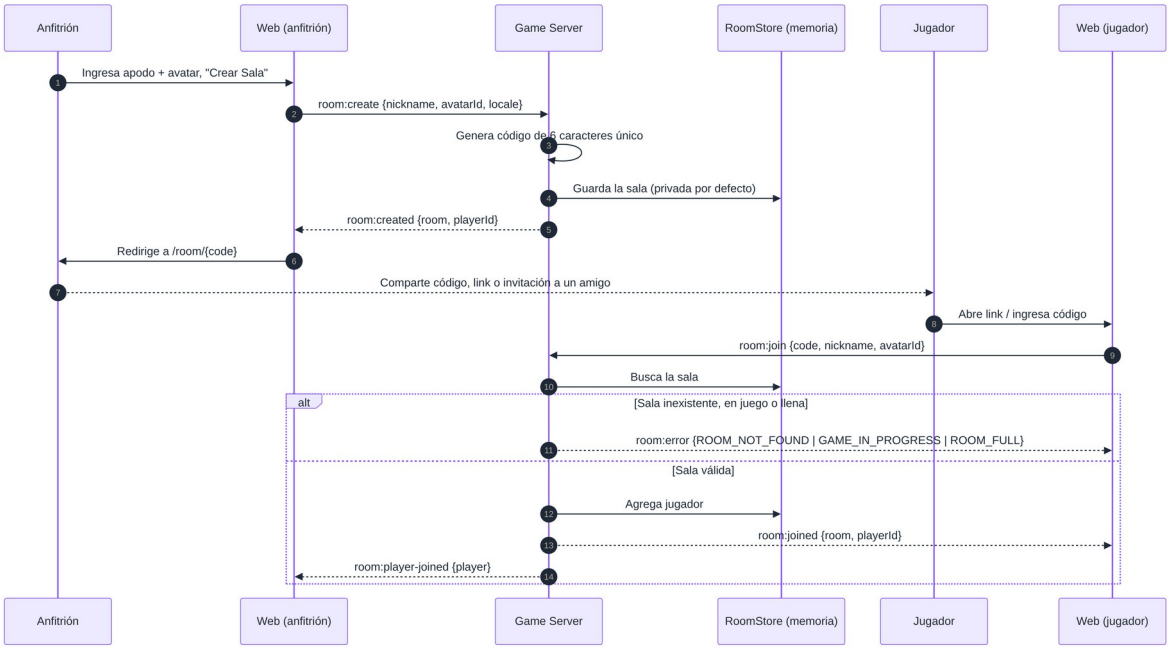


Figura 7 — Secuencia: creación de sala y unión de un jugador

Puntos de diseño: el servidor genera el código con un alfabeto sin caracteres ambiguos y verifica que no esté en uso; los errores de unión se devuelven con un **código** estable (`ROOM_NOT_FOUND`, `GAME_IN_PROGRESS`, `ROOM_FULL`, `NICKNAME_NOT_ALLOWED`) que el cliente traduce al idioma del usuario. Si el servidor está dormido (plan gratis), la web espera hasta 70 segundos mostrando "Despertando el servidor...".

6.2 Escenario 2 — Una ronda de El Impostor

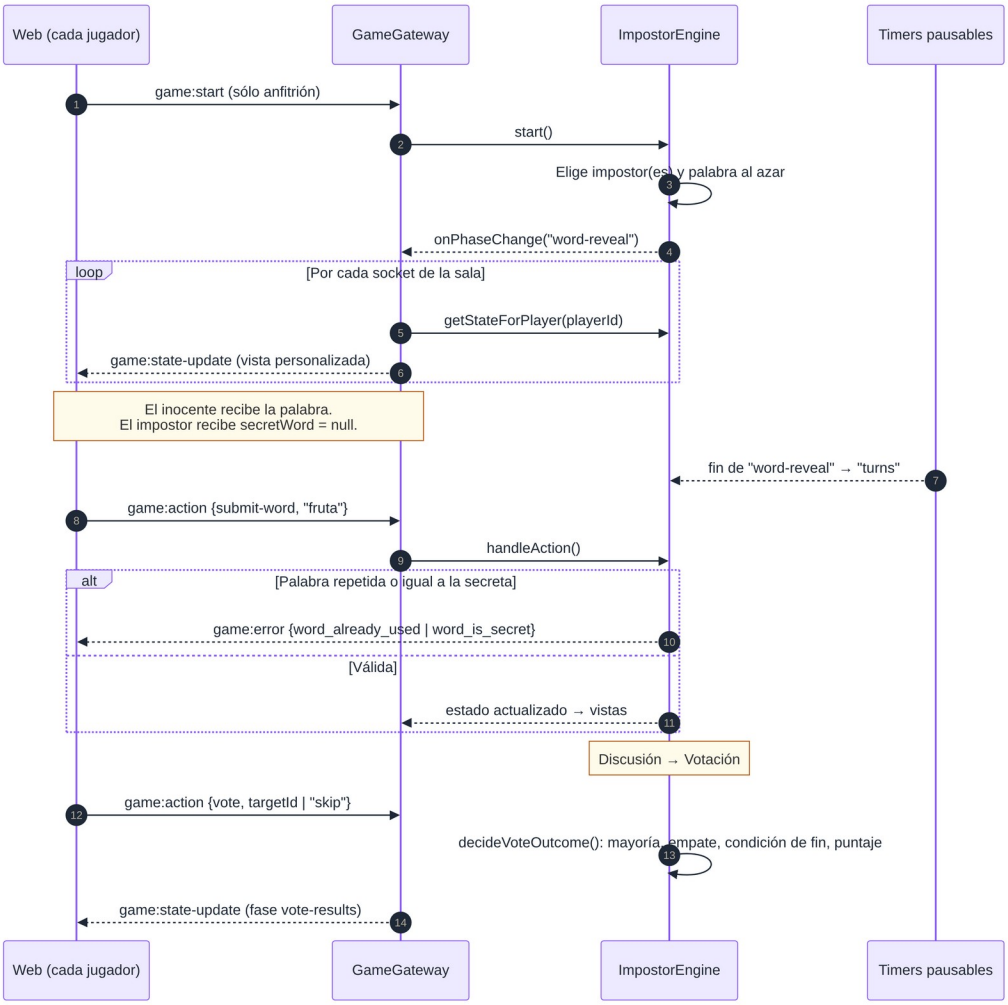


Figura 8 — Secuencia: ronda de El Impostor con servidor autoritativo

Lo central de este escenario es que, en cada cambio, el servidor **no difunde un único estado**: recorre los sockets de la sala y calcula `getStateForPlayer(playerId)` para cada uno. El impostor recibe `secretWord: null` (en el Tiempo, `targetMs: null`); los compañeros impostores sólo se revelan entre sí; los votos individuales se ocultan hasta la fase de resultados (y para siempre si el voto es secreto). En el Impostor dibujo los trazos viajan por un evento aparte (`game:draw`) para no reenviar todo el estado en cada movimiento del pincel.

6.3 Máquina de estados del motor El Impostor

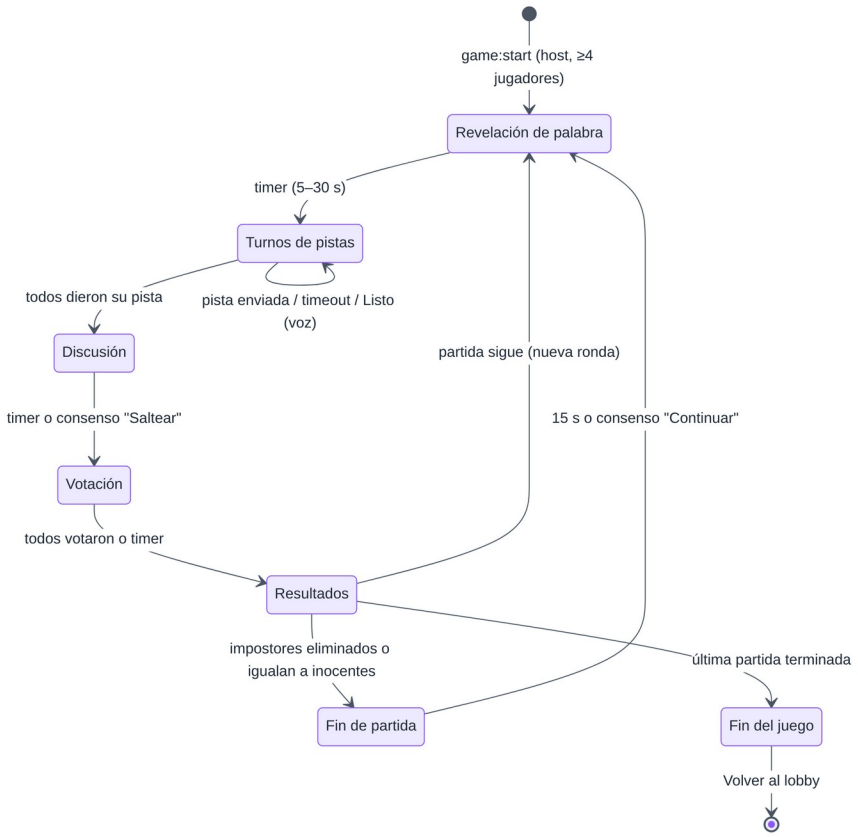


Figura 9 — Estados y transiciones de El Impostor

Estado	Sale por	Condición de salida
Revelación de palabra	Temporizador	`wordRevealTimeSeconds` (5–30 s). En el Tiempo, revelación de rol y del tiempo objetivo.
Turnos	Acción o temporizador	Cada jugador activo envía su pista (chat), toca "Listo" (voz), para el reloj (Tiempo) o termina de dibujar (Dibujo); si se agota el tiempo del turno, se registra.
Discusión	Temporizador o consenso	Termina el tiempo o **todos** los jugadores activos votan "Saltear discusión". Si el tiempo configurado es 0, se omite.
Votación	Todos votaron o temporizador	Los votos no emitidos cuentan como abstención. Se puede votar "saltear" si el anfitrión lo permite.
Resultados	Temporizador	Mayoría simple expulsa. Empate: nadie sale, re-votación entre los empatados o desempate al azar (configurable). Se suma el puntaje y se evalúa el fin de la partida (incluido el tope de rondas).
Fin de partida	15 s o consenso	Se revela el impostor y el puntaje; todos pueden votar "Continuar ya".
Fin del juego	Acción del usuario	Podio, resumen de todas las partidas y vuelta al lobby.

Tabla 19 — Transiciones del motor de deducción (común a los tres juegos)

Si un jugador se desconecta durante la partida, el motor se **pausa**: todos los temporizadores se congelan y se reanudan con el tiempo que les quedaba cuando vuelve (o cuando quien decide toca "Seguir sin esperar").

6.4 Escenario 3 — Fin del juego y registro de estadísticas

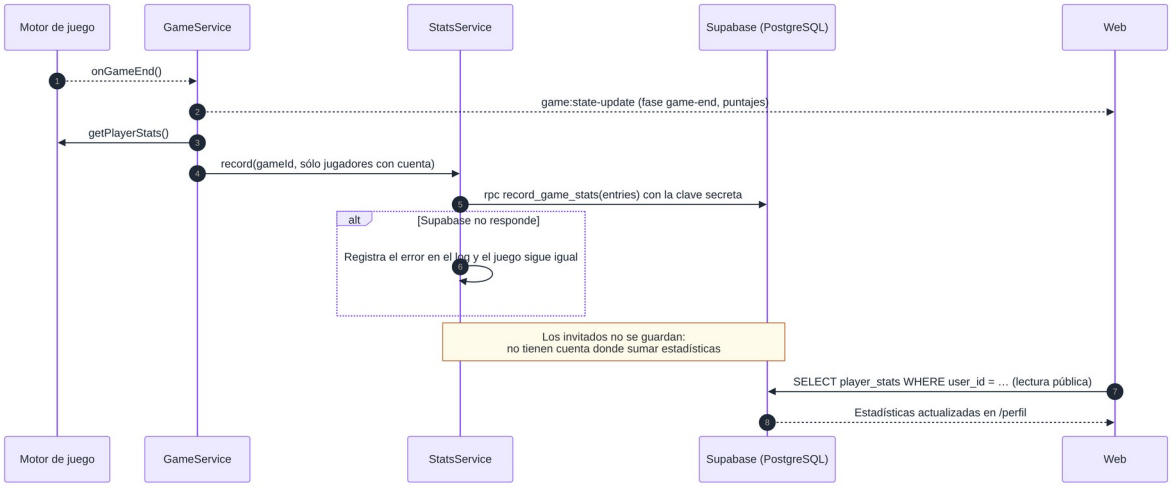


Figura 10 — Secuencia: registro de estadísticas al terminar el juego

La escritura de estadísticas es **asíncrona respecto del juego**: los jugadores ven la pantalla final aunque Supabase no responda. Si la escritura falla se registra el error en el log del servidor y el juego sigue; se aceptó perder esa suma antes que agregar una cola de reintentos (la v2.0 planeaba un *outbox* en Redis, que se descartó junto con Redis).

6.5 Escenario 4 — Inicio de sesión y conexión autenticada

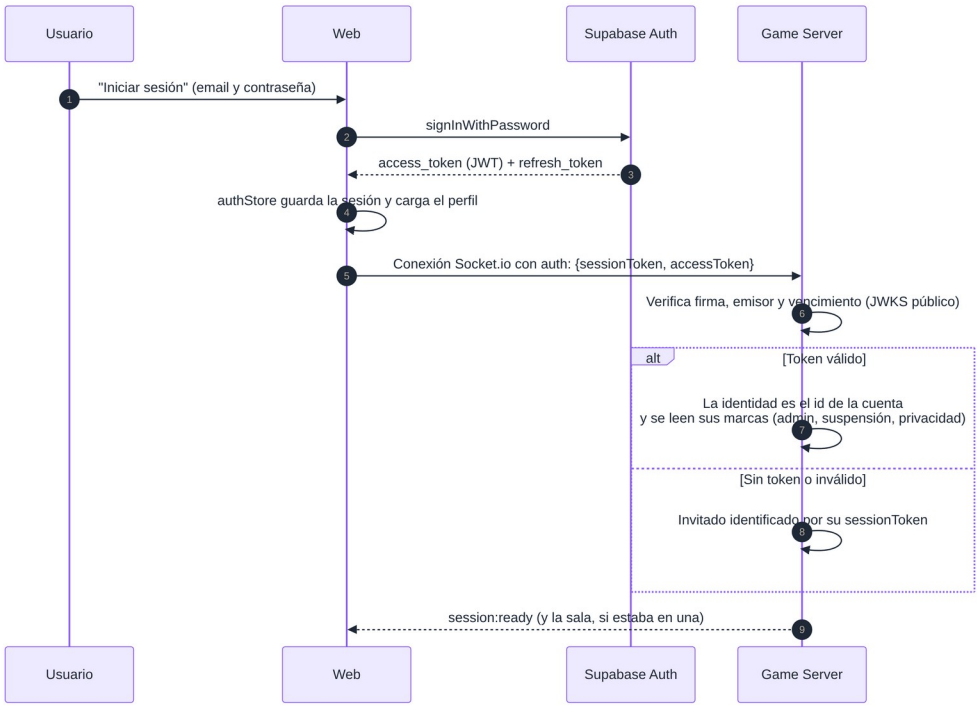


Figura 11 — Secuencia: login con Supabase y conexión autenticada

6.6 Escenario 5 — Desconexión y reconexión

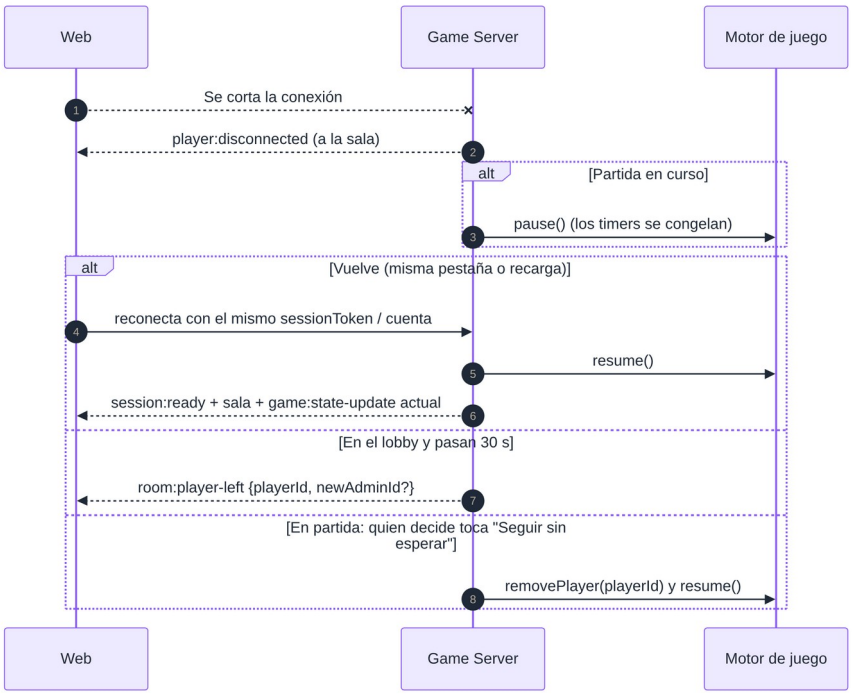


Figura 12 — Secuencia: desconexión, pausa y reconexión

6.7 Catálogo de eventos

Evento	Dirección	Quién puede emitirlo	Descripción
session:ready	S → C	Servidor	Identidad lista (invitado o cuenta); incluye la sala si estaba en una (reconexión).
room:create / room:join	C → S	Cualquiera no suspendido	Crea una sala (el emisor queda como anfitrión) o se une por código. Respuesta `room:created`, `room:joined` o `room:error`.
room:leave / room:kick / room:transfer-admin	C → S	Miembro / anfitrión	Salir, expulsar (el expulsado recibe `room:kicked`) o ceder el rol de anfitrión.
room:update-settings	C → S	Anfitrión	Visibilidad pública/privada, cupo e idioma de la sala.
room:updated / room:player-joined / room:player-left	S → C	Servidor	Cambios de la sala; `player-left` incluye `newAdminId` si se transfirió el rol.
player:disconnected / game:continue-without	S → C / C → S	Servidor / quien decide	Un jugador se desconectó (la partida se pausa); seguir sin esperarlo.
chat:send / chat:message	C ↔ S	Miembro no eliminado	Mensajes de chat (máx. 200 caracteres, insultos censurados).
game:select / game:update-settings / game:start	C → S	Anfitrión	Elegir minijuego, ajustar sus reglas e iniciar.
game:action	C → S	Jugador	`submit-word`, `ready`, `advance`, `start-clock`, `stop-clock`, `finish-drawing`, `skip-discussion`, `vote`, `skip-partida-end`…

Evento	Dirección	Quién puede emitirlo	Descripción
game:draw	C ↔ S	Quien dibuja	Trazos en vivo del Impostor dibujo; el servidor los recorta según la tinta y los reenvía a la sala.
game:state-update / game:error	S → C	Servidor	Vista personalizada del estado para ese jugador; errores con código.
social:*	C ↔ S	Cuentas	Buscar, pedir, aceptar, eliminar amigos; presencia; invitaciones a la sala.
workshop:*	C → S	Cuentas	Crear, editar, publicar, copiar, actualizar, dar like; usar una lista o un preset en el lobby.
report:create / admin: / *account:*	C → S	Todos / admins / cuentas	Reportar; acciones del panel de admin; refrescar o borrar la propia cuenta. `account:suspended` avisa una suspensión.

Tabla 20 — Contrato de eventos en tiempo real

7. Vista de despliegue

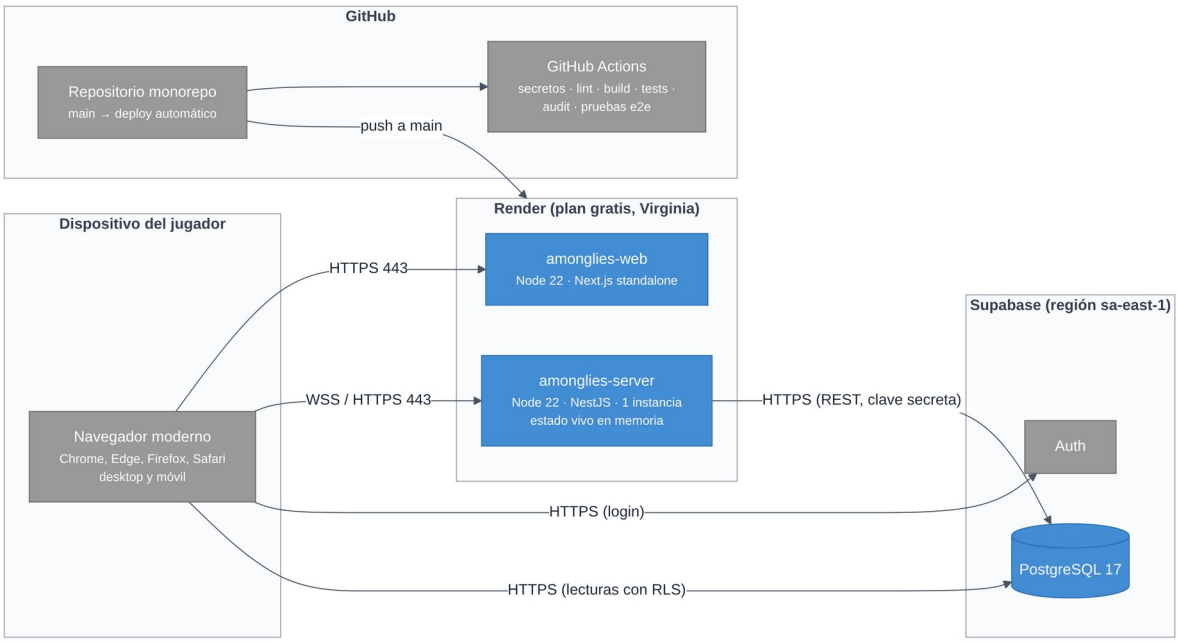


Figura 13 — Despliegue en producción

Nodo	Plataforma	Configuración	Justificación
amonglies-web	Render (Web Service, plan gratis, Virginia)	Node 22; `next build` con salida *standalone*; variables `NEXT_PUBLIC_SERVER_URL` y `NEXT_PUBLIC_SUPABASE_*`.	Deploy automático desde `main`; mismo proveedor que el servidor.
amonglies-server	Render (Web Service, plan gratis, Virginia), 1 instancia	Node 22; `node apps/server/dist/main`; variables `CLIENT_URL`, `SUPABASE_URL`, `SUPABASE_SECRET_KEY`; health check `/health`.	Admite procesos de larga duración y WebSockets (RT-2).
PostgreSQL + Auth	Supabase, región sa-east-1 (São Paulo)	RLS en todas las tablas; la clave secreta sólo en el servidor; 6 migraciones versionadas en el repo.	Auth y base en un mismo servicio con plan gratuito.
CI	GitHub Actions	En cada push y pull request: secretos, lint, build, tests con cobertura, `npm audit` y pruebas de punta a punta.	Evita integrar a `main` código que rompe el contrato o las partidas.

Tabla 21 — Nodos de despliegue

La configuración de los dos servicios de Render está versionada en `render.yaml` (en la raíz del repositorio); las claves secretas se cargan a mano en el panel de Render.

> **El plan gratis de Render duerme el servidor**
>
> A los 15 minutos sin tráfico HTTP el servidor se apaga y tarda alrededor de un minuto en despertar; al despertar, las salas en memoria se pierden. La aplicación lo mitiga: los clientes que están en una sala consultan `GET /health` cada 4 minutos (el servidor no se duerme mientras alguien juega); si igual se pierde la sala, se avisa "La sala ya no existe" y se vuelve al inicio; y al crear o unirse se espera al servidor con el aviso "Despertando el servidor...". El plan pago de Render no duerme el servidor.

Entornos

Entorno	Cómo se levanta	Datos
Local	`npm install` y `npm run dev:all` (Turborepo compila shared antes de las apps). Sin la clave de Supabase el servidor funciona igual para invitados.	Proyecto Supabase del equipo (o ninguno).
CI	Build de producción; servidor y web levantados en el runner; pruebas de punta a punta de invitados.	Sin base de datos.
Producción	Push a `main` → Render construye y despliega web y server automáticamente.	Datos reales.

Tabla 22 — Entornos

8. Conceptos transversales

8.1 Modelo de dominio

Varios términos del juego tienen un significado preciso en el sistema (ver también el glosario, §12):

- Una **sala** agrupa jugadores y tiene un **anfitrión**. En la sala se juega un **juego** de los tres disponibles.
- Un juego tiene N **partidas** (configurable 1–10). En cada partida los impostores son los mismos; se eligen de nuevo al empezar la siguiente.
- Una partida tiene varias **rondas**: actividad (pistas, tiempos o dibujo), discusión y votación. Termina cuando todos los impostores fueron expulsados o cuando quedan tantos impostores como inocentes (o al llegar al tope de rondas, si hay uno: ganan los impostores).
- En El Impostor cada ronda tiene una palabra nueva; en el Impostor dibujo hay **una palabra y un lienzo por partida**, y cada ronda se dibuja sobre el mismo dibujo; en el Impostor por tiempo cada ronda tiene un **tiempo objetivo** nuevo.
- El **puntaje** se calcula por ronda y por partida, y se muestra al terminar cada partida (antes delataría al impostor).

8.2 Diseño de datos

El sistema separa deliberadamente dos clases de datos con necesidades opuestas:

	Estado vivo	Datos persistentes
Ejemplos	Salas, jugadores conectados, fase actual, votos, temporizadores, chat, lienzo.	Cuentas, perfiles, estadísticas, amistades, workshop, reportes.
Vida útil	Minutos u horas; desaparece al cerrar la sala.	Indefinida.
Patrón de acceso	Muchas escrituras por segundo por sala.	Pocas escrituras; lecturas con filtros y agregados.
Requisito crítico	Latencia mínima.	Integridad y control de acceso.
Almacén	**Memoria** del proceso servidor (una instancia).	**PostgreSQL** en Supabase.

Tabla 23 — Por qué hay dos almacenes de datos

8.2.1 Estado vivo en memoria

`RoomStore` guarda un `Map<código, sala>` con los jugadores, el anfitrión, los ajustes, el chat (últimos mensajes) y las listas del workshop elegidas; `GameService` guarda el motor activo de cada sala; `PlayerService` vincula cada socket con su identidad estable (el `sessionToken` del invitado o el id de la cuenta) para poder reconectar. Las salas sin jugadores se limpian solas. La v2.0 planeaba llevar este estado a Redis para tener varias réplicas; se decidió no hacerlo (ADR-05): con una instancia alcanza para el uso real del producto y Redis agregaba costo y complejidad.

8.2.2 Modelo persistente en PostgreSQL

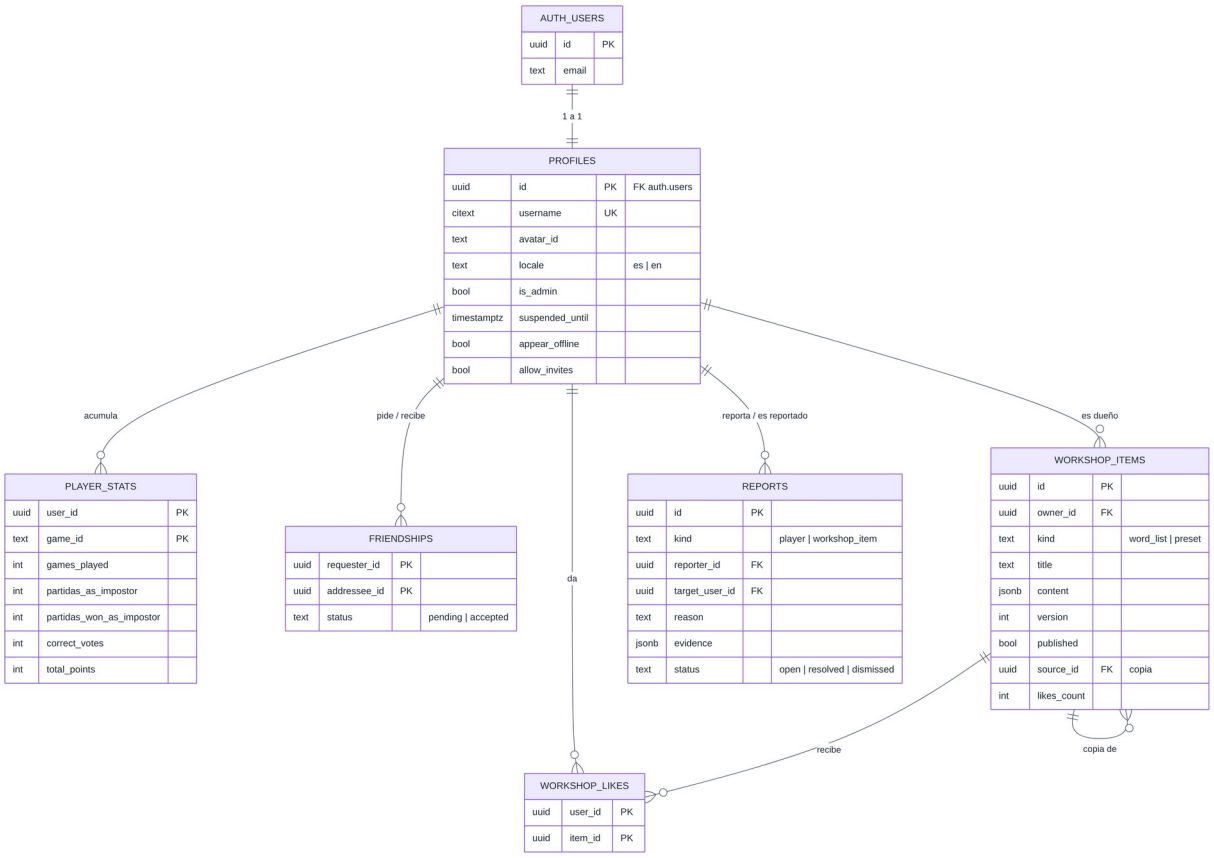


Figura 14 — Modelo entidad–relación de los datos persistentes

Tabla	Propósito	Decisiones de diseño
profiles	Datos del usuario.	Relación 1 a 1 con `auth.users` (un trigger crea el perfil al registrarse). `username` es citext único: evita "Juan" y "juan" duplicados. Marcas de admin, suspensión y privacidad.
player_stats	Estadísticas acumuladas por usuario y juego.	Una fila por (usuario, juego). Las suma la función `record_game_stats`, que sólo puede llamar el servidor.
friendships	Solicitudes y amistades.	Una fila por par, sin importar quién pidió (índice único). Estado `pending` / `accepted`.
workshop_items	Listas de palabras y presets.	`content` jsonb (palabras o ajustes). `version` y `source_id`/`source_version` para que una copia sepa si el original cambió.
workshop_likes	Likes del workshop.	Una fila por (usuario, ítem); el contador `likes_count` lo mantiene un trigger.
reports	Reportes de jugadores y contenido.	Guarda motivo, detalle y evidencia (chat); sólo la lee y escribe el servidor.

Tabla 24 — Tablas persistentes

8.2.3 Políticas de acceso (Row Level Security)

- **RLS activado en todas las tablas** y revisado con los *advisors* de seguridad de Supabase después de cada migración.

- `profiles`: lectura pública; cada usuario sólo puede modificar **su** avatar, idioma y privacidad (permiso por columna: no puede hacerse admin ni levantar su suspensión).
- `player_stats` y `workshop_items` publicados: lectura pública. `friendships`: sólo las dos personas involucradas. Lo privado del workshop: sólo su dueño. `reports`: nadie desde el cliente.
- Las escrituras las hace únicamente el Game Server con la clave secreta, que nunca llega al navegador (ADR-15). Una prueba de punta a punta verifica con el token de un usuario común que no puede saltearse estas reglas.

8.3 Contrato de comunicación

- Todos los eventos y sus datos están tipados en `packages/shared/src/types/events.ts` (`ClientEvents`, `ServerEvents`). Cliente y servidor importan el mismo archivo: si alguien cambia un evento, el otro lado deja de compilar (QS-10).
- Los errores viajan como **códigos** (`ROOM_FULL`, `word_already_used`) y no como texto; el cliente los traduce. Así el servidor no depende del idioma del usuario.
- Las acciones de juego usan un único evento `game:action` con `{ type, payload }`, para que un minijuego nuevo no requiera eventos nuevos en el transporte.

8.4 Seguridad e integridad del juego

Amenaza	Control
Un impostor inspecciona el tráfico o el estado del navegador para ver la palabra (o el tiempo objetivo).	El dato **nunca se envía** al impostor: la vista se calcula por jugador en el servidor.
Un jugador envía acciones fuera de turno, vota dos veces o dibuja de más.	El motor valida fase, turno, rol y si ya votó; el servidor recorta los trazos a la tinta disponible.
Un jugador no anfitrión intenta expulsar, configurar o iniciar.	Los gateways comparan el jugador de la sesión con el anfitrión de la sala.
Datos malformados o excesivos.	Validación de cada evento (tipos, rangos, claves extra) contra los esquemas; mensajes recortados a 200 caracteres; apodos de 2–16.
Insultos y abuso.	Filtro de insultos compartido (chat y pistas censurados; apodos, usuarios y workshop rechazados), reportes con evidencia, suspensiones; máximo 5 reportes cada 10 minutos.
Suplantación de usuario registrado.	JWT firmado por Supabase verificado con su JWKS público; sin token válido, la sesión es de invitado.
Escalada de privilegios desde el cliente (hacerse admin).	Permisos por columna en `profiles`; el servidor verifica en la base que el usuario sea admin en cada acción del panel.
Lectura de datos de otros usuarios.	Row Level Security en PostgreSQL.
Intercepción de tráfico.	HTTPS/WSS en todos los canales; CORS restringido al dominio del frontend (`CLIENT_URL`).
Filtración de secretos.	Claves sólo en variables de entorno del servidor; un escaneo de secretos en el hook pre-commit y en el CI.
Dependencias vulnerables.	`npm audit` de producción en el CI: el build falla con vulnerabilidades altas o críticas.

Tabla 25 — Amenazas y controles

8.5 Manejo del tiempo

Todos los temporizadores (ver palabra, turno, discusión, votación, resultados, fin de partida) viven en el **servidor**, como *timers pausables*: si alguien se desconecta se congelan y al reanudar siguen con el tiempo que les quedaba. El cliente sólo muestra una cuenta regresiva. En el Impostor por tiempo, el reloj de cada jugador se mide en el navegador (`performance.now()`) para no sumar la latencia de la red, pero el servidor **no acepta más tiempo del que realmente pasó** desde que el jugador tocó EMPEZAR.

8.6 Identidad y sesiones

- **Invitado**: el navegador genera un `sessionToken` por pestaña (en `sessionStorage`), que nunca se difunde a otros jugadores; el servidor genera el `playerId` público. Recargar la página conserva la identidad.
- **Registrado**: la identidad es el id de la cuenta, tomado del JWT. El apodo por defecto es su nombre de usuario y sus resultados se suman a su perfil. Una segunda pestaña con la misma cuenta reemplaza a la primera.
- **Reconexión**: al volver, `session:ready` restaura la sala y el estado del juego. En el lobby, si no vuelve en 30 segundos, sale de la sala; en la partida se espera (pausada) hasta que vuelva o hasta que quien decide toque "Seguir sin esperar".

8.7 Internacionalización

La interfaz usa diccionarios por idioma (**español rioplatense e inglés**); el idioma se detecta del navegador la primera vez y luego se recuerda (y, con cuenta, se guarda en el perfil). El idioma de la **sala** (elegido por el anfitrión) define las listas de palabras; el idioma de la **interfaz** es personal de cada jugador. Hay 8 listas de palabras por idioma (40–50 palabras cada una) y las listas del workshop tienen su propio idioma. El portugués planificado en la v2.0 se descartó para traducir completo lo que sí se ofrece.

8.8 Manejo de errores y retroalimentación

- Validaciones inmediatas en el cliente (apodo, formato del código, cantidad de palabras de una lista) para responder sin ir al servidor.
- Errores del servidor con código traducible y mensaje visible en pantalla.
- Al crear o unirse, si el servidor no responde se muestra "Despertando el servidor..." y se espera hasta 70 segundos (plan gratis de Render).
- Si el servidor se reinició y la sala ya no existe, se avisa "La sala ya no existe" y se vuelve al inicio, en lugar de dejar al jugador en una sala fantasma.
- Indicador de "Desconectado del servidor" y pantalla de pausa durante la partida cuando falta alguien.

8.9 Escalabilidad

Decisión: una sola instancia. Todo el estado vivo está en memoria de un proceso. Es adecuado para decenas de salas simultáneas —muy por encima del uso real del producto— y mantiene la operación simple y gratuita. No permite agregar instancias: dos réplicas no compartirían salas.

Si en el futuro hiciera falta escalar, el camino está identificado y no requiere reescribir los motores: llevar `RoomStore` a Redis (detrás de la misma interfaz), usar el adaptador Redis de Socket.io para difundir entre réplicas y ejecutar el motor de cada sala en una sola réplica. Es lo que planeaba la v2.0; se postergó (ADR-05) porque su costo no se justifica con el tráfico actual.

8.10 Observabilidad

- Logs del Logger de NestJS (errores de Supabase, estadísticas que no se guardaron, inicio del servidor), visibles en el panel de Render.
- Endpoint `GET /health` para el health check de Render y para mantener despierto el servidor.
- Los *advisors* de seguridad y rendimiento de Supabase se revisan después de cada migración.

8.11 Estrategia de pruebas

Nivel	Qué se prueba	Herramienta y resultado
Unitarias	Reglas de los tres juegos sin red y con timers simulados: votos y empates (con TDD), fin de partida, puntaje, tinta, tiempo objetivo, validación de ajustes, salas, amigos, workshop, moderación y filtro de insultos.	Jest: **291 tests**; cobertura de ramas mínima exigida en la lógica crítica (100 % en votos y reglas). Prueba de mutación manual: 8 de 8 mutantes detectados.
De punta a punta (invitados)	Partidas reales de los tres juegos en Chromium con bots por socket, en escritorio y celular; recarga, reconexión, reglas que valida el servidor, salas públicas, chat, servidor que se reinicia.	Playwright + socket.io-client: 11 suites (≈117 checks). **Corren en el CI** contra el build recién compilado.
De punta a punta (cuentas)	Login, estadísticas, amigos, workshop, moderación (incluido RLS con el token de un usuario común) y ajustes, con cuentas temporales que se borran al terminar.	5 suites (81 checks), contra Supabase.
Manual	Partidas completas con 4 navegadores contra producción en cada fase; las capturas de este manual salen de esas partidas.	—

Tabla 26 — Estrategia de pruebas

Además, un **hook pre-commit** busca secretos, corre el linter y los tests antes de cada commit, y el **CI** repite todo en cada push. El detalle (ciclo TDD, casos borde, mutación y los 29 bugs que se encontraron en el propio código) está en `docs/TESTING.md` del repositorio.

8.12 Diseño visual y experiencia

La identidad "Neon Noir" combina un fondo oscuro (misterio, mentiras) con acentos brillantes (diversión). Los colores tienen significado estable en toda la aplicación: **violeta** para acciones principales, **rosa** para todo lo relacionado con el impostor, **verde** para inocentes y confirmaciones, **ámbar** para temporizadores y **rojo** para expulsión y errores. El diseño es mobile-first: en pantallas chicas el header pasa a un menú ☰ y el chat a un botón flotante con contador de mensajes no leídos.

El **sonido** se sintetiza en el navegador con Web Audio, sin archivos: efectos para tu turno, mensajes, votación, revelación, victoria y derrota, y una música de fondo suave. Cada jugador lo prende, apaga y regula en Ajustes.

9. Decisiones de arquitectura

Cada decisión se documenta como un **ADR** (*Architecture Decision Record*): el contexto que la motivó, las alternativas que se evaluaron, los criterios usados, la decisión y sus consecuencias —incluidas las negativas—. El foco no está en la tecnología elegida sino en **por qué** se eligió frente a las otras.

9.1 Criterios de evaluación

Para que las decisiones sean comparables, el equipo definió criterios derivados de los objetivos de calidad (§1.4) y de las restricciones (§2), con un peso relativo:

Criterio	Peso	Origen
K1 Latencia / tiempo real	5	Objetivo de calidad 1, O2
K2 Integridad del juego	5	Objetivo de calidad 2, O3
K3 Fricción para el usuario	4	Objetivo de calidad 3, O1
K4 Extensibilidad	3	Objetivo de calidad 4, O4
K5 Costo de operación	3	RT-4, O7
K6 Conocimiento del equipo / curva de aprendizaje	3	RO-1, RO-2
K7 Simplicidad operativa	2	RO-1

Tabla 27 — Criterios de decisión y pesos

En las decisiones con más alternativas se incluye una matriz de puntaje (1 = malo, 5 = excelente), ponderada con estos pesos.

ADR-01 — Comunicación en tiempo real con Socket.io

Contexto. Todos los jugadores de una sala deben enterarse de cada cambio (turno, voto, mensaje) en menos de 150 ms. El servidor necesita *empujar* información sin que el cliente la pida.

Alternativa	K1 (×5)	K2 (×5)	K5 (×3)	K6 (×3)	K7 (×2)	Total
REST + polling cada 1 s	1	4	2	5	5	**60**
Server-Sent Events + REST	4	4	4	3	3	**75**
WebSocket nativo (`ws`)	5	4	5	3	2	**78**
Socket.io	5	4	5	4	4	**85**
Servicio gestionado (Firebase RTDB / Supabase Realtime)	4	2	3	3	5	**63**

Tabla 28 — ADR-01: matriz de decisión de la comunicación

Decisión. Socket.io. Frente a WebSocket nativo aporta, sin código propio, *rooms* para difundir por sala, reconexión automática, respaldo por long-polling en redes que bloquean WebSockets y un adaptador Redis para escalar. SSE se descartó por ser unidireccional (las acciones seguirían necesitando HTTP) y polling por su latencia y costo. Los servicios gestionados se descartaron porque trasladan la lógica al cliente, lo que choca con K2.

Consecuencias. (+) Tiempo real bidireccional con una única conexión; integración nativa con NestJS. (−) Protocolo propio: el cliente debe usar `socket.io-client`; si se usa long-polling con varias réplicas hace falta afinidad de sesión (se mitiga priorizando el transporte WebSocket).

ADR-02 — Servidor autoritativo

Contexto. El juego depende de información oculta y de reglas (turnos, votos) que alguien debe hacer cumplir.

Alternativas. (a) Cliente autoritativo: cada navegador calcula el estado y lo comparte; (b) anfitrión autoritativo: el navegador del anfitrión hace de servidor (P2P/WebRTC); (c) **servidor autoritativo**.

Decisión. (c). Es la única opción en la que un jugador no puede leer ni alterar lo que no le corresponde (K2), y evita que el juego dependa de la conexión del anfitrión.

Consecuencias. (+) Integridad garantizada por diseño; los clientes son simples. (−) Toda la carga está en el servidor y cada acción hace un viaje de ida y vuelta, lo que refuerza la importancia de ADR-01 y ADR-05.

ADR-03 — Monolito modular con NestJS

Contexto. El servidor tiene varios dominios (salas, chat, juego, estadísticas) y debe crecer con nuevos minijuegos, mantenido por un equipo chico.

Alternativa	Ventaja principal	Motivo de descarte
Express “plano” + Socket.io	Mínimo, conocido.	Sin estructura: con tres juegos el código de eventos se vuelve difícil de mantener (K4).
Colyseus (framework de game servers)	Resuelve salas y sincronización de estado.	Modelo de estado propio y más curva de aprendizaje (K6); acopla todo el sistema a una librería de nicho.
Microservicios (salas, juego, chat)	Escalado y despliegue independientes.	Costo operativo y de red desproporcionado para el tamaño del equipo (K5, K7); el juego necesita salas y chat con latencia mínima.
NestJS modular	Módulos por dominio, inyección de dependencias, gateways WebSocket nativos, TypeScript.	— (elegida)

Tabla 29 — ADR-03: alternativas para el servidor

Consecuencias. (+) Límites claros entre módulos; servicios fáciles de probar por inyección de dependencias; si en el futuro un módulo necesitara separarse (por ejemplo estadísticas) ya está aislado. (−) Más código ceremonial (decoradores, módulos) que Express.

ADR-04 — Next.js (App Router) para la aplicación web

Contexto. La web tiene páginas muy interactivas (la sala) y páginas de contenido (inicio, salas públicas, perfiles).

Alternativas. SPA con Vite + React; Next.js.

Decisión. Next.js. Las pantallas de juego se renderizan en el cliente (`"use client"`), pero la página de inicio, el explorador de salas públicas y los perfiles se benefician del renderizado en servidor (primer pintado más rápido, K3) y del enrutado por archivos. La salida *standalone* produce un servidor Node autocontenido fácil de desplegar.

Consecuencias. (+) Rendimiento de carga inicial; convenciones claras. (−) Build más pesado que Vite; hay que marcar explícitamente los componentes de cliente.

ADR-05 — Estado vivo en memoria (sin Redis)

Contexto. El estado de una partida cambia varias veces por segundo y sólo importa mientras la sala existe. La v2.0 planeaba Redis para tener varias réplicas y sobrevivir a reinicios.

Alternativa	K1 (×5)	K5 (×3)	K7 (×2)	Escala horizontal	Sobrevive reinicio
Memoria del proceso	5	5	5	No	No
Redis	5	3	3	Sí	Sí
PostgreSQL	2	4	4	Sí	Sí

Tabla 30 — ADR-05: dónde guardar el estado vivo

Decisión. Memoria, también en la versión final. Redis se descartó al construirla: no hay un plan gratuito de Redis que convenga junto a Render, exigía resolver qué réplica ejecuta cada partida y el tráfico real entra holgado en una instancia. PostgreSQL se descarta para este uso: una escritura transaccional por cada voto o trazo agrega latencia sin beneficio, porque el dato es efímero.

Consecuencias. (+) Latencia mínima, cero infraestructura extra, operación simple. (−) Un reinicio del servidor pierde las salas activas (R-1): se mitiga manteniendo despierto el servidor mientras hay jugadores y avisando con claridad si pasa. `RoomStore` tiene una interfaz propia, por lo que cambiar de almacén no obliga a tocar los gateways.

ADR-06 — PostgreSQL (Supabase) para datos persistentes

Contexto. La versión final guarda usuarios, estadísticas, amistades, el workshop y reportes. Los datos son relacionales (un usuario tiene muchos amigos, ítems y likes; un ítem tiene copias) y las estadísticas son agregaciones por usuario y juego.

Alternativas. MongoDB (documentos); Firebase Firestore; **PostgreSQL gestionado en Supabase**.

Decisión. PostgreSQL en Supabase. Las relaciones N:M y los agregados se expresan naturalmente en SQL, con integridad referencial. Supabase suma, en un mismo servicio gratuito, autenticación y Row Level Security (K5, K7). Además es el modelo que el equipo conoce de la carrera (K6). Se descartó MongoDB porque las estadísticas requieren cruzar colecciones, y Firestore por su modelo de costos por lectura y por acoplar el cliente a la base.

Consecuencias. (+) Consultas de estadísticas simples y consistentes; seguridad declarativa con RLS. (−) Dependencia de un proveedor; el plan gratuito pausa proyectos inactivos (riesgo R-6).

ADR-07 — Cuentas opcionales (invitado primero) con Supabase Auth

Contexto. El objetivo O1 pide jugar sin registrarse, pero O5 pide guardar progreso.

Alternativas. (a) Registro obligatorio; (b) autenticación propia (bcrypt + JWT); (c) **cuenta opcional con Supabase Auth**; (d) Auth0/Clerk.

Decisión. (c). El registro obligatorio contradice el objetivo con mayor impacto en la adopción (K3). Implementar autenticación propia implica manejar contraseñas, recuperación y verificación de email, con mayor riesgo de seguridad. Auth0/Clerk resuelven lo mismo que Supabase pero serían un proveedor más. El servidor sólo **verifica** el JWT; si no hay token, el usuario juega como invitado.

Consecuencias. (+) Entrada inmediata para todos; progreso para quien lo quiera; no se almacenan contraseñas propias. (−) Dos tipos de identidad que el código debe contemplar (`profileId` puede ser nulo).

ADR-08 — Motores de juego enchufables

Contexto. La plataforma pasa de uno a tres minijuegos y debe admitir más. La v2.0 planeaba dos juegos muy distintos (*Dibujá y Adiviná* y *Teléfono Descompuesto*).

Alternativas. (a) Lógica de cada juego dentro de los gateways con condicionales por tipo de juego; (b) **interfaz `GameEngine` + registro** (patrones *Strategy* y *Registry*); y, para elegir los juegos, (c) juegos nuevos con mecánicas propias o (d) **variantes de la mecánica de deducción**.

Decisión. (b) y (d). El resto del sistema conoce sólo la interfaz; la configuración de cada juego se describe con un `settingsSchema` que el lobby convierte en controles. Se eligió construir **Impostor por tiempo** e **Impostor dibujo** porque reutilizan toda la deducción (roles, discusión, votación, desempates, puntaje, pausa) a través de `DeductionEngine`: cada juego nuevo sólo define su actividad de la ronda. Eso permitió terminar tres juegos completos, probados y consistentes entre sí en el tiempo disponible, en lugar de dos juegos con reglas, pantallas y puntajes completamente distintos.

Consecuencias. (+) Un juego nuevo no toca salas, chat ni transporte (QS-7) y comparte las pantallas de discusión, votación y resultados; cada motor se prueba aislado. (−) Los tres juegos se parecen entre sí; un juego de otra familia (sin impostor) tendría que implementar `GameEngine` directamente. El dibujo necesitó un evento propio (`game:draw`) para los trazos en vivo, sin cambiar la interfaz.

ADR-09 — Vistas por jugador calculadas en el servidor

Contexto. Cada jugador debe ver una versión distinta del mismo estado (el impostor no ve la palabra; los votos se ocultan hasta el final).

Alternativas. (a) Enviar el estado completo y ocultar en la interfaz; (b) **calcular una vista por jugador en el servidor**.

Decisión. (b). Con (a) cualquier jugador podría leer la palabra en las herramientas del navegador (K2).

Consecuencias. (+) Integridad por diseño. (−) Un envío por socket en lugar de un único *broadcast* por sala; el costo es despreciable para salas de hasta 12 jugadores.

ADR-10 — Modo voz sin audio propio

Contexto. Muchos grupos juegan en persona o ya están en una llamada.

Alternativas. Integrar voz con WebRTC/LiveKit; **coordinar turnos y dejar el audio a herramientas externas**.

Decisión. Coordinar turnos solamente. Integrar audio multiplica la complejidad (servidores TURN, calidad de red, permisos de micrófono) y el costo, y no aporta a quien ya está en Discord o en la misma habitación.

Consecuencias. (+) Modo voz simple y gratuito. (−) En modo voz no hay registro de lo que se dijo; el sistema confía en los jugadores y el anfitrión puede avanzar turnos manualmente.

ADR-11 — Zustand para el estado del cliente

Contexto. El cliente necesita compartir pocos datos globales: identidad local, sala, chat y sesión.

Alternativas. Redux Toolkit; Context API; Zustand.

Decisión. Zustand: stores pequeños sin *providers*, mínimo código repetitivo y buena compatibilidad con componentes de servidor de Next.js. Redux resulta excesivo para tres stores; Context re-renderiza todo el árbol ante cada mensaje de chat.

ADR-12 — Monorepo con tipos compartidos

Contexto. Cliente y servidor deben coincidir exactamente en nombres y formas de decenas de eventos.

Alternativas. Repositorios separados con tipos duplicados; especificación OpenAPI/AsyncAPI con generación de código; **monorepo con paquete `shared`**.

Decisión. Monorepo con Turborepo. Un cambio en un evento se hace en un único lugar y rompe la compilación del lado que no se actualizó (QS-10). Turborepo garantiza que `shared` se compile antes que las aplicaciones.

Consecuencias. (+) Un solo pull request para cambios de punta a punta. (−) Build algo más complejo y un único repositorio para todo el equipo.

ADR-13 — Despliegue en Render

Contexto. RT-2 exige procesos de larga duración para WebSockets; RT-4 exige bajo costo.

Alternativa	Evaluación
Vercel (web) + Railway (server)	Configuración inicial del MVP. Vercel no admite WebSockets persistentes para el servidor; obligaba a dos proveedores.
Railway (web + server)	Era la elección de la v2.0: un proveedor, deploy desde GitHub, procesos persistentes. La prueba gratuita de la cuenta venció y pasaba a ser pago.
Render (web + server)	Plan gratuito con procesos persistentes y WebSockets, deploy automático desde `main` y configuración versionable (`render.yaml`). Los servicios gratuitos se duermen tras 15 minutos sin tráfico.
VPS propio	Máximo control y costo fijo bajo, pero exige administrar sistema operativo, TLS y actualizaciones (K7).

Tabla 31 — ADR-13: alternativas de despliegue

Decisión. Render para la web y el servidor; Supabase para Auth y PostgreSQL. El costo de que el servidor gratuito se duerma se absorbió en el código (§7 y §8.8) en lugar de pagar un plan; pasar al plan pago es un cambio de configuración, sin tocar código.

ADR-14 — Listado de salas públicas por REST

Contexto. El explorador de salas públicas muestra una lista que cambia, pero no requiere precisión al instante.

Alternativas. Suscripción por Socket.io a un canal global; **endpoint REST consultado cada 5 segundos**.

Decisión. REST (`GET /rooms/public`, consultado cada 5 segundos desde la página de inicio). La lista tolera 5 segundos de retraso; no obliga a abrir un socket antes de que el usuario elija una sala; y un canal global difundiría cada cambio de cada sala a todos los que están mirando la lista, un costo que crece con el éxito del producto.

ADR-15 — Resultados escritos sólo por el servidor

Contexto. Las estadísticas, los likes, las amistades y los reportes deben ser confiables.

Decisión. Los clientes sólo **leen** (RLS). Todas las escrituras que importan las hace el Game Server, único que conoce el desenlace real de una partida, con la clave secreta: las estadísticas, con la función `record_game_stats`; el workshop, las amistades y los reportes, después de validar cada pedido.

Consecuencias. (+) Nadie puede "inflar" sus estadísticas, likes o reportes desde el navegador. (−) El servidor necesita credenciales privilegiadas que deben protegerse (§8.4), y sin la clave el servidor funciona sólo para invitados.

ADR-16 — Amigos y workshop por el socket del juego

Contexto. Los amigos necesitan presencia en vivo ("en una sala", "jugando") e invitaciones instantáneas; el workshop, escrituras validadas.

Alternativas. (a) Supabase Realtime (suscripciones a cambios de la base) y escrituras directas desde el navegador con RLS; (b) **eventos por el mismo socket del servidor de juego**.

Decisión. (b). El servidor ya sabe quién está conectado y en qué sala, así que la presencia sale gratis de su memoria; no hace falta escribir en la base cada cambio de estado. Las escrituras pasan por el servidor, que valida (por ejemplo, 10 a 300 palabras por lista) y aplica el filtro de insultos; las **lecturas** del workshop van directo a Supabase, que las filtra con RLS.

Consecuencias. (+) Una sola conexión; reglas en un solo lugar; presencia sin costo de base. (−) Las solicitudes se guardan en la base, pero las invitaciones son sólo en vivo (vencen a los 5 minutos).

ADR-17 — Moderación: filtro compartido, reportes y panel de admin

Contexto. Con salas públicas y contenido creado por usuarios aparece el riesgo de abuso (R-4).

Alternativas. (a) Sólo expulsar desde la sala; (b) moderación automática con un servicio externo; (c) **filtro de insultos propio + reportes + panel de administración**.

Decisión. (c). El filtro vive en el paquete compartido (lo usan la web, para avisar antes, y el servidor, que decide): censura con `***` el chat y las pistas, y rechaza apodos, usuarios y contenido del workshop con insultos (contempla mayúsculas, acentos, letras estiradas y *leetspeak* sin marcar palabras inocentes como "computadora"). Los reportes adjuntan el chat como evidencia y los admins —una marca en el perfil— los resuelven desde `/admin`.

Consecuencias. (+) Sin costo ni dependencias externas; probado con casos borde. (−) Un filtro por lista nunca es perfecto: por eso existen los reportes.

10. Requerimientos de calidad

10.1 Árbol de calidad

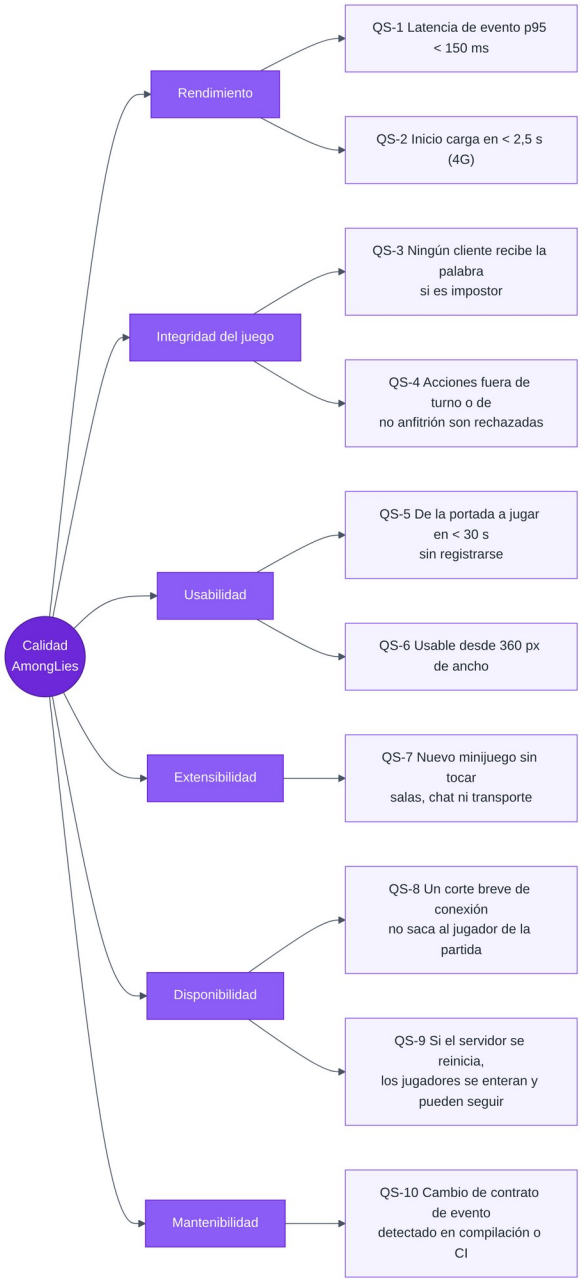


Figura 15 — Árbol de calidad con los escenarios que lo concretan

10.2 Escenarios de calidad

Cada escenario describe un estímulo, la respuesta esperada y una medida verificable.

ID	Atributo	Estímulo	Respuesta esperada	Medida / resultado
QS-1	Rendimiento	Un jugador vota en una sala de 8 jugadores.	Los 8 ven el contador actualizado.	p95 < 150 ms en la misma región.
QS-2	Rendimiento	Un usuario abre la página de inicio con 4G.	La página es interactiva.	LCP < 2,5 s (con el servidor despierto).

ID	Atributo	Estímulo	Respuesta esperada	Medida / resultado
QS-3	Integridad	El impostor inspecciona el tráfico y el estado de la aplicación.	No encuentra la palabra ni el tiempo objetivo.	Verificado en tests unitarios de los tres motores.
QS-4	Integridad	Un jugador vota dos veces, juega fuera de turno o envía `game:start` sin ser anfitrión.	El servidor rechaza la acción.	100 % de rechazos en la suite e2e `servidor` (15 checks).
QS-5	Usabilidad	Una persona recibe un link de invitación por primera vez.	Está dentro del lobby.	< 30 s, sin registrarse.
QS-6	Usabilidad	Se juega una partida completa en un celular de 360 px de ancho.	Todas las acciones son accesibles sin zoom.	0 páginas con scroll horizontal; e2e `dibujo-celular` y `chat` en celular.
QS-7	Extensibilidad	Se agrega un minijuego nuevo.	No se modifican salas, chat ni transporte.	Cumplido dos veces: Tiempo y Dibujo.
QS-8	Disponibilidad	A un jugador se le corta la conexión o recarga la página en medio de la partida.	Vuelve a su lugar; la partida lo espera pausada.	e2e `recarga` y `reconexion` (23 checks).
QS-9	Disponibilidad	El servidor se reinicia o se dormía.	Los jugadores se enteran y pueden seguir; crear una sala espera a que despierte.	e2e `sala-perdida`, que apaga y prende el servidor (6 checks).
QS-10	Mantenibilidad	Un desarrollador cambia la forma de un evento sólo en el servidor.	El error se detecta antes de desplegar.	Falla la compilación o el CI.

Tabla 32 — Escenarios de calidad

11. Riesgos y deuda técnica

Esta sección registra con honestidad los problemas conocidos del sistema terminado y qué pasó con la deuda técnica detectada en el MVP.

11.1 Riesgos

ID	Riesgo	Prob.	Impacto	Mitigación (estado)
R-1	Un reinicio del servidor (deploy o plan gratis que lo duerme) pierde las salas activas.	Media	Alto	Keep-alive mientras hay jugadores, aviso de sala perdida y espera al despertar (hecho). Plan pago de Render para eliminarlo.
R-2	Una desconexión breve saca al jugador de la partida.	Alta	Alto	Reconexión con pausa de la partida (hecho, RF-17).
R-3	Límites de los planes gratuitos ante un pico de uso.	Media	Medio	Una instancia alcanza para el uso actual; salas con cupo; plan pago si crece.
R-4	Abuso en salas públicas y en el workshop (spam, insultos).	Media	Medio	Salas privadas por defecto, filtro de insultos, reportes, suspensiones y panel de admin (hecho).
R-5	Trampas fuera del sistema (compartir pantalla, contarse la palabra).	Media	Bajo	No mitigable técnicamente; es un juego entre amigos.
R-6	Supabase pausa los proyectos gratuitos tras 7 días sin actividad.	Alta	Medio	Pasó en octubre de 2026 y se restauró desde el panel. Mientras está pausado, el juego sigue para invitados; las cuentas no andan. Plan pago o actividad periódica.
R-7	Trazos del dibujo muy frecuentes saturan la conexión.	Baja	Medio	Puntos agrupados por mensaje, límites de puntos por mensaje y por turno, tinta limitada (hecho).
R-8	Dependencias con vulnerabilidades nuevas.	Media	Alto	`npm audit` en el CI frena el deploy (en octubre de 2026 detectó una crítica de Next.js, que se actualizó).

Tabla 33 — Riesgos

11.2 Deuda técnica del MVP: estado

ID	Hallazgo en el MVP	Estado en la versión final
TD-1	`advance` (avanzar turno en modo voz) no verificaba que lo emita el anfitrión.	Resuelto: sólo el anfitrión, con test.
TD-2	La pantalla "Fin del juego" podía no llegar a verse.	Resuelto: se muestra mientras la fase sea `game-end`.
TD-3	Datos de eventos usados sin validar (`data as any`).	Resuelto: validación de cada evento y de los ajustes contra el esquema, con tests de casos borde.
TD-4	`game:back-to-lobby` no verificaba permisos.	Resuelto: sólo el anfitrión, con test e2e.
TD-5	Mensajes de error del servidor en inglés mostrados tal cual.	Resuelto: errores por código, traducidos en el cliente.
TD-6	Desconexión = salida inmediata.	Resuelto: reconexión con pausa (RF-17).
TD-7	No había pruebas automáticas.	Resuelto: 291 tests unitarios y 16 suites de punta a punta; CI y hook pre-commit.

ID	Hallazgo en el MVP	Estado en la versión final
TD-8	El evento del separador "Partida N · Ronda M" no estaba declarado.	Resuelto: se emite como `chat:message`.
TD-9 (nueva)	El estado vivo no sobrevive a un reinicio del servidor.	Aceptada (ADR-05); mitigada (R-1).
TD-10 (nueva)	No hay historial partida por partida en el perfil, sólo estadísticas acumuladas.	Pendiente: requeriría guardar cada partida (tablas de la v2.0).

Tabla 34 — Deuda técnica

12. Glosario

Término	Definición
Sala	Espacio virtual donde un grupo de jugadores se reúne, chatea y juega. Se identifica con un código de 6 caracteres.
Código de sala	Identificador de 6 caracteres (letras y números sin ambigüedades como O/0 o I/1) que se comparte para invitar.
Anfitrión (host)	Jugador que administra la sala: elige juego, configura, inicia, expulsa. Es quien la crea, salvo que transfiera el rol.
Administrador (admin)	Usuario con permiso de moderación sobre toda la plataforma: revisa reportes y suspende cuentas. No confundir con el anfitrión de una sala.
Invitado	Usuario que juega sin cuenta, sólo con apodo y avatar.
Usuario registrado	Usuario con cuenta; conserva perfil, estadísticas, amigos y su colección del workshop.
Sala pública / privada	Una sala pública aparece en el listado de la página de inicio; una privada sólo es accesible con código, link o invitación.
Juego	Sesión completa de un minijuego desde "Iniciar" hasta la pantalla final.
Partida	Subdivisión del juego con los mismos impostores. Termina cuando un bando gana.
Ronda	Ciclo actividad → discusión → votación dentro de una partida.
Impostor / Inocente	Roles secretos: el impostor no conoce la palabra (o el tiempo objetivo); los inocentes sí.
Pista	Palabra que cada jugador da en su turno en El Impostor para demostrar que conoce la palabra secreta sin decirla.
Tiempo objetivo	En el Impostor por tiempo, la duración secreta que cada inocente intenta "parar" en su turno.
Tinta	En el Impostor dibujo, cuánto se puede dibujar por turno (100 % = una línea de lado a lado del lienzo).
Eliminado / espectador	Jugador expulsado por votación: sigue viendo la partida pero no puede chatear ni votar.
Modo chat / modo voz	En modo chat las pistas se escriben; en modo voz se dicen en voz alta y la app sólo controla los turnos.
Puntaje	Puntos por ronda (votar bien, sobrevivir) y por partida (ganar), que se muestran al terminar cada partida.
Workshop	Sección donde los usuarios crean, publican y guardan listas de palabras y presets de reglas.
Preset	Juego + todos sus ajustes, guardado para aplicarlo en el lobby con un clic.
Reporte / suspensión	Aviso a los admins sobre un jugador o contenido; una suspensión impide entrar a salas y publicar por un tiempo o para siempre.
Motor de juego (engine)	Componente del servidor que implementa las reglas de un minijuego como máquina de estados.
Vista de jugador	Versión del estado del juego filtrada para un jugador concreto.
Servidor autoritativo	Arquitectura en la que sólo el servidor decide el estado; los clientes envían intenciones.
Evento	Mensaje con nombre (por ejemplo `room:join`) que viaja por la conexión en tiempo real.
RLS	Row Level Security: reglas de PostgreSQL que limitan qué filas puede leer o escribir cada usuario.

Término	Definición
ADR	Architecture Decision Record: registro de una decisión de arquitectura con su contexto y consecuencias.
Prueba de punta a punta (e2e)	Prueba que usa el sistema completo como un usuario: navegador real, servidor y base.

Tabla 35 — Glosario

PARTE II

Manual de Usuario

Cómo jugar en AmongLies — para jugadores y anfitriones

M1. Bienvenida

AmongLies es un sitio web para jugar minijuegos con amigos, desde la computadora o el celular, sin instalar nada. Uno de ustedes crea una **sala**, comparte un código o un link y el resto entra. Dentro de la sala eligen un juego y la aplicación hace de moderador: reparte roles secretos, marca los turnos, controla el tiempo, cuenta los votos y lleva el puntaje. En los tres juegos hay uno o más **impostores** escondidos entre ustedes.

Los tres minijuegos

Juego	De qué se trata	Jugadores
🎭 **El Impostor**	Todos reciben una palabra secreta menos el impostor. Por turnos, cada uno da una pista. Después discuten y votan para expulsar a quien crean que es el impostor, que tiene que disimular y adivinar la palabra.	4 o más
⏱️ **Impostor por tiempo**	Todos conocen un tiempo secreto (por ejemplo, 16 segundos) menos el impostor. En su turno, cada uno arranca un reloj que no se ve y lo para cuando cree que llegó. El impostor tiene que deducir el tiempo mirando a los demás.	4 o más
🎨 **Impostor dibujo**	Entre todos dibujan la palabra secreta en el mismo lienzo, de a poquito (la tinta es limitada). El impostor no sabe qué se dibuja: tiene que sumar algo sin delatarse.	4 o más

Tabla 36 — Minijuegos disponibles

Qué necesitás

- Un navegador actualizado (Chrome, Edge, Firefox o Safari) en computadora, tablet o celular.
- Conexión a internet estable. No hace falta instalar nada ni crear una cuenta.
- Amigos: la cantidad mínima depende del juego (ver tabla anterior).
- Para el **modo voz** de El Impostor: estar en la misma habitación o en una llamada (Discord, Meet, WhatsApp…). AmongLies no transmite audio.

M2. Antes de jugar: conceptos clave

Leer esta sección antes del primer juego ayuda a entender todo lo que aparece después en pantalla.

Concepto	Qué significa para vos
Sala	Es el "lugar" donde se juntan. Tiene un **código de 6 caracteres** (por ejemplo K7QPW2) y un link. Mientras haya gente adentro, la sala existe; cuando todos se van, desaparece.
Anfitrión 👑	Quien crea la sala. Elige el juego, lo configura, lo inicia y puede expulsar jugadores. Puede pasarle la corona a otro. Si se va, la corona pasa automáticamente a otro jugador.
Invitado o cuenta	Podés jugar como **invitado** (sólo apodo y avatar) o con una **cuenta** gratuita. Con cuenta se guardan tus estadísticas y podés tener amigos y usar el workshop.
Sala privada o pública	Una sala **privada** (así se crean) sólo se encuentra con el código, el link o una invitación. Una **pública** aparece en la página de inicio y cualquiera puede entrar hasta llenar el cupo.
Juego, partida y ronda	Un **juego** tiene una o más **partidas**; en cada partida los impostores son los mismos. Cada partida tiene **rondas** (actividad → discusión → votación) hasta que un bando gana.
Roles secretos	Al empezar cada partida sabés si sos impostor o inocente. Nadie más puede ver tu rol, ni siquiera el anfitrión.
Puntaje	Sumás puntos por votar a un impostor (si sos inocente), por sobrevivir cada ronda (si sos impostor) y por ganar la partida. Los puntos se muestran al terminar cada partida.
Modo chat y modo voz	Las pistas, los tiempos y la discusión pueden ir por el chat o decirse **en voz alta** (si están juntos o en una llamada). En modo voz la app sólo marca de quién es el turno.

Tabla 37 — Conceptos clave

El recorrido general

Este diagrama muestra el camino típico desde que entrás al sitio hasta que terminás de jugar:

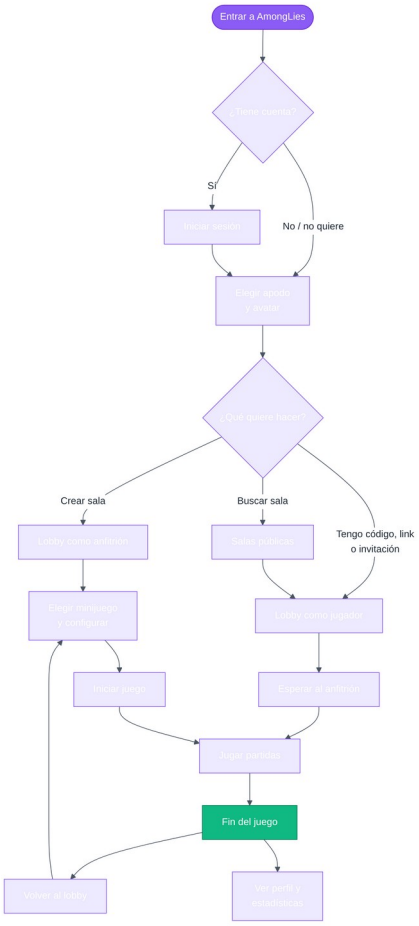


Figura 16 — Mapa de navegación: del ingreso al fin del juego

M3. Tipos de usuario y permisos

No todos pueden hacer lo mismo. La siguiente matriz muestra exactamente qué puede hacer cada tipo de usuario. Un mismo usuario puede tener varios tipos a la vez: por ejemplo, un usuario registrado que creó la sala es también anfitrión.

Acción	Invitado	Registrado	Anfitrión	Admin	Eliminado †
Crear una sala o unirse a una	✓	✓	—	✓	—
Ver y unirse a salas públicas	✓	✓	—	✓	—
Chatear en la sala	✓	✓	✓	✓	×
Jugar (pistas, tiempos, dibujar, votar)	✓	✓	✓	✓	×
Reportar a un jugador	✓	✓	✓	✓	✓
Elegir juego, configurarlo e iniciarlo	×	×	✓	×	×
Hacer pública la sala; definir cupo e idioma	×	×	✓	×	×
Expulsar jugadores / pasar la corona	×	×	✓	×	×
Guardar estadísticas	×	✓	(con cuenta)	✓	(cuenta igual)
Tener amigos e invitarlos a la sala	×	✓	(con cuenta)	✓	—
Crear y publicar en el workshop	× (sólo ver)	✓	(con cuenta)	✓	—
Usar listas y presets del workshop en el lobby	×	×	✓ (con cuenta)	×	×
Panel de admin: reportes y suspensiones	×	×	×	✓	—

Tabla 38 — Matriz de roles y permisos

† **Eliminado**: jugador expulsado por votación. Queda como espectador hasta que termina la partida. **Admin**: usuario con permiso de moderación sobre toda la plataforma (no es lo mismo que el anfitrión de una sala).

> **Tus datos están protegidos por diseño**
> Los permisos no dependen de lo que muestra la pantalla: el servidor rechaza cualquier acción que no te corresponda. Por ejemplo, si un jugador que no es anfitrión intentara iniciar el juego, el servidor lo ignora; y si sos el impostor, **la palabra secreta nunca se envía a tu dispositivo**, así que nadie puede "espiarla". Las estadísticas, los likes y los reportes sólo los guarda el servidor, y nadie puede modificar los datos de otro.

M4. Mapa de pantallas

Pantalla	La ve	Cómo se llega	Sección
Inicio (y salas públicas)	Todos	Entrando al sitio.	M5.1, M6.5
Iniciar sesión / Crear cuenta	Invitados	"Iniciar sesión" arriba a la derecha.	M5.2
Unirse por link	Todos	Abriendo un link de invitación.	M6.4
Lobby (anfitrión / jugador)	Jugadores de la sala	Al crear o unirse a una sala.	M6
El Impostor	Jugadores de la sala	Cuando el anfitrión lo inicia.	M7
Impostor dibujo	Jugadores de la sala	Cuando el anfitrión lo inicia.	M8
Impostor por tiempo	Jugadores de la sala	Cuando el anfitrión lo inicia.	M9
Mi perfil	Registrados	Tu @usuario arriba a la derecha.	M10
Amigos	Registrados	"Amigos" arriba a la derecha.	M11
Workshop	Todos (crear: registrados)	"Workshop" arriba a la derecha.	M12
Ajustes	Todos	El engranaje ⚙️ arriba a la derecha.	M13
Panel de admin	Admins	"Admin" arriba a la derecha.	M14

Tabla 39 — Pantallas del sistema

En el celular, los accesos de arriba a la derecha están en el menú ☰.

M5. Primeros pasos

M5.1 Cómo elegir tu apodo y avatar

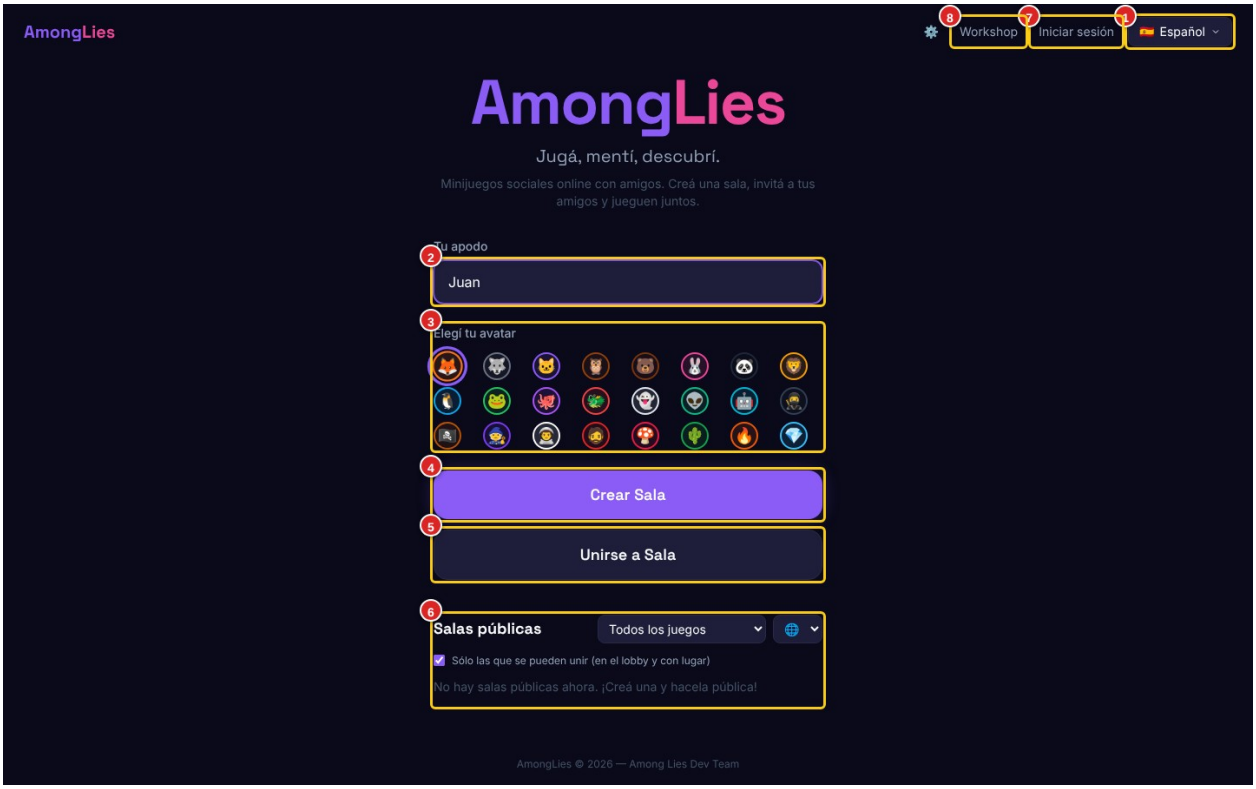


Figura 17 — Pantalla de inicio

1	Selector de idioma: Español o English. Cambia todos los textos al instante.
2	Tu apodo: entre 2 y 16 caracteres. Es el nombre que verán los demás.
3	Avatar: tocá uno de los 24 personajes. El elegido queda resaltado.
4	"Crear Sala": crea una sala nueva y te convierte en anfitrión.
5	"Unirse a Sala": despliega un campo para escribir el código que te pasaron.
6	Salas públicas: las salas abiertas a cualquiera, con filtros por juego e idioma (M6.5).
7	"Iniciar sesión": entrar con tu cuenta o crear una (opcional, M5.2).
8	Workshop: listas de palabras y presets hechos por los jugadores (M12). Al lado, ⚙️ abre los Ajustes (M13).

1. Entrá al sitio de AmongLies.
2. Escribí tu apodo (2).
3. Tocá el avatar que más te guste (3).
4. Elegí si querés crear una sala (4) o unirte a una (5).

Tu apodo, avatar e idioma quedan guardados en ese navegador: la próxima vez ya aparecen completos. Si tenés cuenta, el apodo es tu nombre de usuario.

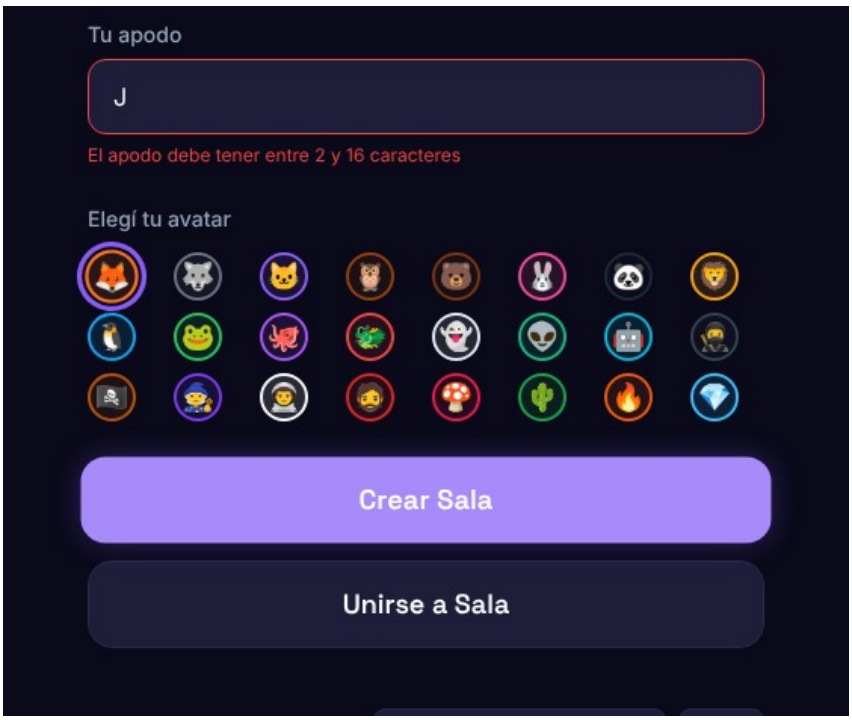


Figura 18 — Si el apodo es demasiado corto o largo, el campo se marca en rojo con el motivo

Los apodos con insultos no se aceptan ("Ese apodo no está permitido").

M5.2 Cómo crear una cuenta o iniciar sesión (opcional)

No es obligatorio. Creá una cuenta si querés guardar tus estadísticas, tener amigos y usar el workshop.

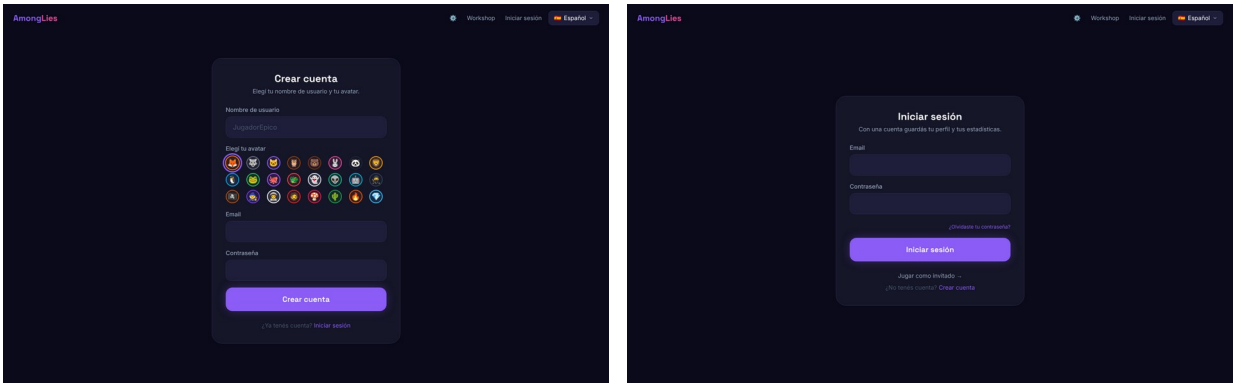


Figura 19 — Crear cuenta (izq.) e iniciar sesión (der.)

1. Tocá **Iniciar sesión** arriba a la derecha.
2. Si no tenés cuenta, tocá **Crear cuenta**: elegí tu **nombre de usuario** (único, de 3 a 16 letras, números o guión bajo), tu avatar, tu email y una contraseña.
3. Si ya tenés, escribí tu email y contraseña y tocá **Iniciar sesión**. Si te la olvidaste, tocá **¿Olvidaste tu contraseña?** y te llega un mail para elegir otra.
4. Listo: arriba a la derecha aparecen tu @usuario, **Amigos** y **Cerrar sesión**.

Si preferís no registrarte, tocá **Jugar como invitado**.

M5.3 Cómo cambiar el idioma

Tocá el selector de idioma arriba a la derecha (por ejemplo "🇪🇸 Español") y elegí otro; también está en **Ajustes**. El cambio es inmediato y se recuerda. El idioma de las **palabras del juego** lo define el anfitrión para toda la sala.

M6. Salas

M6.1 Cómo crear una sala

1. En la pantalla de inicio, con tu apodo y avatar elegidos, tocá **Crear Sala**.
2. Se abre el **lobby** de tu nueva sala y sos el anfitrión (tenés la corona 👑).
3. Invitá a tus amigos (M6.2), elegí el juego y configuralo (M6.7).

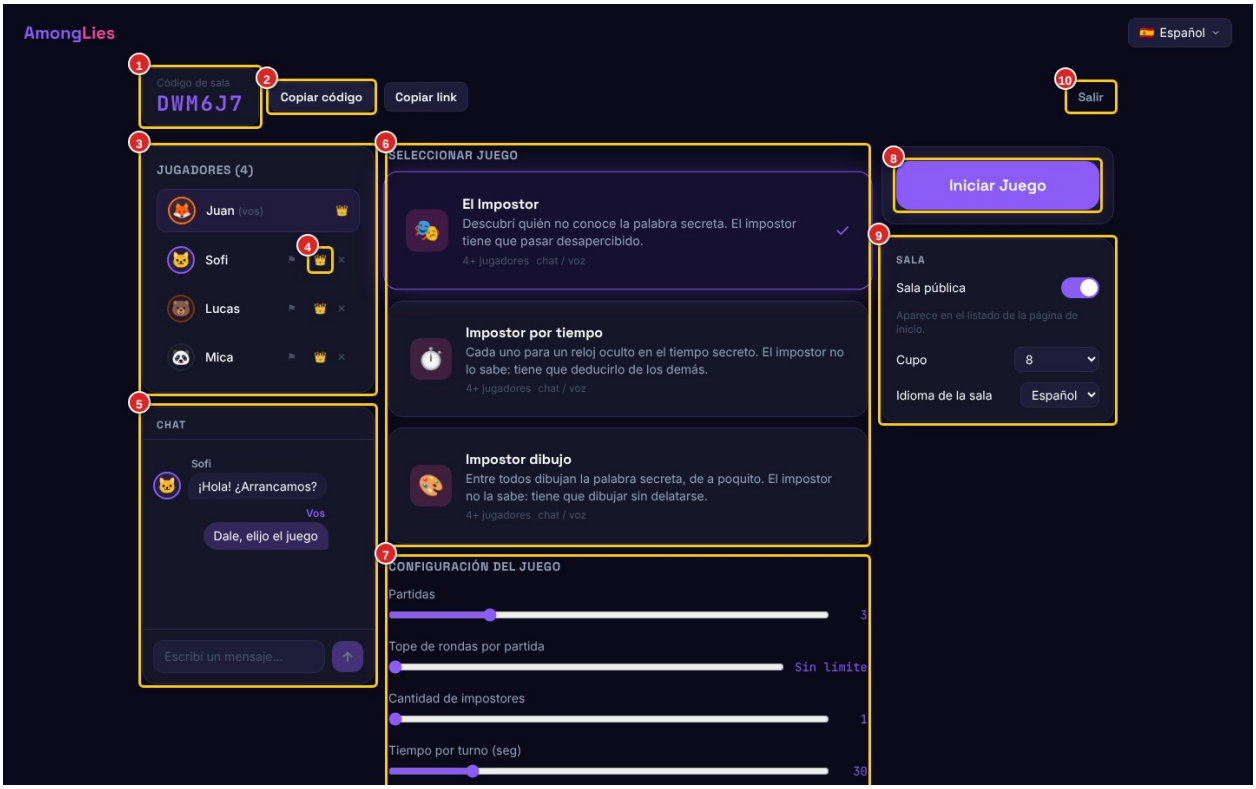


Figura 20 — Lobby visto por el anfitrión

1	Código de la sala: compartilo para que otros se unan.
2	"Copiar código" y "Copiar link": copian al portapapeles para pegar en WhatsApp, Discord, etc.
3	Lista de jugadores conectados. Tu fila dice "(vos)"; el anfitrión tiene la corona.
4	Controles del anfitrión en cada jugador: 👑 le pasa la corona; 🚫 lo expulsa; 🚩 lo reporta.
5	Chat de la sala.
6	Selector de juego: tocá la tarjeta del juego que quieran jugar.
7	Configuración del juego (sólo la modifica el anfitrión).
8	"Iniciar Juego": se habilita cuando hay jugadores suficientes (4).
9	Ajustes de la sala: pública o privada, cupo e idioma de las palabras.
10	"Salir": abandona la sala.

M6.2 Cómo invitar a tus amigos

- **Con el link** (recomendado): tocá **Copiar link** y mandalo. Al abrirlo, tus amigos sólo eligen apodo y avatar y ya están adentro.
- **Con el código**: tocá **Copiar código** o dictalo. Tus amigos lo escriben en "Unirse a Sala".

- **Con una invitación** (si tenés cuenta): en el lobby aparece **Invitar amigos** con tus amigos en línea; les llega un aviso con el botón **Unirme** (M11).
- **Haciéndola pública**: activá **Sala pública** y cualquier persona podrá encontrarla en la página de inicio.

M6.3 Cómo unirse con un código

1. En la pantalla de inicio, elegí apodo y avatar.
2. Tocá **Unirse a Sala**.
3. Escribí el código de 6 caracteres (se pasa a mayúsculas solo).
4. Tocá **Unirse**.



Figura 21 — Campo para ingresar el código de sala

M6.4 Cómo unirse con un link

Al abrir un link de invitación se muestra directamente la sala. Revisá tu apodo y avatar y tocá **Unirse**. Si la sala está en medio de un juego, esperá a que vuelvan al lobby y tocá **Unirse** otra vez.

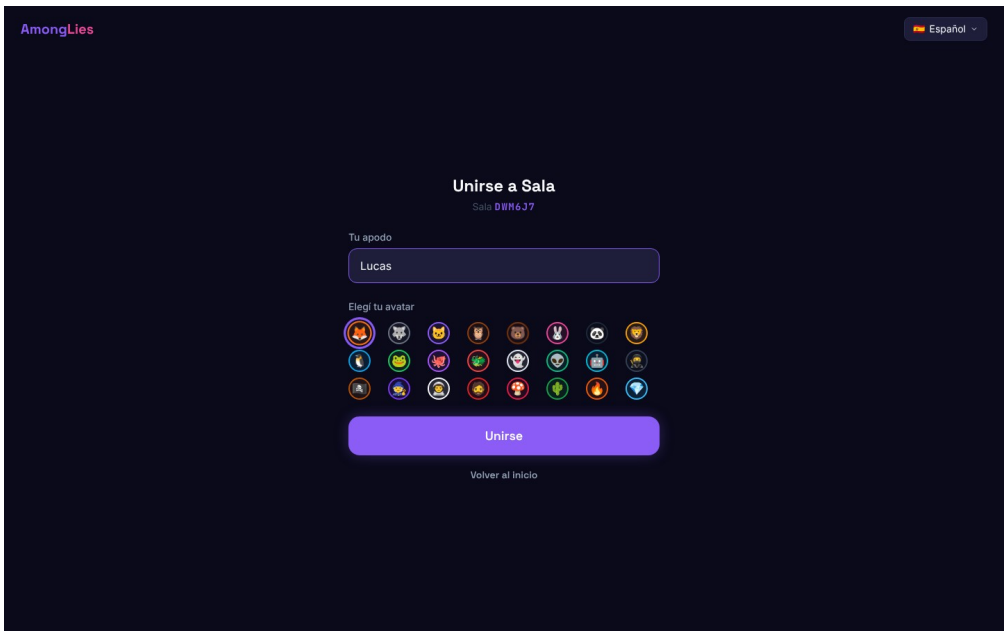


Figura 22 — Unirse desde un link de invitación

M6.5 Cómo encontrar una sala pública



Figura 23 — Salas públicas en la página de inicio

1. En la página de inicio, bajá hasta **Salas públicas**.
2. Filtrá por juego o idioma si querés. Por defecto se muestran **sólo las que se pueden unir** (en el lobby y con lugar).
3. Cada sala muestra el juego, quién la creó, el idioma, cuántos jugadores tiene (y el cupo) y si están esperando o jugando.
4. Tocá **Unirse** en la que prefieras. La lista se actualiza sola cada 5 segundos.

M6.6 Qué ve un jugador en el lobby

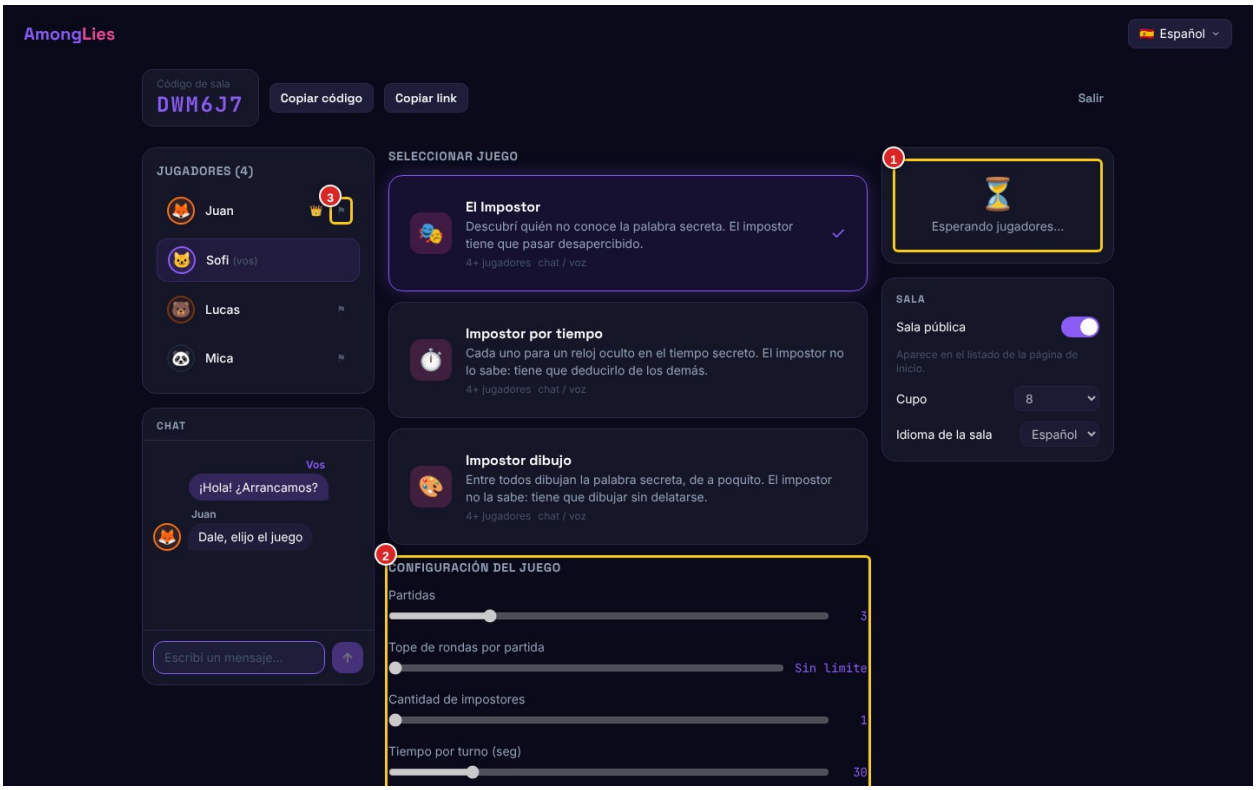


Figura 24 — Lobby visto por un jugador que no es anfitrión

1	Mensaje de espera: sólo el anfitrión puede iniciar el juego.
2	La configuración elegida por el anfitrión se ve, pero los controles están bloqueados.
3	⚑ Reportar: si alguien se porta mal, avisale a los administradores (M14).

M6.7 Cómo configurar el juego (anfitrión)

Tocá la tarjeta del juego en **Seleccionar juego** y ajustá los parámetros. Los cambios se ven al instante en la pantalla de todos. Los tres juegos comparten estas reglas:

Parámetro	Rango	Por defecto	Qué controla
Partidas	1 – 10	3	Cuántas partidas tiene el juego (los impostores cambian en cada una).
Tope de rondas por partida	0 – 20	Sin límite	Si se llega al tope sin que nadie gane, ganan los impostores.
Cantidad de impostores	1 – 5	1	Impostores por partida (siempre bastante menos que los jugadores).
Tiempo de discusión	0 – 300 s	120 s	Duración del debate. Con 0 se saltea la discusión.
Tiempo de votación	10 – 120 s	30 s	Tiempo para votar.
Voto secreto	Sí / No	No	En los resultados sólo se ve el recuento, no quién votó a quién.
Permitir votar "saltear"	Sí / No	Sí	Se puede votar no expulsar a nadie.

Parámetro	Rango	Por defecto	Qué controla
Si hay empate	Nadie sale / Re-votar / Al azar	Nadie sale	Qué pasa si dos o más empatan en votos.
El impostor ve la categoría	Sí / No	No	Le da al impostor una pista (por ejemplo, "Animales").
Modo de comunicación	Chat / Voz	Chat	Si se juega escribiendo o hablando en voz alta.

Tabla 40 — Reglas comunes de los tres juegos

Cada juego suma sus propios ajustes: en El Impostor, el **tiempo por turno** y el **tiempo para ver la palabra**; en el Impostor por tiempo, el **rango del tiempo objetivo** (M9); en el Impostor dibujo, la **tinta por turno** y los **colores** (M8). Las **listas de palabras** se eligen abajo: las del juego (8 categorías) y, si tenés cuenta, las tuyas del workshop. En el Impostor dibujo sólo aparecen las que se pueden dibujar.

Con cuenta, también podés **guardar estos ajustes como preset** y aplicarlos en otra sala con un clic:

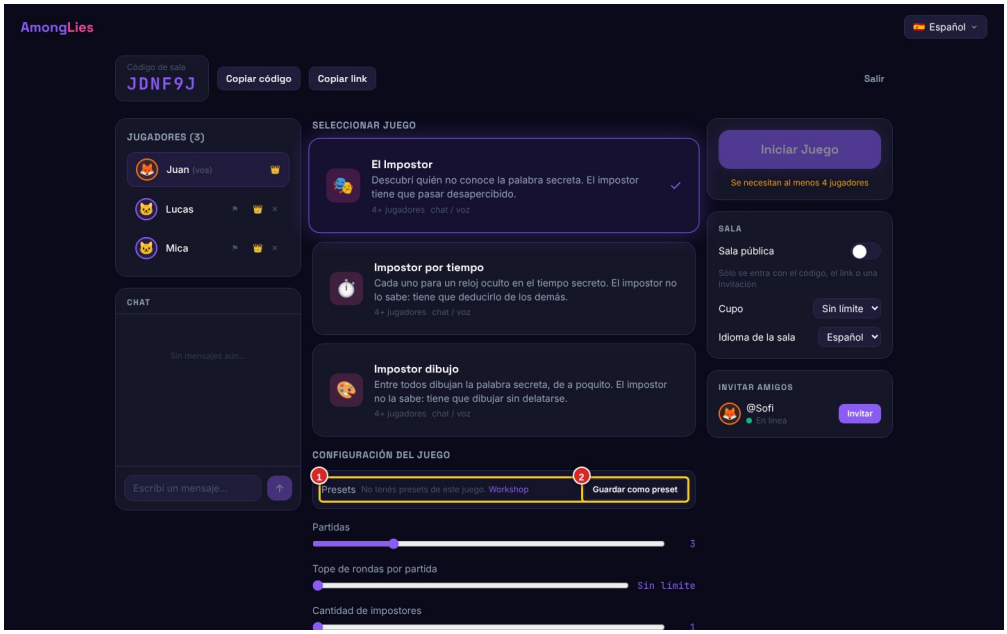


Figura 25 — Presets: aplicar uno guardado (1) o guardar los ajustes actuales (2)

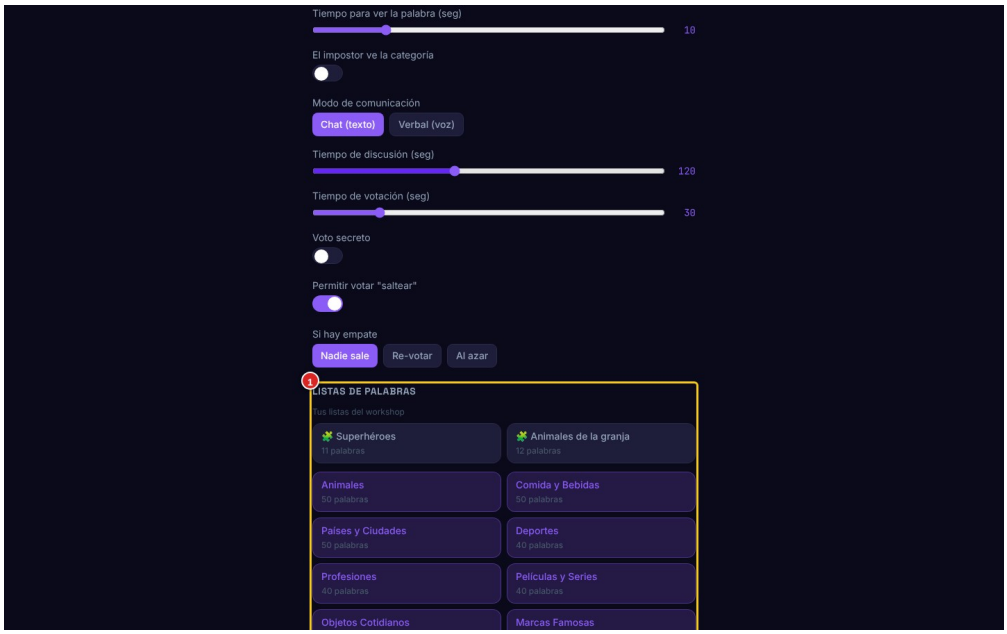


Figura 26 — Listas de palabras: tus listas del workshop aparecen arriba de las del juego

M6.8 Cómo gestionar jugadores (anfitrión)

- **Expulsar**: tocá ✕ junto al jugador. Vuelve a la pantalla de inicio.
- **Pasar la corona**: tocá 👑 junto al jugador. Pasa a ser el anfitrión y vos un jugador más.
- Si el anfitrión se va, la corona pasa automáticamente a otro jugador. Si se **desconecta** en medio de una partida, mientras tanto decide el jugador conectado más antiguo.
- **Reportar**: tocá ⚑ junto al jugador (M14).

M6.9 Cómo usar el chat

En la computadora el chat está siempre visible: en el lobby a la izquierda y durante el juego en una columna a la derecha. Escribí y presioná **Enter** o el botón de enviar. Los mensajes tienen hasta 200 caracteres y los largos se cortan en varias líneas. Los insultos se reemplazan por asteriscos. Entre rondas aparecen separadores como "Partida 1 · Ronda 2".

En el celular, durante el juego, el chat se abre con el **botón flotante** 💬 (abajo a la derecha). Un globo rojo indica cuántos mensajes no leíste. Se cierra con la ✕.

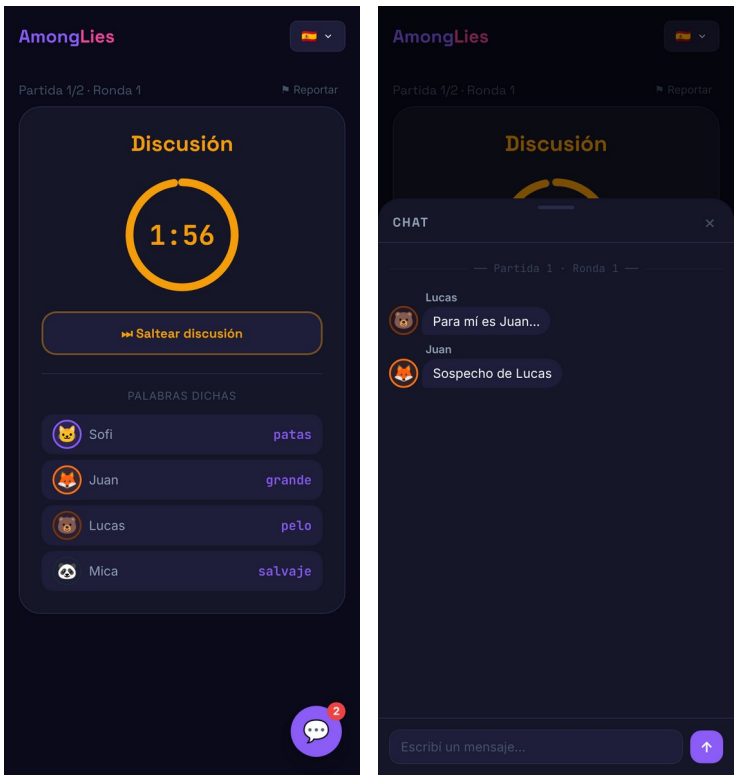


Figura 27 — Celular: botón flotante con mensajes no leídos (izq.) y panel de chat abierto (der.)

M6.10 Cómo salir de una sala

Tocá **Salir** arriba a la derecha del lobby. Si eras el anfitrión, otro jugador recibe la corona. Cuando la última persona se va, la sala desaparece.

M7. Cómo jugar El Impostor

Objetivo y reglas

- Al empezar cada ronda, todos reciben la **misma palabra secreta**, excepto el **impostor**, que sólo sabe que es impostor (y, si el anfitrión lo activó, la categoría).
- Los **inocentes** ganan la partida cuando **expulsan a todos los impostores**.
- Los **impostores** ganan cuando **quedan tantos impostores como inocentes**, o si se llega al tope de rondas.
- Si la votación empata, según la configuración: nadie es expulsado, se vuelve a votar entre los empatados o se decide al azar.
- Si hay más de un impostor, cada impostor ve quiénes son sus compañeros.
- **Puntaje**: inocente que vota a un impostor +2; impostor que sobrevive la ronda +2; al ganar la partida, +1 a cada inocente o +3 a cada impostor.

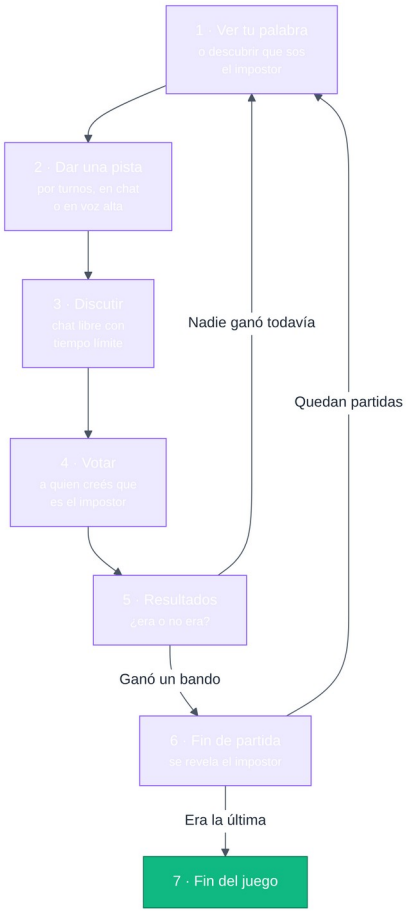


Figura 28 — Las fases de una partida (iguales en los tres juegos)

M7.1 Ver tu palabra

Al comenzar cada ronda aparece tu palabra durante unos segundos. Memorizala: después ya no se muestra. Si sos el impostor, en su lugar aparece "**¡Sos el impostor!**" y la etiqueta 🕵️ **IMPOSTOR** queda visible sólo para vos durante toda la partida.

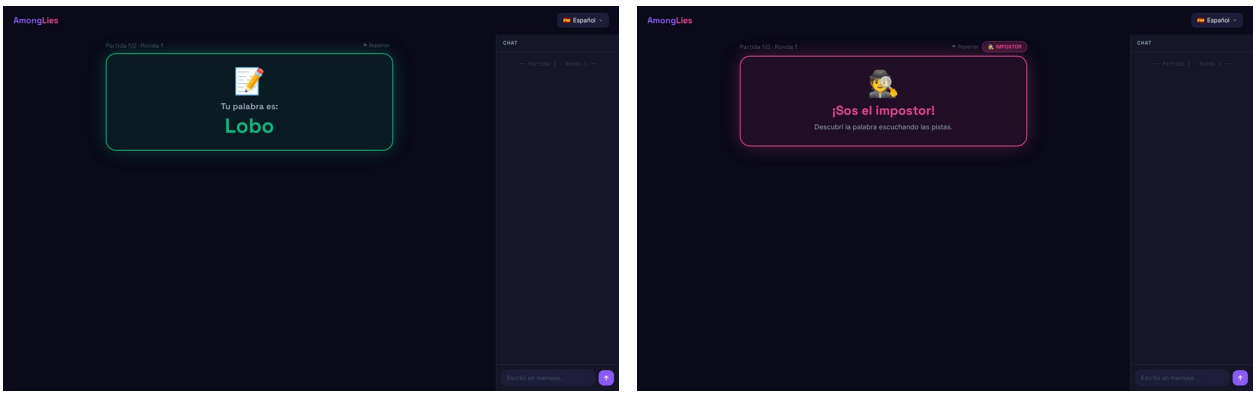


Figura 29 — Lo que ve un inocente (izq.) y lo que ve el impostor (der.)

Desde la segunda ronda, debajo de la palabra se muestran los jugadores ya expulsados en esa partida.

M7.2 Dar tu pista (modo chat)

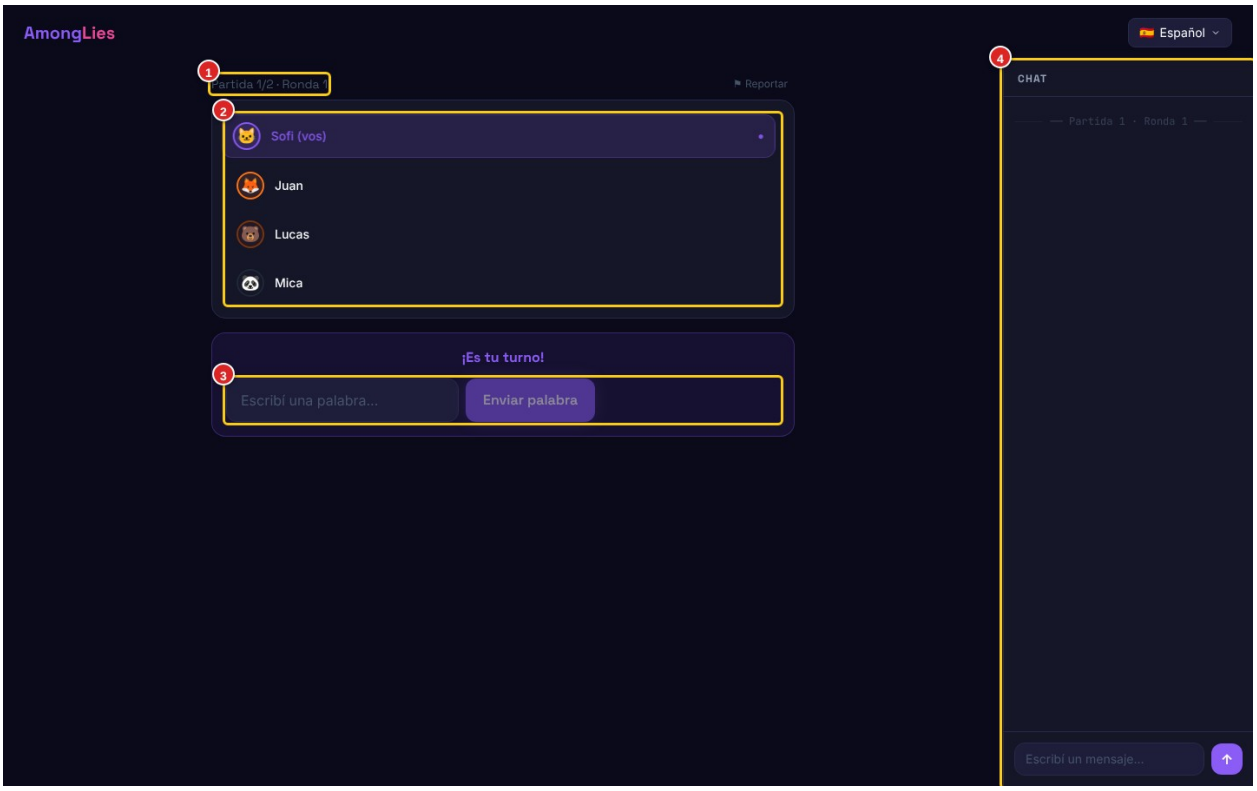


Figura 30 — Turnos en modo chat, cuando es tu turno

1	Partida y ronda actuales.
2	Orden de turnos: el jugador resaltado es quien tiene el turno; los que ya jugaron muestran su pista.
3	Cuando es tu turno: escribí una palabra relacionada con la palabra secreta y tocá **Enviar palabra**.
4	Chat de la sala, siempre disponible.

- No podés repetir una pista que ya dijo otro jugador.
- Si sos inocente, no podés escribir la palabra secreta.
- Si se acaba el tiempo del turno, se pasa al siguiente y queda registrado como "(timeout)".
- Consejo: si sos inocente, dá una pista que demuestre que sabés la palabra sin regalársela al impostor. Si sos impostor, escuchá las pistas anteriores e intentá encajar.

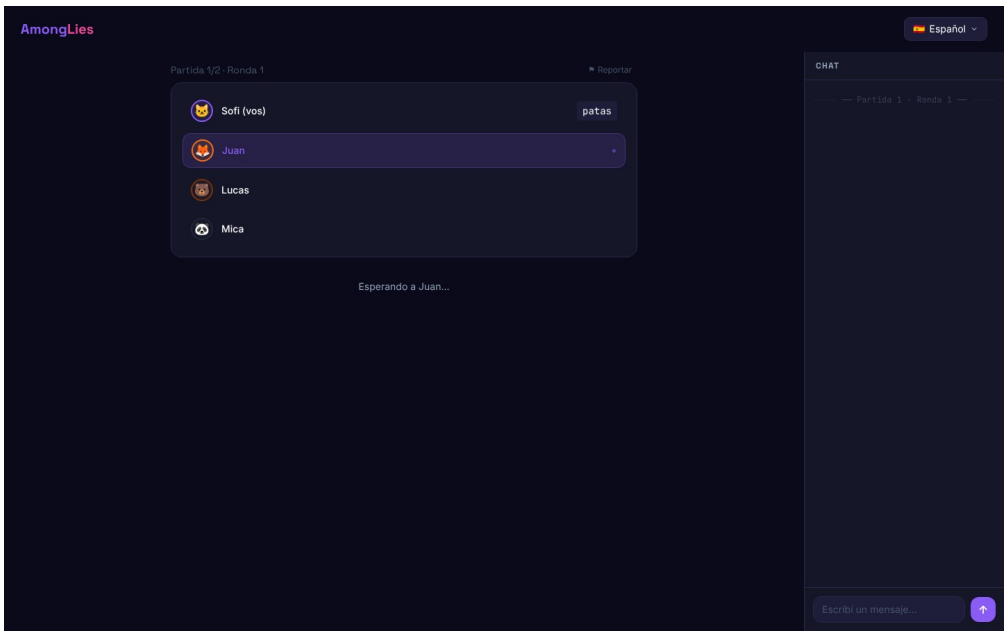


Figura 31 — Cuando no es tu turno: "Esperando a Lucas..."

M7.3 Dar tu pista (modo voz)

En modo voz la aplicación muestra en grande de quién es el turno. Esa persona dice su pista en voz alta y toca **Listo**. El anfitrión ve además un botón **Siguiente** para avanzar si alguien se olvida de tocar "Listo".

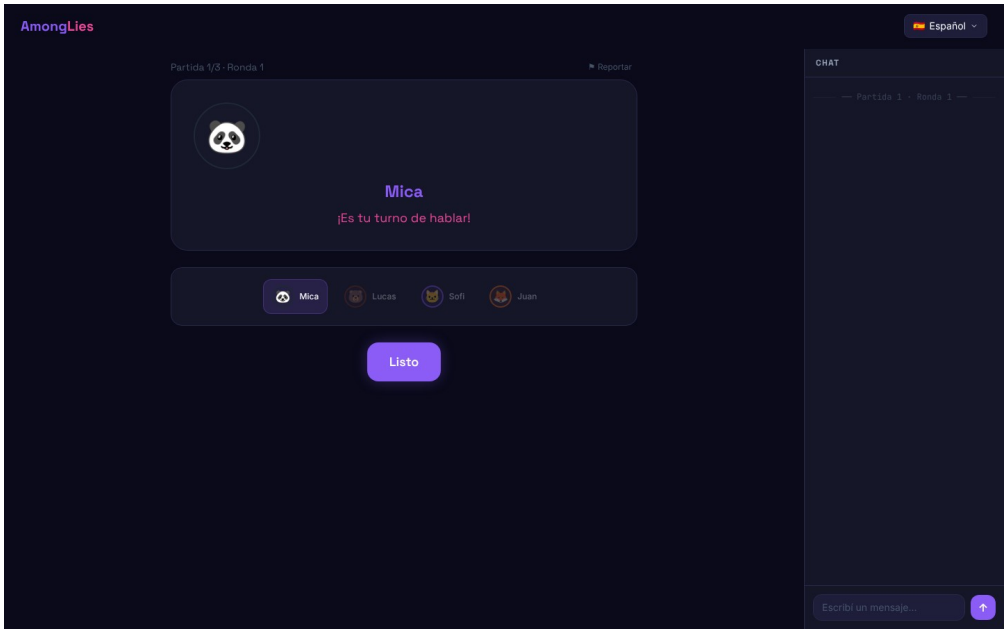


Figura 32 — Modo voz: "¡Es tu turno de hablar!"

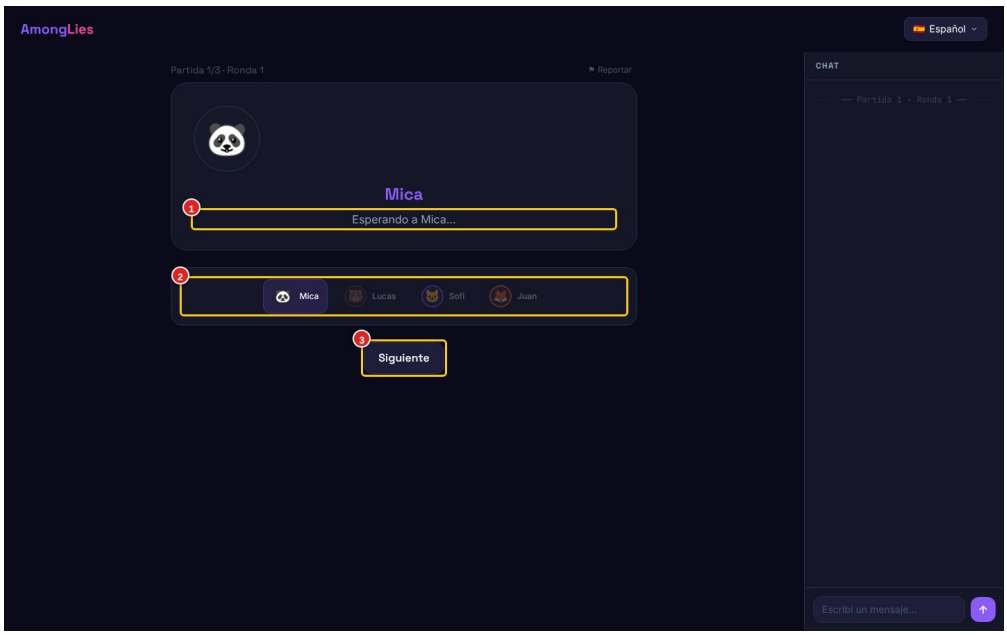


Figura 33 — Modo voz visto por el anfitrión cuando habla otro jugador

1	Jugador que tiene la palabra.
2	Orden de turnos.
3	**Siguiente** (sólo anfitrión): pasa al siguiente turno.

M7.4 Discutir

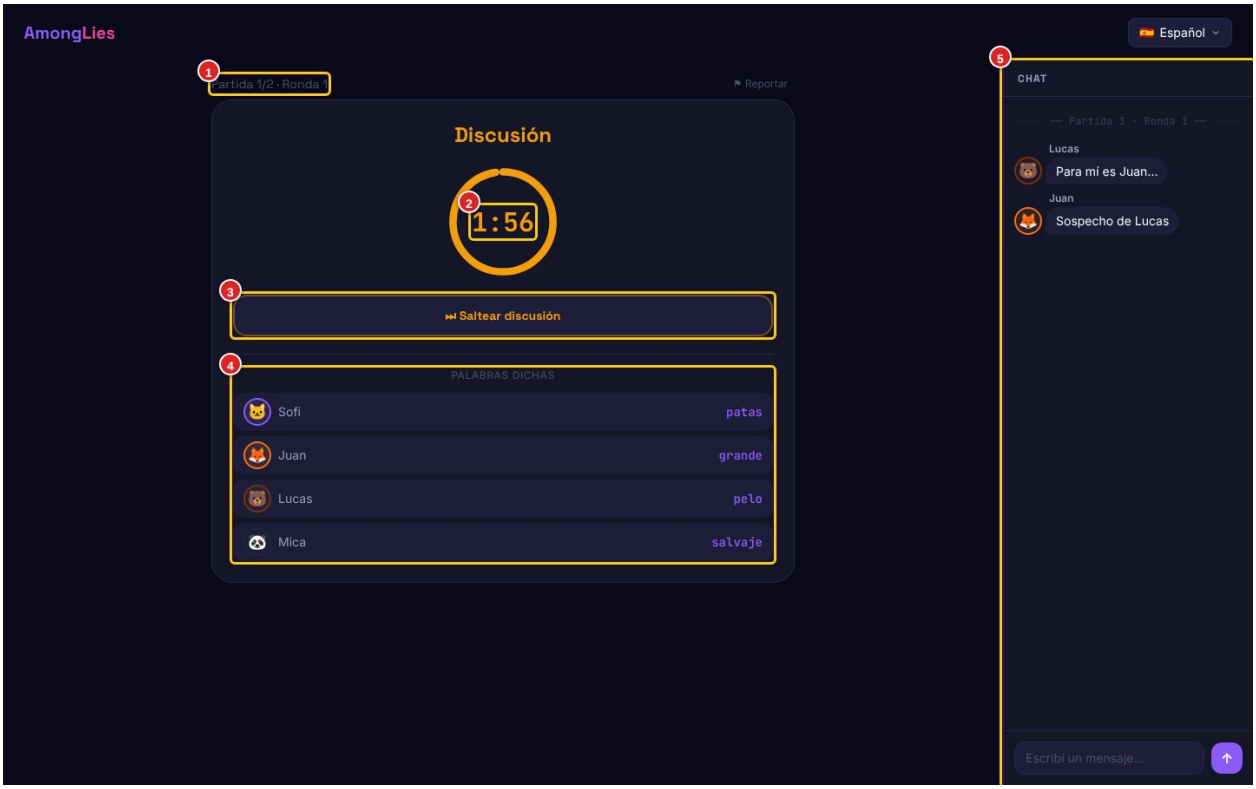


Figura 34 — Fase de discusión

1	Partida y ronda actuales.
2	Tiempo restante de discusión.

3	**Saltear discusión**: si **todos** los jugadores activos lo tocan, se pasa directo a votar.
4	Resumen de las pistas que dio cada jugador en esta ronda.
5	Chat para debatir quién es el impostor.

M7.5 Votar

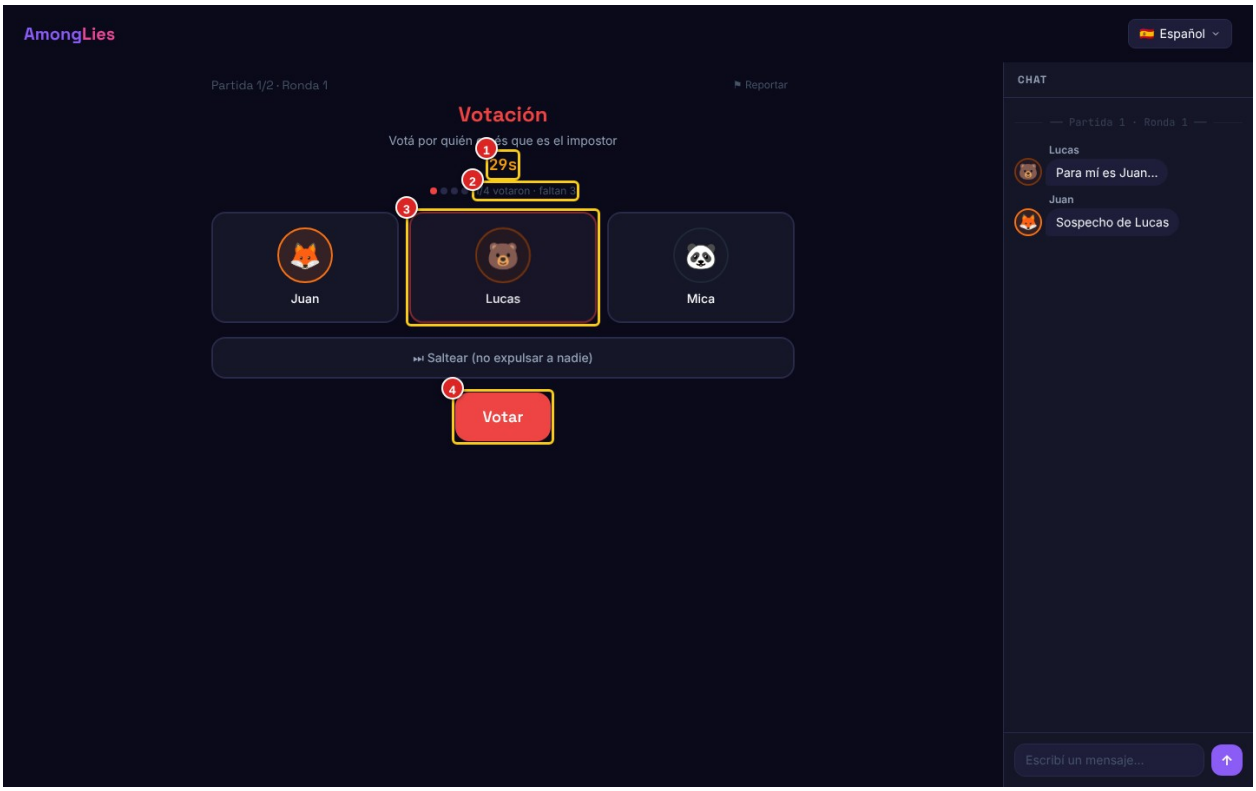


Figura 35 — Fase de votación

1	Tiempo restante para votar.
2	Cuántos votaron y cuántos faltan (sin revelar a quién votaron).
3	Tocá la tarjeta del jugador que creés que es el impostor (no aparecés vos). Abajo está "Saltear (no expulsar a nadie)", si el anfitrión lo permite.
4	Tocá **Votar** para confirmar. El voto no se puede cambiar.

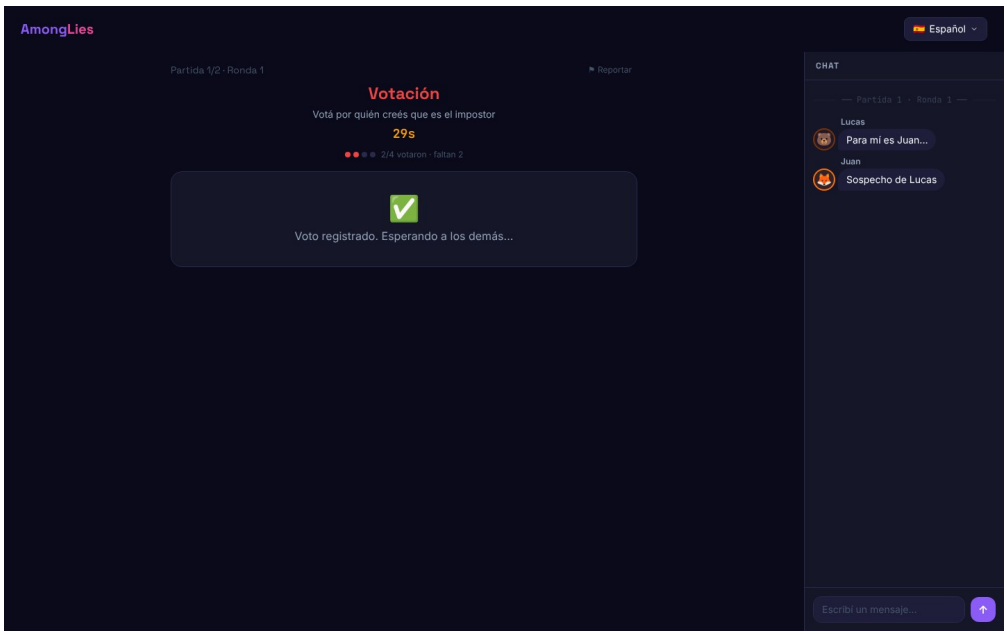


Figura 36 — Después de votar: "Voto registrado. Esperando a los demás..."

Si no votás antes de que se acabe el tiempo, tu voto cuenta como abstención.

M7.6 Ver los resultados

La revelación es escalonada para dar suspenso: primero quién fue expulsado, después si era o no el impostor, y por último el detalle con el recuento de votos y "quién votó a quién" (salvo con voto secreto). La palabra se revela recién cuando sale un impostor o termina la partida.

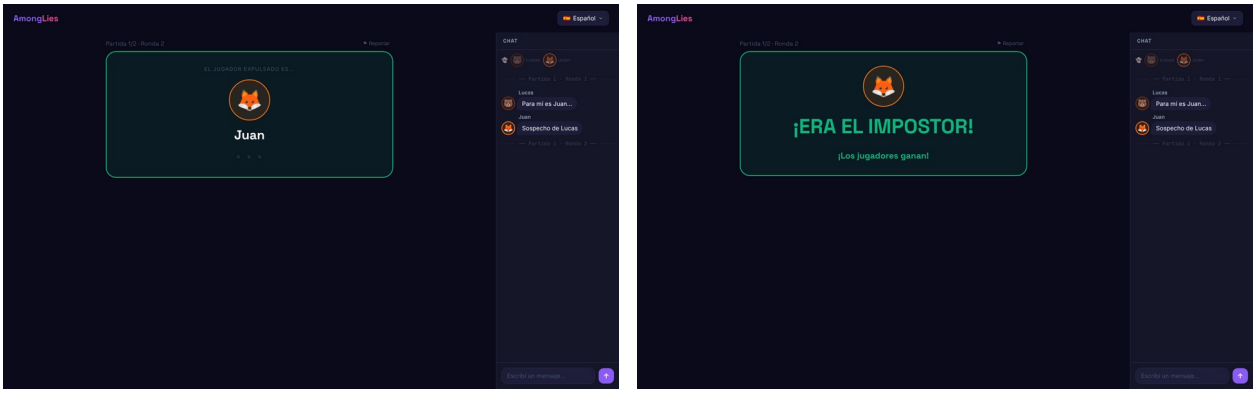


Figura 37 — Revelación: "El jugador expulsado es..." (izq.) y "¡Era el impostor!" (der.)

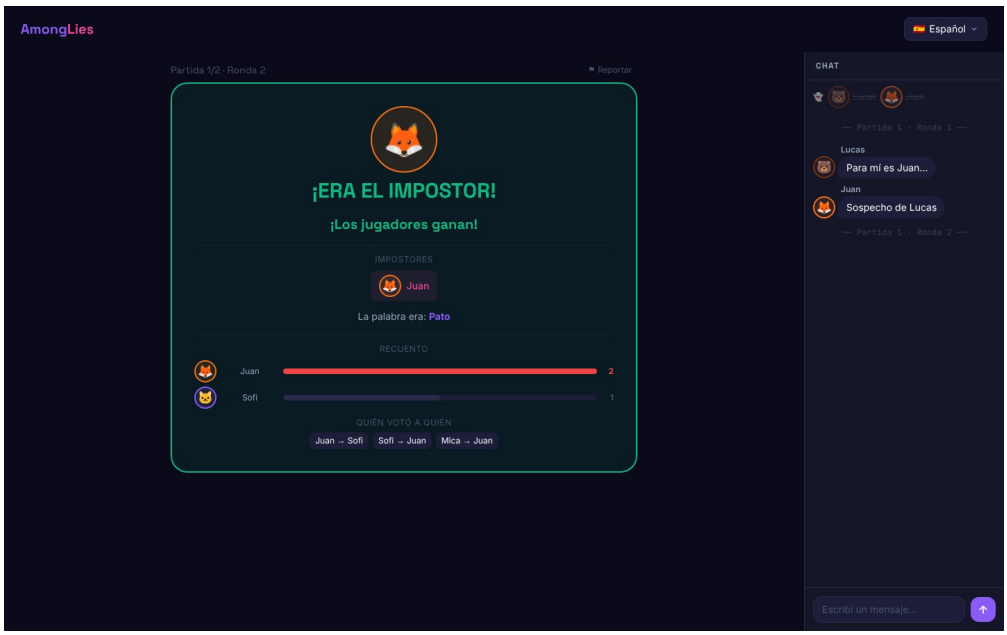


Figura 38 — Detalle: impostores, palabra, recuento y "quién votó a quién"

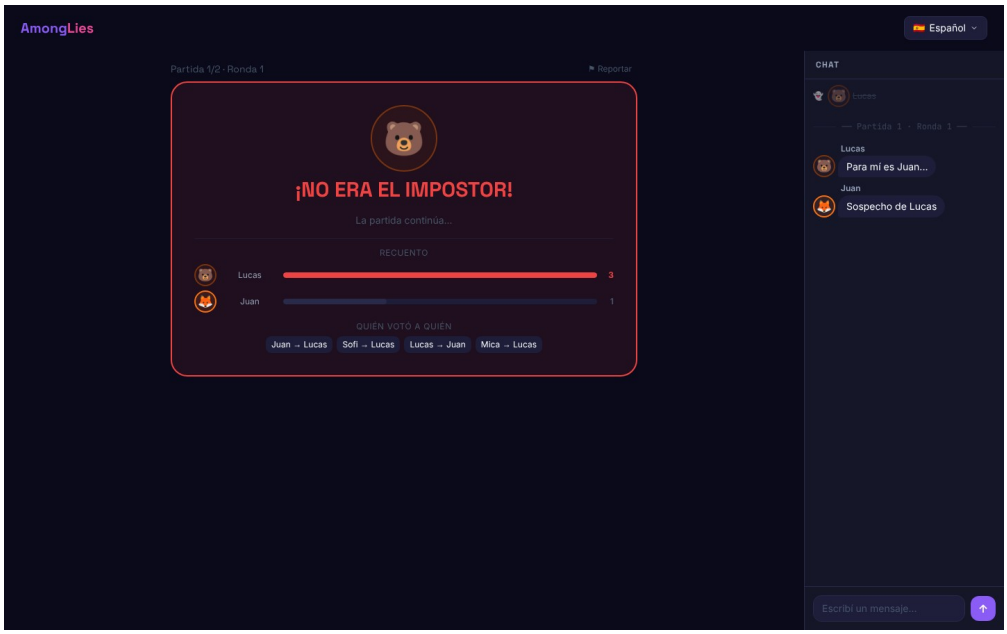


Figura 39 — Cuando el expulsado era inocente: "¡No era el impostor!" — la partida continúa

M7.7 Si te eliminan

Si te expulsan, seguís dentro de la sala como **espectador**: ves todo lo que pasa, pero no podés dar pistas, chatear ni votar. Un cartel rojo te lo recuerda. En la próxima partida volvés a jugar normalmente.

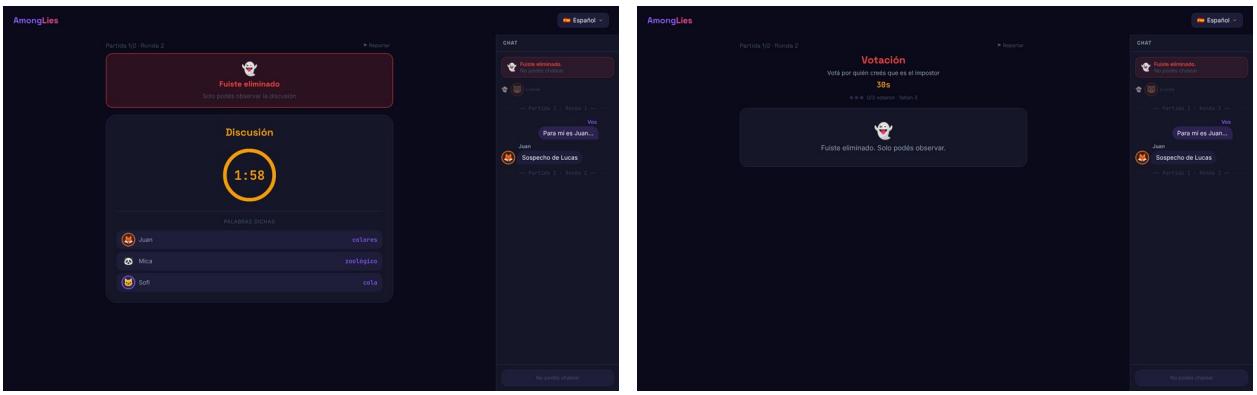


Figura 40 — Vista de un jugador eliminado durante la discusión (izq.) y la votación (der.)

M7.8 Fin de partida



Figura 41 — Pantalla de fin de partida

1	Bando ganador de la partida.
2	Se revela quién era el impostor.
3	Jugadores expulsados en esta partida.
4	Resumen de cada ronda: palabra y resultado ("Impostor libre" o "Impostor atrapado").
5	Cuenta regresiva hasta la próxima partida (15 s).
6	**Continuar ya** (más abajo, junto al **puntaje** de la partida): si todos lo tocan, la próxima partida empieza sin esperar.

M7.9 Fin del juego

Al terminar la última partida se muestra el bando ganador, el resumen de todas las partidas y el **podio** con el puntaje final. Tocá **Volver al Lobby** para jugar otra vez con la misma sala; el anfitrión puede cambiar la configuración o el juego. Si tenés cuenta, tus estadísticas se actualizan en ese momento.

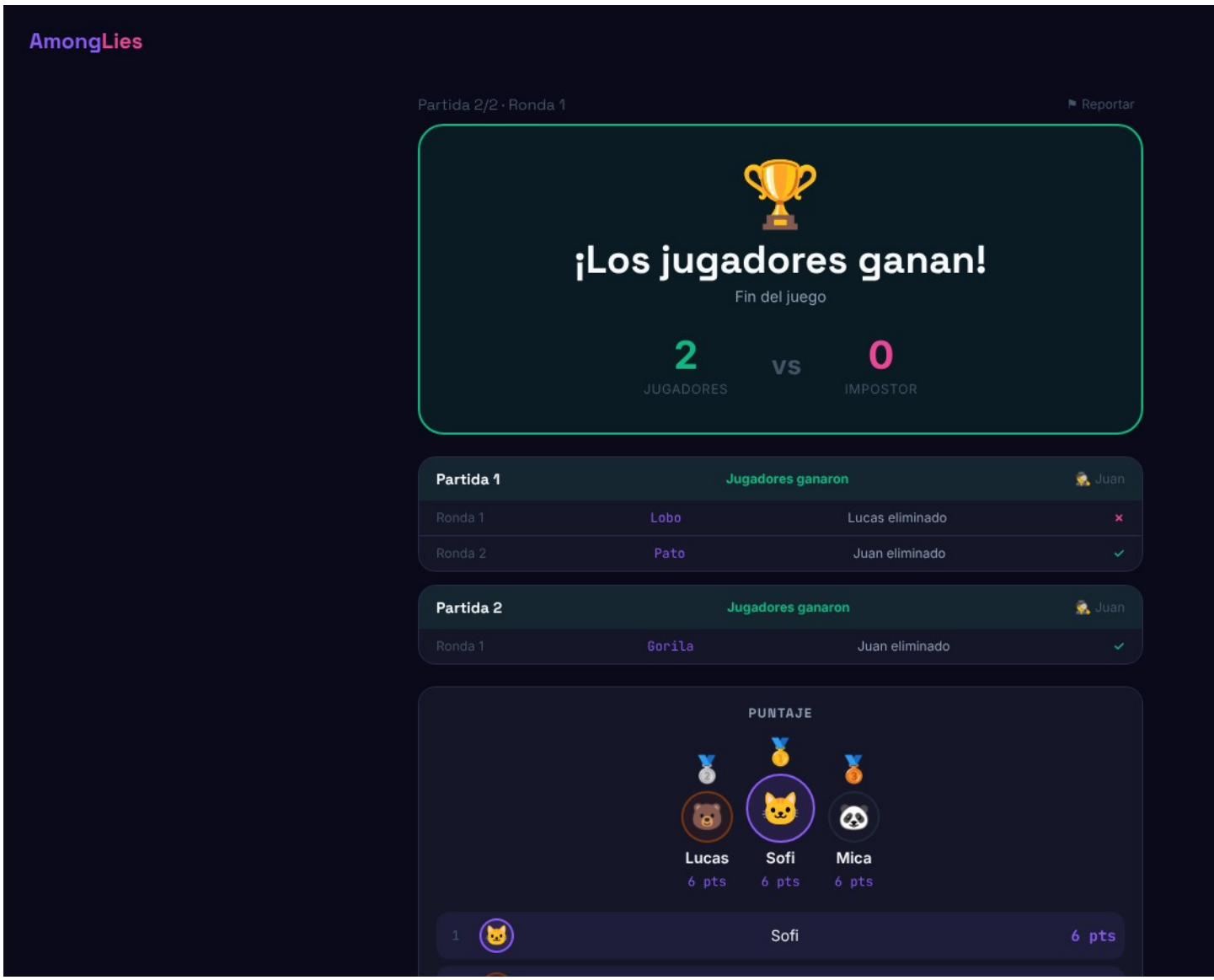


Figura 42 — Fin del juego con el podio

M8. Cómo jugar Impostor dibujo

Reglas

- En cada partida hay **una palabra secreta y un solo lienzo**. Los inocentes ven la palabra ("Dibujá: Enfermero"); el impostor no.
- Por turnos, cada jugador agrega un poco al dibujo. La **tinta es limitada**: por defecto alcanza para una línea de lado a lado del lienzo, así que cada trazo cuenta.
- Cuando todos dibujaron se discute y se vota, como en El Impostor. Si nadie gana, la ronda siguiente **se sigue dibujando sobre el mismo dibujo**.
- El impostor tiene que sumar algo que encaje sin saber qué es. Los inocentes, dibujar lo suficiente para que se note que saben, sin regalarle la palabra al impostor.
- La palabra se revela al terminar la partida, junto al dibujo terminado.

Cómo dibujar

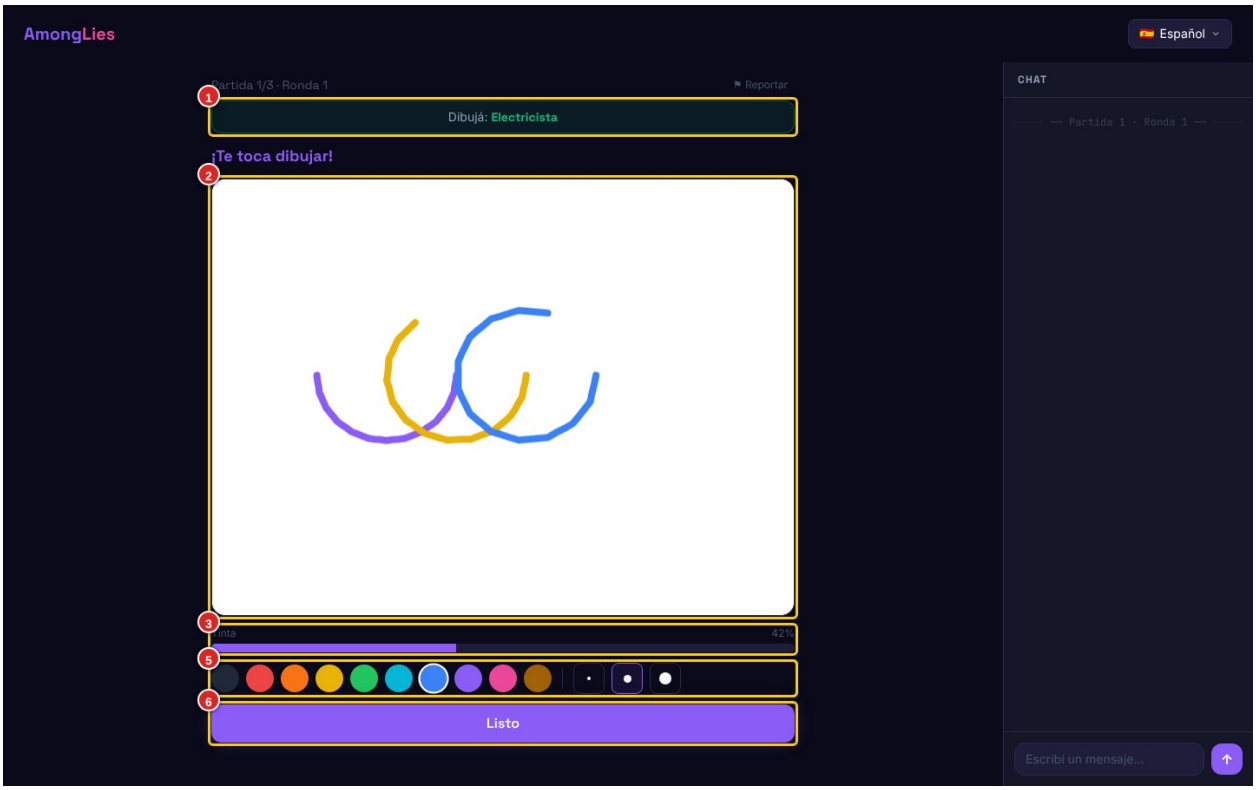


Figura 43 — Tu turno de dibujar

1	La palabra que hay que dibujar (el impostor ve "¡Sos el impostor!").
2	El lienzo: dibujá arrastrando el mouse o el dedo.
3	Tinta que te queda en este turno.
4 y 5	Color (10 colores; el anfitrión puede fijar uno por jugador) y grosor del pincel (fino, mediano o grueso).
6	**Listo**: terminás tu turno. Se habilita cuando usaste un mínimo de tinta.

1. Elegí un color y un grosor.
2. Dibujá sobre el lienzo; la barra de tinta baja mientras dibujás y, cuando se acaba, se termina tu turno.
3. Tocá **Listo** cuando quieras pasar el turno. No hay "deshacer": pensá bien cada trazo.

Mientras dibuja otro, ves sus trazos aparecer **en vivo**. En la votación y en los resultados el dibujo sigue a la vista.

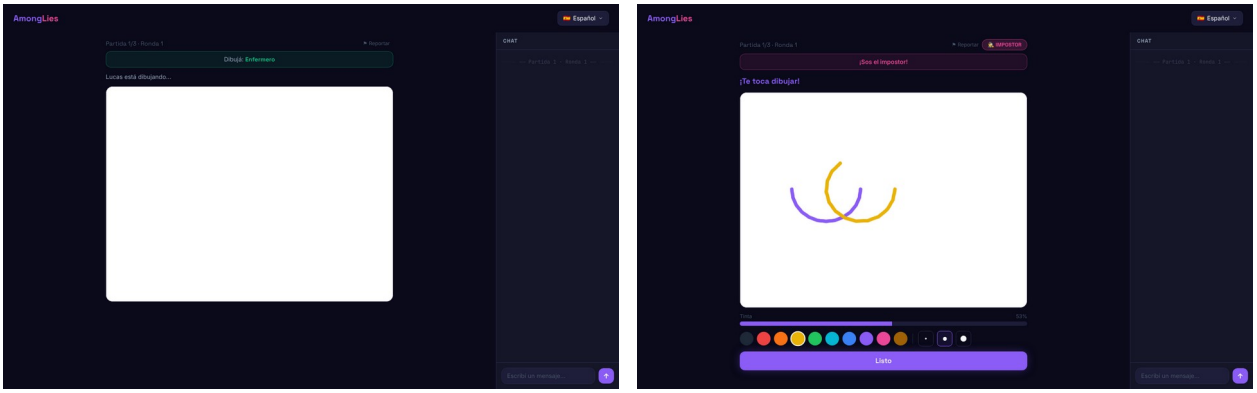


Figura 44 — Mirando dibujar a otro (izq.) y el turno del impostor, que no sabe la palabra (der.)

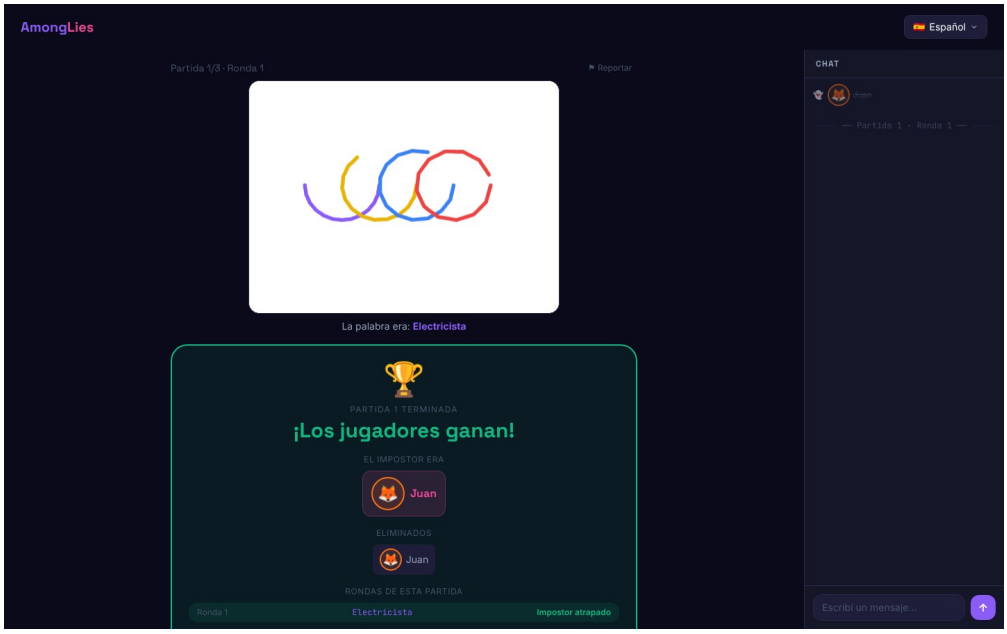


Figura 45 — Fin de partida: el dibujo terminado y la palabra

Ajustes propios

Parámetro	Rango	Por defecto	Qué controla
Veces que dibuja cada uno por ronda	1 – 3	1	Cuántas vueltas de dibujo hay antes de discutir.
Tinta por turno	20 % – 1000 %	100 %	Cuánto se puede dibujar (100 % = una línea de lado a lado).
Tinta mínima para terminar	0 % – 100 %	20 %	Cuánto hay que dibujar antes de poder tocar Listo.
Tiempo máximo por turno	10 – 120 s	30 s	Si se acaba, el turno termina solo.
Colores	Libres / Uno por jugador	Libres	Con "uno por jugador" se puede ver quién dibujó qué.

Tabla 41 — Ajustes del Impostor dibujo

M9. Cómo jugar Impostor por tiempo

Reglas

- Cada ronda hay un **tiempo objetivo** secreto (por ejemplo, 16,0 segundos). Los inocentes lo ven al empezar; el impostor no.
- En tu turno tocás **EMPEZAR**: corre un reloj **que nadie ve**, ni siquiera vos. Tocás **PARAR** cuando creés que pasó el tiempo objetivo.
- Apenas parás, todos ven tu tiempo. Los inocentes van a estar cerca del objetivo; el impostor tiene que adivinarlo mirando los tiempos de los que jugaron antes.
- Después se discute y se vota, como en El Impostor. El tiempo objetivo se revela en los resultados.
- Si no arrancás o no parás antes del tiempo máximo por turno, queda registrado como "se le pasó".

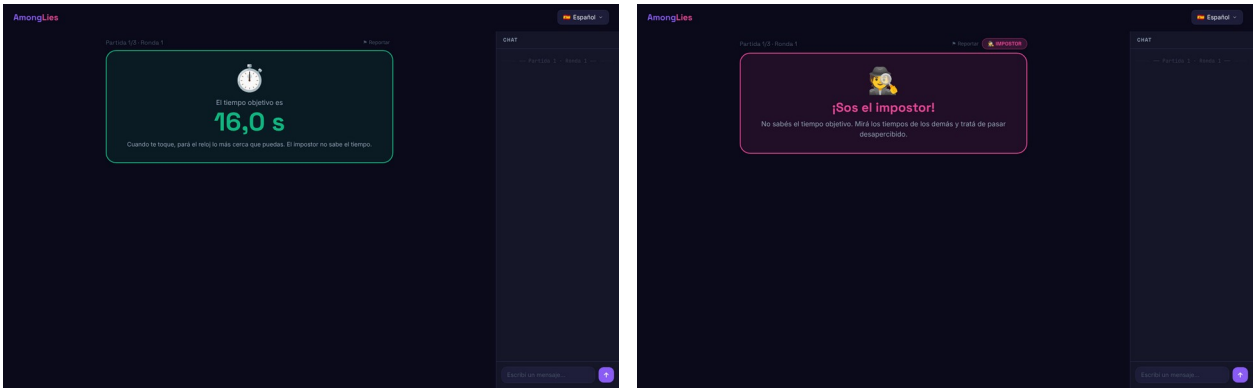


Figura 46 — Al empezar: el inocente ve el tiempo objetivo (izq.); el impostor no (der.)

Tu turno

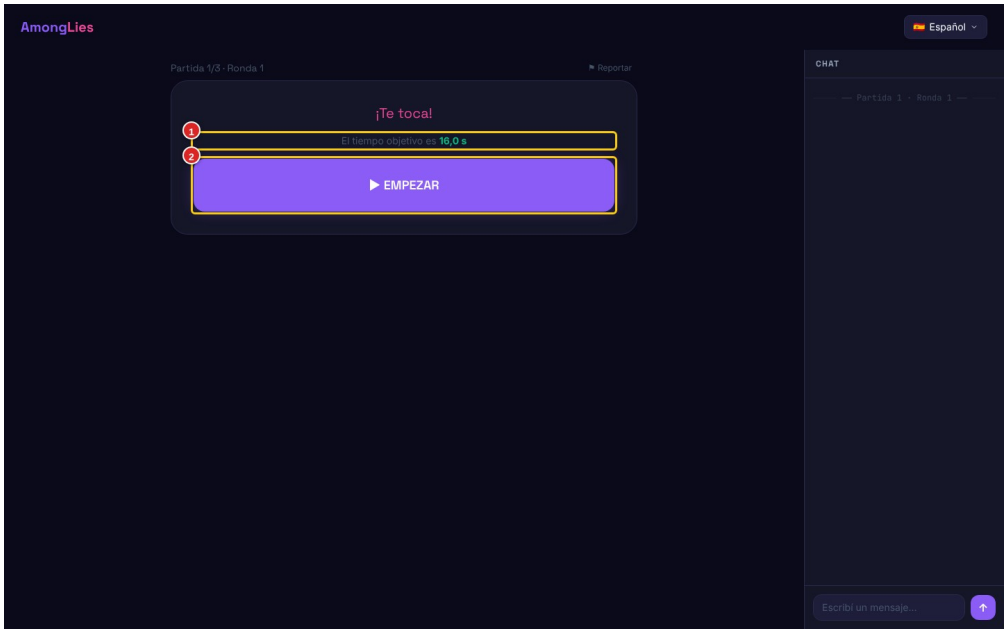


Figura 47 — Tu turno

1	El tiempo objetivo (sólo si sos inocente).
2	**EMPEZAR**: arranca el reloj oculto. Después el botón pasa a **PARAR**.

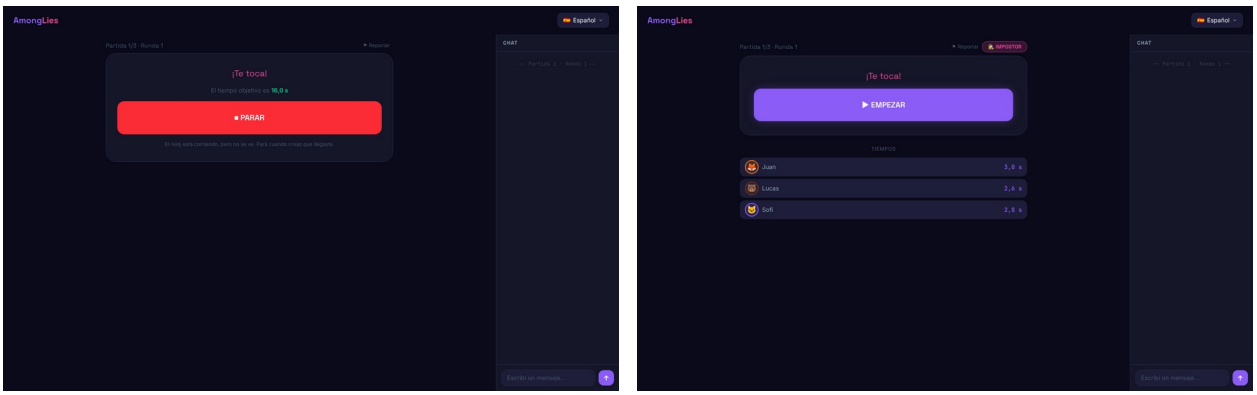


Figura 48 — El reloj corriendo sin verse (izq.) y el turno del impostor, que sólo ve los tiempos de los demás (der.)

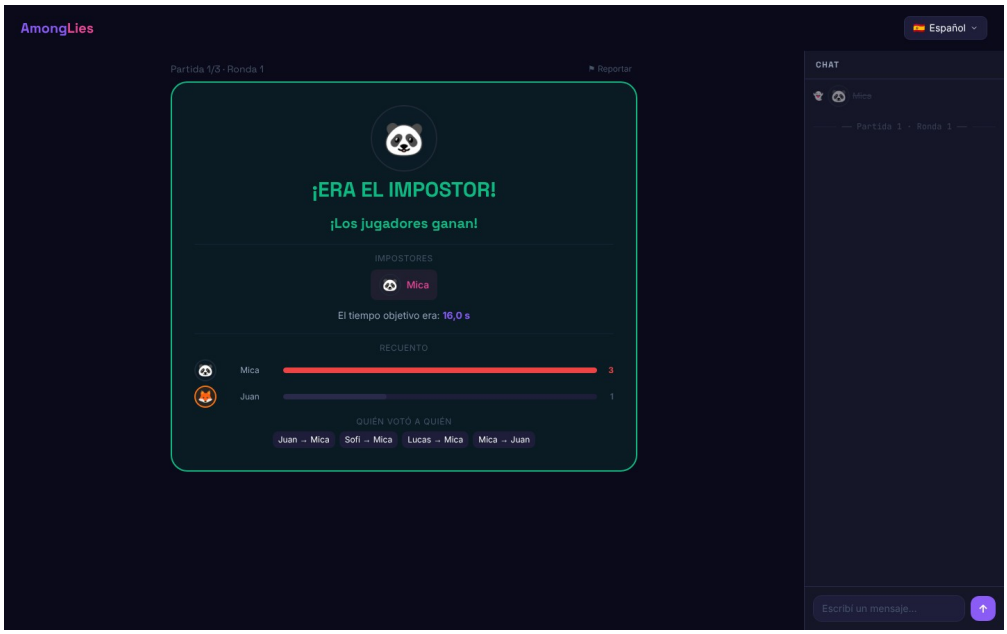


Figura 49 — Resultados: el impostor expulsado y el tiempo objetivo revelado

Ajustes propios

Parámetro	Rango	Por defecto	Qué controla
Tiempo objetivo mínimo	2 – 30 s	5 s	El tiempo objetivo se elige al azar entre el mínimo y el máximo.
Tiempo objetivo máximo	3 – 60 s	20 s	
Tiempo máximo por turno	10 – 120 s	30 s	Si no parás antes, "se le pasó".
Tiempo para ver tu rol	3 – 30 s	8 s	Cuánto se muestra el rol y el tiempo objetivo al empezar.

Tabla 42 — Ajustes del Impostor por tiempo

M10. Cómo ver tu perfil y estadísticas

> **Sólo para usuarios registrados**
> Las partidas que jugás como invitado no se guardan en ningún perfil.

1. Iniciá sesión (M5.2).
2. Tocá tu **@usuario** arriba a la derecha.

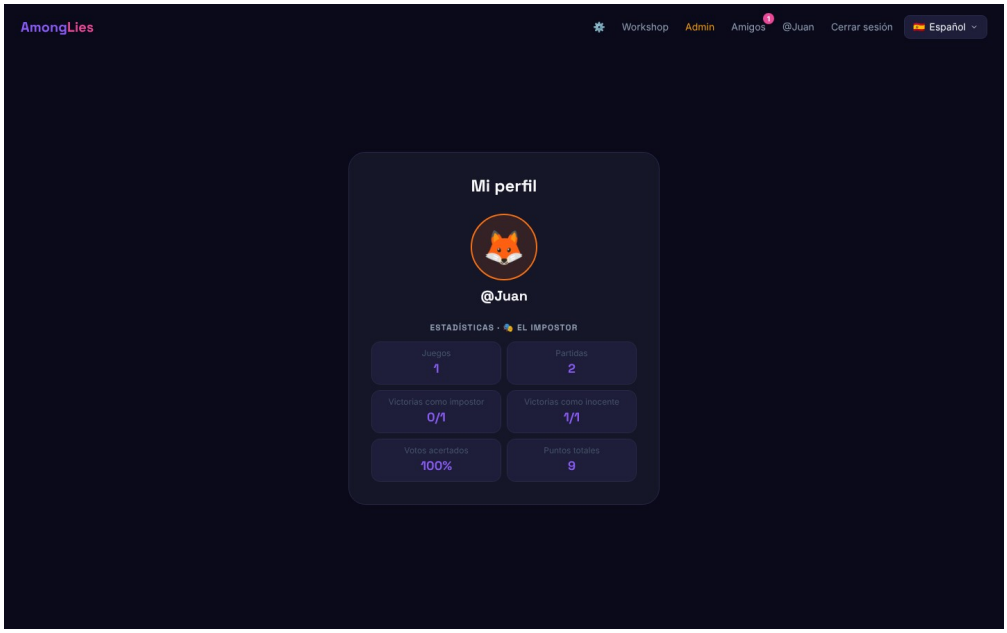


Figura 50 — Mi perfil con las estadísticas de El Impostor

Hay un bloque de estadísticas por cada juego que jugaste:

- **Juegos** y **partidas** jugadas.
- **Victorias como impostor** y **como inocente** (por ejemplo, 1/1 = ganaste la única partida que jugaste en ese rol).
- **Votos acertados**: de tus votos como inocente, cuántos fueron a un impostor.
- **Puntos totales** sumados en todos tus juegos.

Las estadísticas se actualizan automáticamente al terminar cada juego.

M11. Amigos e invitaciones

> **Sólo para usuarios registrados**
> Los invitados ven en "Amigos" que necesitan una cuenta.

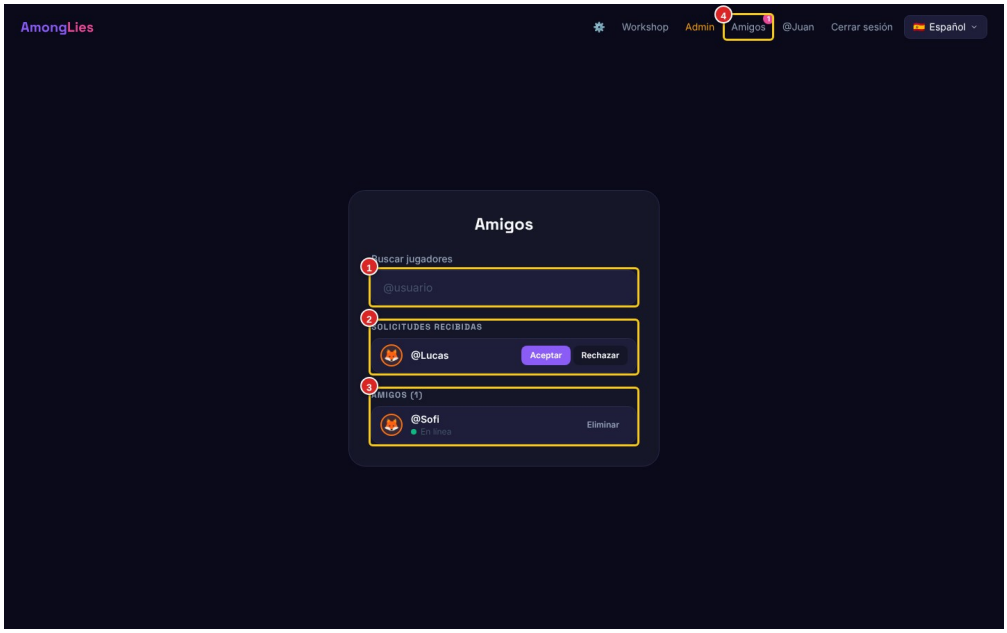


Figura 51 — La página de Amigos

1	Buscar jugadores por su @usuario y tocar **Agregar** para mandarle una solicitud.
2	Solicitudes que te mandaron: **Aceptar** o **Rechazar**. Las que mandaste aparecen en "Solicitudes enviadas".
3	Tus amigos con su estado: desconectado, en línea, **en una sala** (con el botón **Unirme**) o jugando. **Eliminar** lo saca de tu lista.
4	**Amigos** en el menú, con un globo cuando tenés solicitudes nuevas.

Cuando alguien te manda una solicitud, te aparece un aviso abajo a la derecha (con **Ver**). Si los dos se mandan solicitud, quedan amigos directamente.

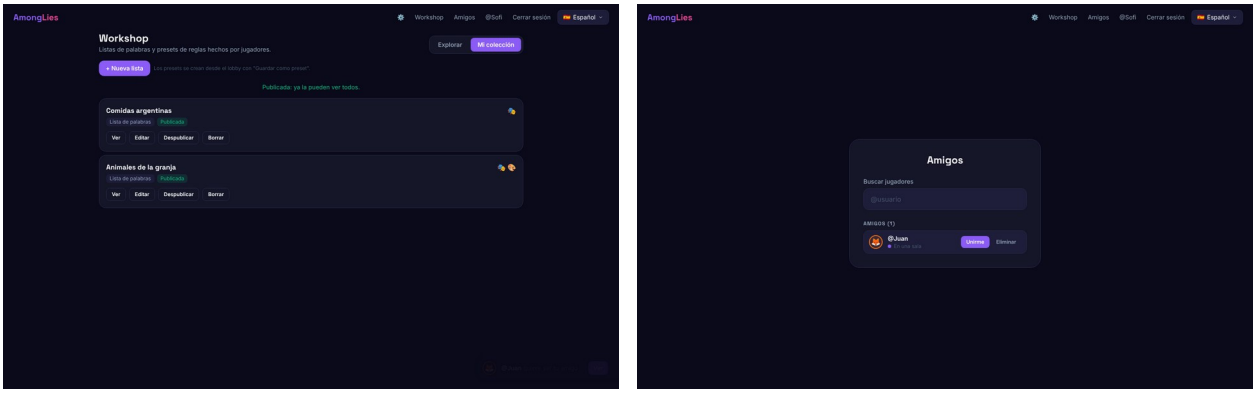


Figura 52 — Aviso de solicitud nueva (izq.) y un amigo que está en una sala, con "Unirme" (der.)

M11.1 Cómo invitar a un amigo a tu sala

1. Creá una sala (o entrá a una).
2. En el lobby, en **Invitar amigos**, aparecen tus amigos en línea. Tocá **Invitar**.

3. A tu amigo le llega un aviso "te invitó a su sala" con el botón **Unirme**, que lo lleva directo.

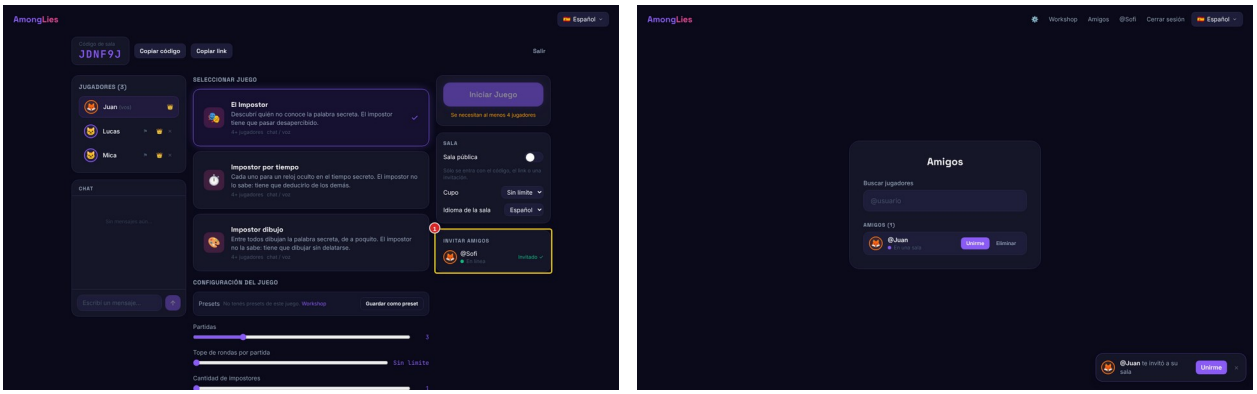


Figura 53 — Invitar desde el lobby (izq.) y el aviso que recibe tu amigo (der.)

Las invitaciones vencen a los 5 minutos y sólo te pueden invitar tus amigos. En **Ajustes** podés aparecer desconectado o no aceptar invitaciones (M13).

M12. Workshop: listas de palabras y presets

El **workshop** es donde los jugadores comparten **listas de palabras** propias (para jugar con palabras distintas a las del juego) y **presets** (un juego con todos sus ajustes). Cualquiera puede verlo; para crear, guardar y dar like hace falta una cuenta.

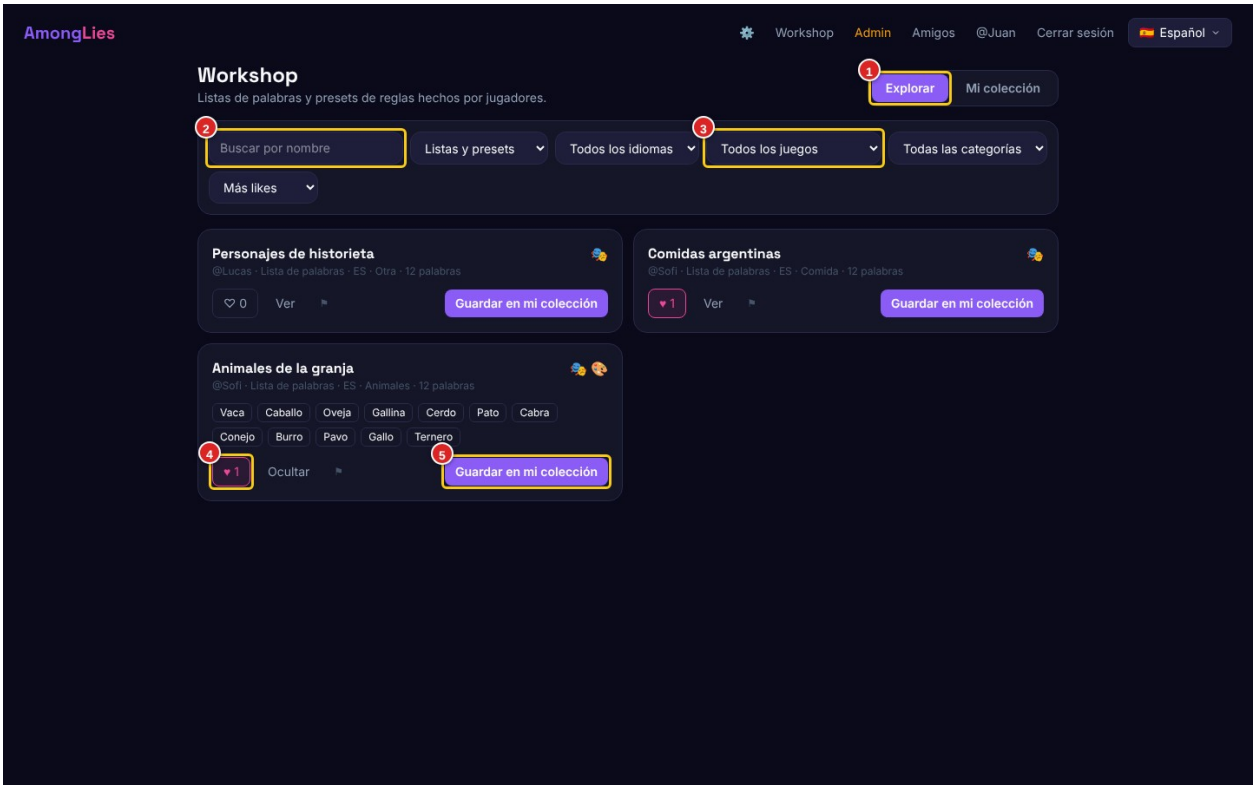


Figura 54 — Explorar el workshop

1	**Explorar** (lo publicado por todos) y **Mi colección** (lo tuyo y lo que guardaste).
2	Buscar por nombre.
3	Filtros: tipo (listas o presets), idioma, juego, categoría y orden (más likes o más nuevos).
4	**Me gusta** ❤️ con el contador de likes. **Ver** muestra las palabras.
5	**Guardar en mi colección**: guarda una copia que podés usar y editar.

M12.1 Cómo crear una lista de palabras

1. Entrá a **Workshop → Mi colección → + Nueva lista**.
2. Poné un nombre, el idioma de las palabras y una categoría. Marcá **Se pueden dibujar** si son cosas concretas (así sirve para el Impostor dibujo).
3. Escribí las palabras, una por renglón o separadas por coma: entre 10 y 300, sin repetir.
4. Tocá **Guardar**. Queda **privada** en tu colección; tocá **Publicar** para que la vean todos.

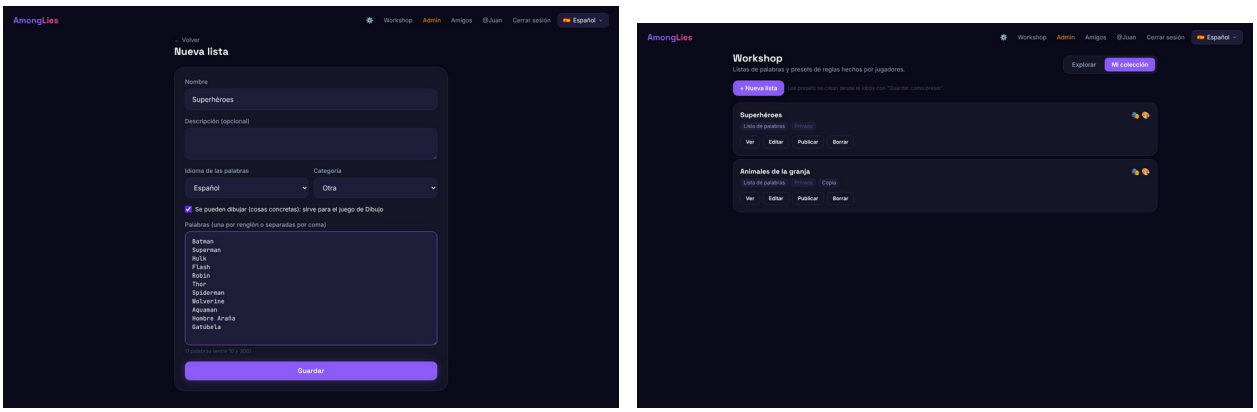


Figura 55 — Editor de listas (izq.) y Mi colección (der.)

Si guardaste una lista de otro y el autor saca una versión nueva, tu copia avisa **"Hay una versión nueva"** y la podés **Actualizar**. También podés editar tu copia: queda marcada como "Editada por vos".

M12.2 Cómo usarlo en el lobby

- **Listas**: en el lobby, tus listas aparecen arriba de las del juego (Figura 26). Los demás jugadores ven el nombre de la lista, no las palabras.
- **Presets**: elegí uno en **Presets** y tocá **Aplicar**; o tocá **Guardar como preset** para guardar los ajustes actuales en tu colección (Figura 25).

M13. Ajustes y sonido

Tocá el engranaje ⚙ arriba a la derecha (en el celular, en el menú ☰).

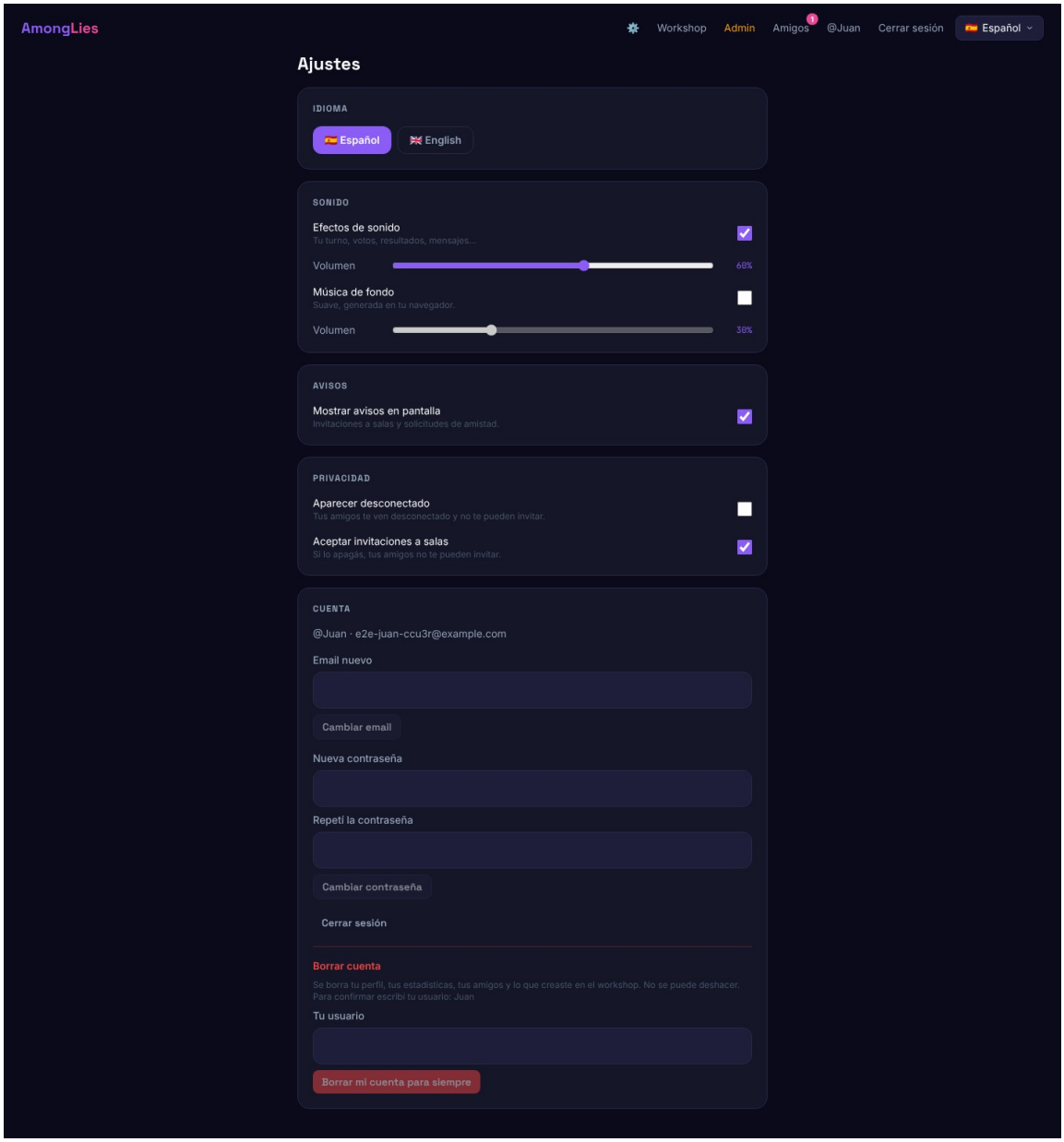


Figura 56 — Ajustes de una cuenta

Sección	Qué podés hacer
Idioma	Español o English.
Sonido	Prender o apagar los **efectos** (tu turno, mensajes, votación, resultados…) y la **música de fondo**, y regular el volumen de cada uno. Todo el sonido se genera en el navegador.
Avisos	Mostrar u ocultar los avisos en pantalla (invitaciones y solicitudes).
Privacidad (con cuenta)	**Aparecer desconectado** (tus amigos te ven desconectado y no te pueden invitar) y **Aceptar invitaciones a salas**.
Cuenta (con cuenta)	Cambiar el email o la contraseña, cerrar sesión y **borrar la cuenta**: hay que escribir tu usuario para confirmar; se borran tu perfil, tus estadísticas, tus amigos y lo que creaste en el workshop.

Tabla 43 — Ajustes

Idioma, sonido y avisos se guardan en el navegador; la privacidad, en tu cuenta.

M14. Reportar y moderación

Si alguien insulta, hace trampa o hace spam, podés **reportarlo** para que lo revise un administrador.

M14.1 Cómo reportar

1. Tocá ⚑ junto al jugador en el lobby, **Reportar** durante la partida, o ⚑ en un ítem del workshop.
2. Elegí el motivo (insultos o acoso, trampa, spam, contenido inapropiado u otro) y, si querés, contá qué pasó.
3. Tocá **Enviar reporte**. Se adjuntan los últimos mensajes del chat de la sala como evidencia.

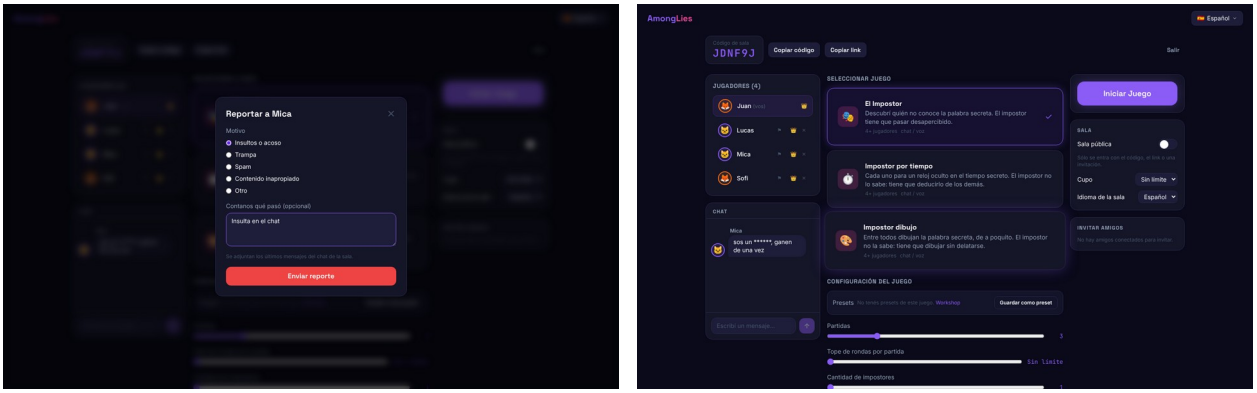


Figura 57 — Reportar a un jugador (izq.); en el chat los insultos llegan censurados (der.)

Además, los insultos se censuran solos con asteriscos en el chat y en las pistas, y no se aceptan apodos, usuarios ni listas del workshop con insultos. Se pueden enviar hasta 5 reportes cada 10 minutos.

M14.2 Panel de admin (sólo administradores)

Los administradores ven **Admin** arriba a la derecha.

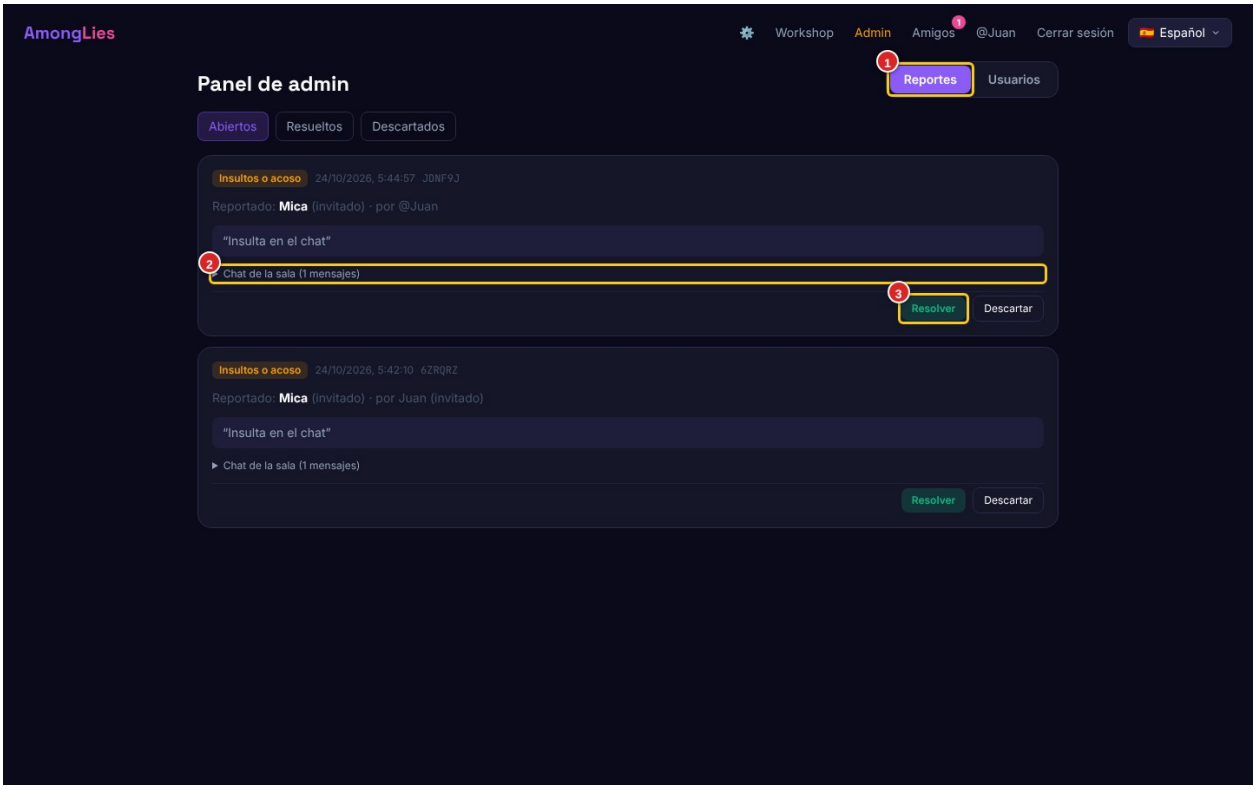


Figura 58 — Panel de admin: reportes abiertos

1	**Reportes** (abiertos, resueltos y descartados) y **Usuarios**.
2	La evidencia: el chat de la sala en el momento del reporte (con los insultos censurados).
3	**Resolver** o **Descartar** el reporte, con una nota opcional. Al lado están las acciones: **Suspender** 1, 7 o 30 días o para siempre (con un motivo que ve el usuario) y, si es del workshop, ocultar o borrar el ítem.

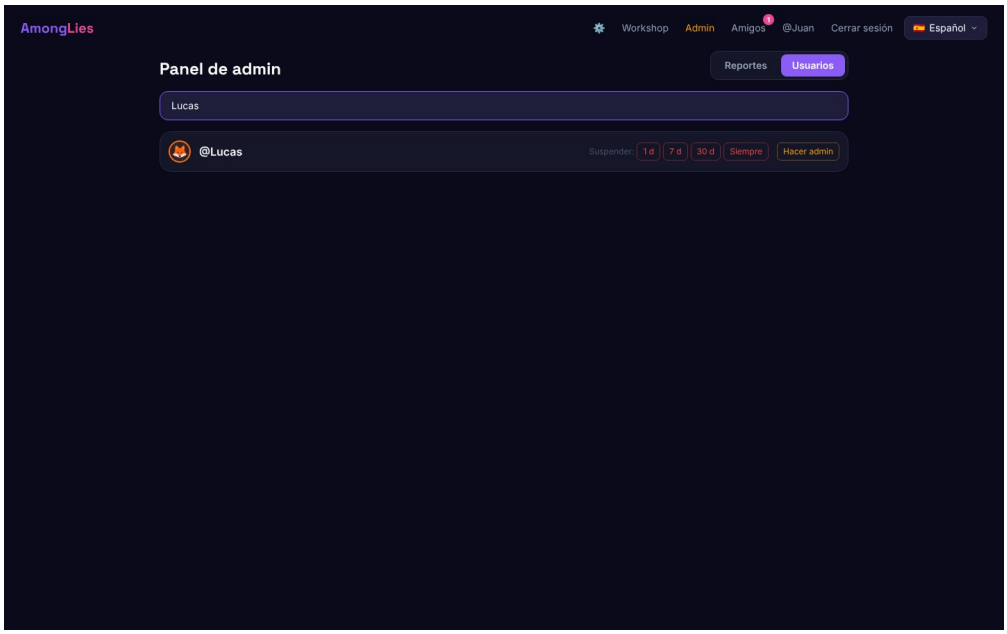


Figura 59 — Usuarios: buscar, suspender o levantar la suspensión y nombrar otros admins

Un usuario suspendido sale de su sala con un aviso y, hasta que venza la suspensión, no puede crear ni entrar a salas ni publicar en el workshop.

M15. Uso desde el celular

AmongLies está pensado también para celulares. Las diferencias principales:

- Arriba, los accesos (Workshop, Amigos, Ajustes, tu cuenta) están en el menú ☰.
- El lobby se organiza en una sola columna: "Iniciar juego" y los ajustes de la sala arriba, jugadores y chat debajo.
- Durante el juego el chat se abre con el botón flotante 💬 (M6.9).
- En el Impostor dibujo se dibuja con el dedo.
- Si bloqueás la pantalla o cambiás de aplicación, la partida te espera pausada hasta que vuelvas (o hasta que el anfitrión decida seguir sin vos).

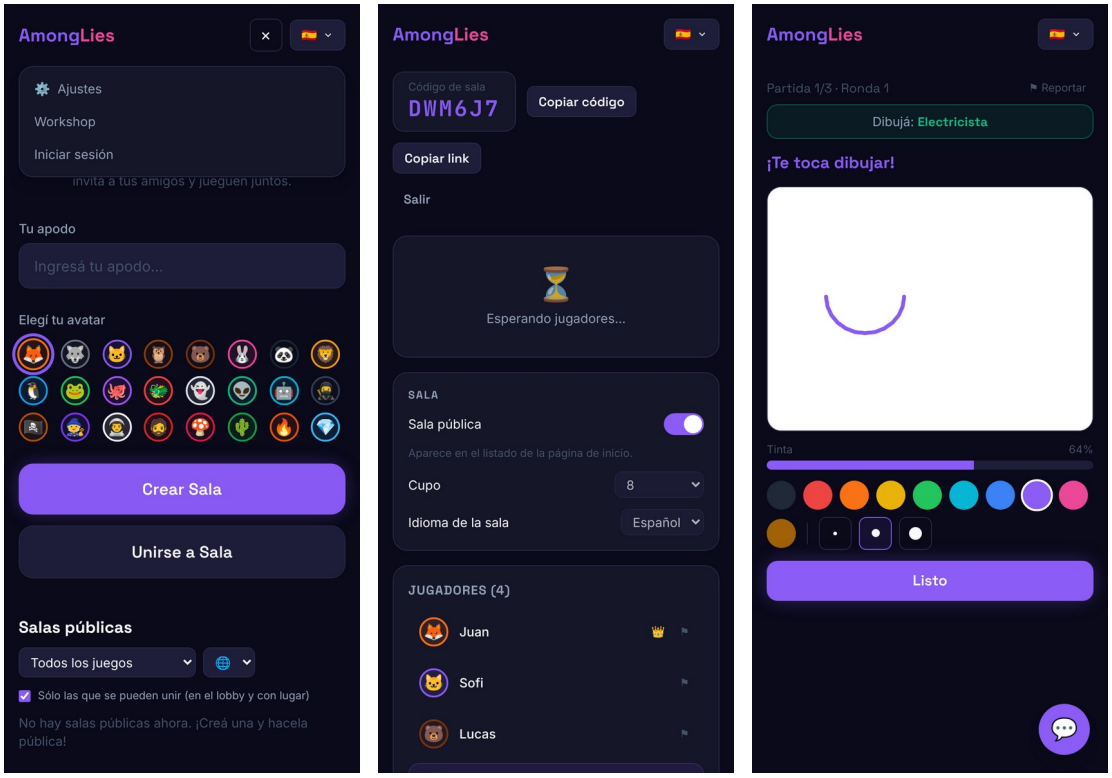


Figura 60 — En un celular: menú ☰, lobby e Impostor dibujo

M16. Problemas comunes

Qué ves	Por qué pasa	Qué hacer
"El apodo debe tener entre 2 y 16 caracteres"	El apodo es muy corto o muy largo.	Escribí un apodo de 2 a 16 caracteres.
"Ese apodo no está permitido"	El apodo tiene insultos.	Elegí otro.
"Ingresá un código de sala válido"	El código no tiene 6 caracteres.	Revisá el código; no incluye las letras O ni I ni los números 0 y 1.
"No existe una sala con ese código"	El código está mal escrito o la sala se cerró porque todos salieron.	Pedile el link al anfitrión o creá una sala nueva.
"La partida ya está en curso"	La sala está en medio de un juego.	Esperá a que vuelvan al lobby e intentá de nuevo.
"La sala está llena"	Se alcanzó el cupo.	Pedile al anfitrión que aumente el cupo o buscá otra sala.
"Despertando el servidor..."	El servidor estaba dormido (pasaron 15 minutos sin uso).	Esperá: puede tardar hasta un minuto. Después todo anda normal.
"La sala ya no existe: el servidor se reinició"	El servidor se reinició (por ejemplo, por una actualización).	Creá una sala nueva y volvé a invitar.
"Partida en pausa"	Se desconectó un jugador.	Esperá a que vuelva; quien decide puede tocar "Seguir sin esperar".
El botón "Iniciar Juego" está gris	No hay jugadores suficientes.	Los tres juegos necesitan al menos 4 jugadores; debajo del botón se indica qué falta.
"Esa palabra ya fue usada, elegí otra."	Otro jugador ya dio esa pista en esta ronda.	Escribí una pista distinta.
"No podés decir la palabra secreta."	Escribiste la palabra secreta como pista.	Dá una pista relacionada, no la palabra.
"Seleccioná al menos una lista de palabras..."	El anfitrión desmarcó todas las listas.	Marcá al menos una.
"Dibujá un poco más para poder terminar"	Usaste menos tinta que la mínima.	Seguí dibujando un poco y tocá Listo.
"Tu cuenta está suspendida..."	Un administrador suspendió tu cuenta.	Esperá a la fecha indicada; podés seguir mirando tu perfil.
Volviste a la pantalla de inicio de golpe	El anfitrión te expulsó, o cerraste la pestaña.	Volvé a entrar con el link si el anfitrión está de acuerdo.
No podés chatear ni votar	Fuiste eliminado en esta partida.	Es normal: en la próxima partida volvés a jugar.
No escucho a nadie en modo voz	AmongLies no transmite audio.	Júntense en persona o abran una llamada aparte.
No se oye ningún sonido	Los navegadores no reproducen sonido hasta que tocás algo; o está apagado en Ajustes.	Tocá cualquier parte de la página y revisá Ajustes → Sonido.
No se guardaron mis estadísticas	Jugaste como invitado.	Iniciá sesión antes de jugar.

Tabla 44 — Problemas comunes y soluciones

M17. Preguntas frecuentes

¿Es gratis?

Sí. Jugar y crear una cuenta es gratuito.

¿Tengo que registrarme?

No. Podés jugar como invitado. La cuenta sirve para guardar estadísticas, tener amigos y usar el workshop.

¿Puede el anfitrión ver quién es el impostor?

No. El anfitrión es un jugador más: sólo ve su propio rol. Ni siquiera el anfitrión recibe la información secreta de los demás.

¿Cuántas personas pueden jugar en una sala?

Por defecto no hay límite; el anfitrión puede fijar un cupo. Los tres juegos necesitan al menos 4 jugadores; con más de 10 las rondas se hacen largas.

¿Se guardan mis mensajes de chat?

No. El chat existe sólo mientras la sala está abierta. Si alguien te reporta, los últimos mensajes de la sala se adjuntan al reporte para que lo revise un administrador.

¿Puedo unirme a una sala que ya empezó?

No durante el juego: la sala acepta jugadores nuevos cuando está en el lobby. Apenas terminen y vuelvan al lobby, podés unirte con el mismo link o código.

¿Qué pasa si se me corta internet?

Volvé a abrir la página (o esperá a que se reconecte sola): volvés a tu lugar en la sala y la partida te espera pausada.

¿Puedo jugar con mis propias palabras?

Sí: creá una lista en el workshop (M12) y elegila en el lobby.

Anexo A — Fuentes de los diagramas (Mermaid)

Estas son las fuentes exactas a partir de las cuales se generaron todas las figuras de diagramas del documento. Se renderizan con Mermaid 11 (los estilos de color se agregan al renderizar). Para modificar un diagrama se edita su texto y se vuelve a generar la imagen (docs-as-code).

d01_contexto — C4 Nivel 1 — Diagrama de Contexto del Sistema

```
flowchart LR
  inv["👤 Jugador invitado<br/><small>Juega sin cuenta, con apodo y avatar</small>"]:::person
  reg["👤 Usuario registrado<br/><small>Estadísticas, amigos y workshop</small>"]:::person
  host["👑 Anfitrión de sala<br/><small>Crea y configura la sala; modera</small>"]:::person
  adm["🛡️ Administrador<br/><small>Revisa reportes y suspende cuentas</small>"]:::person
  sys["AmongLies<br/><small>Plataforma web de minijuegos sociales<br/>multijugador en tiempo
real</small>"]:::system
  sb["Supabase<br/><small>Auth (registro, login, JWT)<br/>y base PostgreSQL</small>"]:::ext
  voice["Apps de voz externas<br/><small>Discord / Meet / presencial</small>"]:::ext
  inv -- "Crea/une salas, juega, chatea<br/>[HTTPS + WSS]" --> sys
  reg -- "Juega, perfil, amigos,<br/>workshop, ajustes" --> sys
  host -- "Configura, inicia, expulsa" --> sys
  adm -- "Panel de moderación" --> sys
  sys -- "Verifica JWT; guarda estadísticas,<br/>amigos, workshop y reportes" --> sb
  reg -- "Login / registro" --> sb
  inv -. "Hablan en modo voz<br/>(fuera del sistema)" .- voice
```

d02_contenedores — C4 Nivel 2 — Diagrama de Contenedores

```
flowchart TB
  user["👤 Jugador<br/><small>Invitado o registrado</small>"]:::person
  subgraph B["Sistema AmongLies"]
    direction TB
    web["Aplicación Web<br/><small>Next.js 16 · React 19 · Zustand<br/>UI, estado local, i18n,
sonido</small>"]:::container
    gs["Game Server<br/><small>NestJS 11 · Socket.io 4<br/>Salas, chat, motores de
juego,<br/>amigos, workshop, moderación<br/>Estado vivo en memoria</small>"]:::container
    pg[("PostgreSQL (Supabase)<br/><small>Perfiles, estadísticas, amistades,<br/>workshop,
reportes · RLS</small>")]:::db
    shared["Paquete shared<br/><small>Tipos, eventos, constantes,<br/>listas de palabras, filtro
de insultos</small>"]:::comp
  end
  auth["Supabase Auth"]:::ext
  user -- "HTTPS" --> web
  web -- "Eventos en tiempo real [WSS · Socket.io]<br/>+ GET /rooms/public, /health [HTTPS]" -->
gs
  web -- "Login / registro<br/>[HTTPS]" --> auth
  web -- "Lee perfil, estadísticas<br/>y workshop [HTTPS · RLS]" --> pg
  gs -- "Escribe estadísticas, amistades,<br/>workshop y reportes<br/>[HTTPS · clave secreta]"
--> pg
  gs -- "Verifica JWT (JWKS)" --> auth
  shared -. "importado en build" .-> web
  shared -. "importado en build" .-> gs
```

d03_componentes_server — C4 Nivel 3 — Componentes del Game Server

```
flowchart TB
  web["Aplicación Web"]:::container
  subgraph GS["Game Server (NestJS)"]
    direction TB
    subgraph T["Capa de transporte"]
      rg["RoomGateway<br/><small>handshake (JWT opcional),
room:create/join/<br/>leave/kick/transfer-admin/update-settings</small>"]:::comp
      cg["ChatGateway<br/><small>chat:send · filtro de insultos</small>"]:::comp
      gg["GameGateway<br/><small>game:select/start/action/draw ·<br/>vistas por
jugador</small>"]:::comp
      sg["SocialGateway<br/><small>social:* amigos, presencia,<br/>invitaciones</small>"]:::comp
      wg["WorkshopGateway<br/><small>workshop:* listas y presets</small>"]:::comp
      mg["ModerationGateway<br/><small>report:create, admin:*,<br/>account:*</small>"]:::comp
      http["RoomsController · HealthController<br/><small>GET /rooms/public · GET
/health</small>"]:::comp
    end
    subgraph D["Capa de dominio"]
      ps["PlayerService + AuthService<br/><small>identidad (invitado o cuenta),<br/>reconexión y
pausa</small>"]:::comp
      rs["RoomStore<br/><small>salas en memoria, salas públicas,<br/>listas del workshop por
sala</small>"]:::comp
      gsvc["GameService + registro<br/><small>crea el motor del juego elegido</small>"]:::comp
      subgraph E["Motores de juego (GameEngine)"]
        de["DeductionEngine<br/><small>partidas, rondas, votos, puntaje,<br/>timers
pausables</small>"]:::comp
        ie["ImpostorEngine"]:::comp
        te["TimeEngine"]:::comp
        dr["DrawingEngine"]:::comp
      end
      soc["SocialService"]:::comp
      ws["WorkshopService"]:::comp
      mod["ModerationService +<br/>AccountService"]:::comp
      ss["StatsService"]:::comp
    end
  end
  pg[("Supabase<br/>PostgreSQL")]:::db
  web --> rg & cg & gg & sg & wg & mg & http
  rg --> ps & rs
  cg --> rs
  http --> rs
  gg --> gsvc --> de
  de --> ie & te & dr
  gsvc --> ss
  sg --> soc
  wg --> ws
  mg --> mod
  ss & soc & ws & mod --> pg
```

d04_componentes_web — C4 Nivel 3 — Componentes de la Aplicación Web

```
flowchart TB
  user["👤 Jugador"]:::person
  subgraph W["Aplicación Web (Next.js App Router)"]
    direction TB
    subgraph P["Páginas (rutas)"]
      home["/ · Inicio<br/><small>apodo, avatar, crear/unirse,<br/>salas
públicas</small>"]:::comp
      room["/room/[code] · Sala<br/><small>lobby + juego</small>"]:::comp
      acc["/login · /registro · /recuperar<br/>/perfil · /ajustes"]:::comp
      soc["/amigos · /workshop · /admin"]:::comp
    end
    subgraph C["Componentes de UI"]
      lobby["Lobby<br/><small>PlayerList, Chat, GameSelector,
GameSettings,<br/>RoomSettingsCard, WorkshopLobbyTools</small>"]:::comp
      games["Juegos<br/><small>ImpostorGame, TimeGame, DrawingGame<br/>+ fases comunes
(deduction/)</small>"]:::comp
      glob["Globales del layout<br/><small>Header, RoomEvents,
SocialInit,<br/>SoundManager</small>"]:::comp
    end
    subgraph S["Estado y servicios"]
      stores["Stores Zustand<br/><small>player, room, auth,<br/>social,
settings</small>"]:::comp
      hooks["Hooks<br/><small>useSocket, useGame,<br/>useTranslation,
useGameSounds</small>"]:::comp
      sock["lib/socket<br/><small>Socket.io tipado</small>"]:::comp
      sb["lib/supabase<br/><small>Auth + lecturas con RLS</small>"]:::comp
      i18n["lib/i18n<br/><small>es · en</small>"]:::comp
    end
  end
  gs["Game Server"]:::container
  sup["Supabase"]:::ext
  user --> home & room & acc & soc
  room --> lobby & games
  home & room --> stores
  glob --> hooks --> sock --> gs
  games --> hooks
  acc & soc --> sb --> sup
  C --> i18n
```

d05_clases_motor — C4 Nivel 4 — Código: contrato de los motores de juego

```
classDiagram
  direction LR
  class GameEngine {
    <<interface>>
    +start() void
    +handleAction(playerId, action, ctx) string|null
    +getStateForPlayer(playerId) GameView
    +canChat(playerId) boolean
    +pause() void
    +resume() void
    +removePlayer(playerId) void
    +getPlayerStats() PlayerGameStats[]
    +destroy() void
  }
  class GameRegistration {
    <<interface>>
    +validateStart(players, settings, wordLists) string|null
    +create(players, settings, callbacks, wordLists) GameEngine
  }
  class DeductionEngine {
    <<abstract>>
    #partidas, rondas, turnos, votos, puntaje
    #startRoundActivity()*
    #beginTurn()*
    #handleActivityAction()*
    #buildView()*
  }
  class ImpostorEngine {
    pistas por turno (chat o voz)
  }
  class TimeEngine {
    EMPEZAR / PARAR contra un tiempo oculto
  }
  class DrawingEngine {
    lienzo compartido con tinta limitada
  }
  class GameDefinition {
    +id: GameId
    +minPlayers: number
    +supportedModes
    +settingsSchema
  }
  GameEngine <|.. DeductionEngine
  DeductionEngine <|-- ImpostorEngine
  DeductionEngine <|-- TimeEngine
  DeductionEngine <|-- DrawingEngine
  GameRegistration ..> GameEngine : crea
  GameRegistration ..> GameDefinition : usa
```

d06_estados_impostor — Máquina de estados del motor de deducción

```
stateDiagram-v2
  direction TB
  [*] --> WordReveal : game:start (host, ≥4 jugadores)
  WordReveal --> Turns : timer (5–30 s)
  Turns --> Turns : pista enviada / timeout / Listo (voz)
  Turns --> Discussion : todos dieron su pista
  Discussion --> Voting : timer o consenso "Saltear"
  Voting --> VoteResults : todos votaron o timer
  VoteResults --> WordReveal : partida sigue (nueva ronda)
  VoteResults --> PartidaEnd : impostores eliminados o igualan a inocentes
  VoteResults --> GameEnd : última partida terminada
  PartidaEnd --> WordReveal : 15 s o consenso "Continuar"
  GameEnd --> [*] : Volver al lobby
  WordReveal : Revelación de palabra
  Turns : Turnos de pistas
  Discussion : Discusión
  Voting : Votación
  VoteResults : Resultados
  PartidaEnd : Fin de partida
  GameEnd : Fin del juego
```

d07_seq_sala — Escenario 1 — Crear una sala y unirse

```
sequenceDiagram
  autonumber
  participant H as Anfitrión
  participant W1 as Web (anfitrión)
  participant GS as Game Server
  participant RS as RoomStore (memoria)
  participant J as Jugador
  participant W2 as Web (jugador)
  H->>W1: Ingresa apodo + avatar, "Crear Sala"
  W1->>GS: room:create {nickname, avatarId, locale}
  GS->>GS: Genera código de 6 caracteres único
  GS->>RS: Guarda la sala (privada por defecto)
  GS-->>W1: room:created {room, playerId}
  W1->>H: Redirige a /room/{code}
  H-->>J: Comparte código, link o invitación a un amigo
  J->>W2: Abre link / ingresa código
  W2->>GS: room:join {code, nickname, avatarId}
  GS->>RS: Busca la sala
  alt Sala inexistente, en juego o llena
    GS-->>W2: room:error {ROOM_NOT_FOUND | GAME_IN_PROGRESS | ROOM_FULL}
  else Sala válida
    GS->>RS: Agrega jugador
    GS-->>W2: room:joined {room, playerId}
    GS-->>W1: room:player-joined {player}
  end
```

d08_seq_ronda — Escenario 2 — Una ronda de El Impostor (servidor autoritativo)

```
sequenceDiagram
  autonumber
  participant W as Web (cada jugador)
  participant GG as GameGateway
  participant E as ImpostorEngine
  participant T as Timers pausables
  W->>GG: game:start (sólo anfitrión)
  GG->>E: start()
  E->>E: Elige impostor(es) y palabra al azar
  E-->>GG: onPhaseChange("word-reveal")
  loop Por cada socket de la sala
    GG->>E: getStateForPlayer(playerId)
    GG-->>W: game:state-update (vista personalizada)
  end
  Note over W,GG: El inocente recibe la palabra.<br/>El impostor recibe secretWord = null.
  T-->>E: fin de "word-reveal" → "turns"
  W->>GG: game:action {submit-word, "fruta"}
  GG->>E: handleAction()
  alt Palabra repetida o igual a la secreta
    E-->>W: game:error {word_already_used | word_is_secret}
  else Válida
    E-->>GG: estado actualizado → vistas
  end
  Note over E: Discusión → Votación
  W->>GG: game:action {vote, targetId | "skip"}
  E->>E: decideVoteOutcome(): mayoría, empate, condición de fin, puntaje
  GG-->>W: game:state-update (fase vote-results)
```

d09_seq_stats — Escenario 3 — Fin del juego y registro de estadísticas

```
sequenceDiagram
  autonumber
  participant E as Motor de juego
  participant GS as GameService
  participant S as StatsService
  participant PG as Supabase (PostgreSQL)
  participant W as Web
  E-->>GS: onGameEnd()
  GS-->>W: game:state-update (fase game-end, puntajes)
  GS->>E: getPlayerStats()
  GS->>S: record(gameId, sólo jugadores con cuenta)
  S->>PG: rpc record_game_stats(entries) con la clave secreta
  alt Supabase no responde
    S->>S: Registra el error en el log y el juego sigue igual
  end
  Note over S,PG: Los invitados no se guardan:<br/>no tienen cuenta donde sumar estadísticas
  W->>PG: SELECT player_stats WHERE user_id = … (lectura pública)
  PG-->>W: Estadísticas actualizadas en /perfil
```

d10_seq_login — Escenario 4 — Inicio de sesión y conexión autenticada

```
sequenceDiagram
  autonumber
  participant U as Usuario
  participant W as Web
  participant A as Supabase Auth
  participant GS as Game Server
  U->>W: "Iniciar sesión" (email y contraseña)
  W->>A: signInWithPassword
  A-->>W: access_token (JWT) + refresh_token
  W->>W: authStore guarda la sesión y carga el perfil
  W->>GS: Conexión Socket.io con auth: {sessionToken, accessToken}
  GS->>GS: Verifica firma, emisor y vencimiento (JWKS público)
  alt Token válido
    GS->>GS: La identidad es el id de la cuenta<br/>y se leen sus marcas (admin, suspensión,
privacidad)
  else Sin token o inválido
    GS->>GS: Invitado identificado por su sessionToken
  end
  GS-->>W: session:ready (y la sala, si estaba en una)
```

d11_seq_reconexion — Escenario 5 — Desconexión, pausa y reconexión

```
sequenceDiagram
  autonumber
  participant W as Web
  participant GS as Game Server
  participant E as Motor de juego
  W--xGS: Se corta la conexión
  GS-->>W: player:disconnected (a la sala)
  alt Partida en curso
    GS->>E: pause() (los timers se congelan)
  end
  alt Vuelve (misma pestaña o recarga)
    W->>GS: reconecta con el mismo sessionToken / cuenta
    GS->>E: resume()
    GS-->>W: session:ready + sala + game:state-update actual
  else En el lobby y pasan 30 s
    GS-->>W: room:player-left {playerId, newAdminId?}
  else En partida: quien decide toca "Seguir sin esperar"
    GS->>E: removePlayer(playerId) y resume()
  end
```

d12_er — Modelo entidad–relación

```
erDiagram
  AUTH_USERS ||--|| PROFILES : "1 a 1"
  PROFILES ||--o{ PLAYER_STATS : "acumula"
  PROFILES ||--o{ FRIENDSHIPS : "pide / recibe"
  PROFILES ||--o{ WORKSHOP_ITEMS : "es dueño"
  WORKSHOP_ITEMS ||--o{ WORKSHOP_ITEMS : "copia de"
  PROFILES ||--o{ WORKSHOP_LIKES : "da"
  WORKSHOP_ITEMS ||--o{ WORKSHOP_LIKES : "recibe"
  PROFILES ||--o{ REPORTS : "reporta / es reportado"
  AUTH_USERS {
    uuid id PK
    text email
  }
  PROFILES {
    uuid id PK "FK auth.users"
    citext username UK
    text avatar_id
    text locale "es | en"
    bool is_admin
    timestamptz suspended_until
    bool appear_offline
    bool allow_invites
  }
  PLAYER_STATS {
    uuid user_id PK
    text game_id PK
    int games_played
    int partidas_as_impostor
    int partidas_won_as_impostor
    int correct_votes
    int total_points
  }
  FRIENDSHIPS {
    uuid requester_id PK
    uuid addressee_id PK
    text status "pending | accepted"
  }
  WORKSHOP_ITEMS {
    uuid id PK
    uuid owner_id FK
    text kind "word_list | preset"
    text title
    jsonb content
    int version
    bool published
    uuid source_id FK "copia"
    int likes_count
  }
  WORKSHOP_LIKES {
    uuid user_id PK
    uuid item_id PK
  }
  REPORTS {
    uuid id PK
    text kind "player | workshop_item"
    uuid reporter_id FK
    uuid target_user_id FK
    text reason
    jsonb evidence
    text status "open | resolved | dismissed"
  }
```

d13_despliegue — Despliegue en producción

```
flowchart LR
  subgraph Cliente["Dispositivo del jugador"]
    br["Navegador moderno<br/><small>Chrome, Edge, Firefox, Safari<br/>desktop y
móvil</small>"]:::ext
  end
  subgraph GH["GitHub"]
    repo["Repositorio monorepo<br/><small>main → deploy automático</small>"]:::ext
    ci["GitHub Actions<br/><small>secretos · lint · build · tests ·<br/>audit · pruebas
e2e</small>"]:::ext
  end
  subgraph RD["Render (plan gratis, Virginia)"]
    websvc["amonglies-web<br/><small>Node 22 · Next.js standalone</small>"]:::container
    gs1["amonglies-server<br/><small>Node 22 · NestJS · 1 instancia<br/>estado vivo en
memoria</small>"]:::container
  end
  subgraph SB["Supabase (región sa-east-1)"]
    sauth["Auth"]:::ext
    spg[("PostgreSQL 17")]:::db
  end
  br -- "HTTPS 443" --> websvc
  br -- "WSS / HTTPS 443" --> gs1
  gs1 -- "HTTPS (REST, clave secreta)" --> spg
  br -- "HTTPS (login)" --> sauth
  br -- "HTTPS (lecturas con RLS)" --> spg
  repo --> ci
  repo -- "push a main" --> RD
```

d14_calidad — Árbol de calidad

```
flowchart LR
  Q((Calidad<br/>AmongLies)):::root
  Q --> P[Rendimiento]:::a
  Q --> I[Integridad del juego]:::a
  Q --> U[Usabilidad]:::a
  Q --> X[Extensibilidad]:::a
  Q --> D[Disponibilidad]:::a
  Q --> M[Mantenibilidad]:::a
  P --> P1["QS-1 Latencia de evento p95 < 150 ms"]:::s
  P --> P2["QS-2 Inicio carga en < 2,5 s (4G)"]:::s
  I --> I1["QS-3 Ningún cliente recibe la palabra<br/>si es impostor"]:::s
  I --> I2["QS-4 Acciones fuera de turno o de<br/>no anfitrión son rechazadas"]:::s
  U --> U1["QS-5 De la portada a jugar en < 30 s<br/>sin registrarse"]:::s
  U --> U2["QS-6 Usable desde 360 px de ancho"]:::s
  X --> X1["QS-7 Nuevo minijuego sin tocar<br/>salas, chat ni transporte"]:::s
  D --> D1["QS-8 Un corte breve de conexión<br/>no saca al jugador de la partida"]:::s
  D --> D2["QS-9 Si el servidor se reinicia,<br/>los jugadores se enteran y pueden seguir"]:::s
  M --> M1["QS-10 Cambio de contrato de evento<br/>detectado en compilación o CI"]:::s
```

d15_flujo_usuario — Manual — Mapa de navegación

```
flowchart TB
  A([Entrar a AmongLies]) --> B{¿Tiene cuenta?}
  B -- "No / no quiere" --> C[Elegir apodo<br/>y avatar]
  B -- "Sí" --> L[Iniciar sesión]
  L --> C
  C --> D{¿Qué quiere hacer?}
  D -- "Crear sala" --> E[Lobby como anfitrión]
  D -- "Tengo código, link<br/>o invitación" --> F[Lobby como jugador]
  D -- "Buscar sala" --> G[Salas públicas] --> F
  E --> H[Elegir minijuego<br/>y configurar]
  H --> I[Iniciar juego]
  F --> J[Esperar al anfitrión] --> K
  I --> K[Jugar partidas]
  K --> M[Fin del juego]
  M --> N[Volver al lobby] --> H
  M --> O[Ver perfil y<br/>estadísticas]
  style A fill:#8B5CF6,color:#fff,stroke:#6D28D9
  style M fill:#10B981,color:#fff,stroke:#047857
```

d16_flujo_impostor — Manual — Fases de una partida

```
flowchart TB
  A["1 · Ver tu palabra<br/><small>o descubrir que sos<br/>el impostor</small>"] --> B["2 · Dar
una pista<br/><small>por turnos, en chat<br/>o en voz alta</small>"]
  B --> C["3 · Discutir<br/><small>chat libre con<br/>tiempo límite</small>"]
  C --> D["4 · Votar<br/><small>a quien creés que<br/>es el impostor</small>"]
  D --> E["5 · Resultados<br/><small>¿era o no era?</small>"]
  E -- "Nadie ganó todavía" --> A
  E -- "Ganó un bando" --> F["6 · Fin de partida<br/><small>se revela el impostor</small>"]
  F -- "Quedan partidas" --> A
  F -- "Era la última" --> G["7 · Fin del juego"]
  style G fill:#10B981,color:#fff,stroke:#047857
```

d17_capas — Servidor autoritativo

```
flowchart LR
  subgraph Clientes["Clientes (navegadores)"]
    c1["Jugador A<br/><small>sólo emite intenciones</small>"]
    c2["Jugador B"]
    c3["Jugador C"]
  end
  subgraph Server["Game Server — única fuente de verdad"]
    v["Valida<br/><small>turno, rol, permisos,<br/>reglas del juego</small>"] --> a["Aplica al
estado<br/><small>motor de juego</small>"] --> f["Filtra por
jugador<br/><small>getStateForPlayer</small>"]
  end
  c1 -- "evento: game:action" --> v
  f -- "vista A" --> c1
  f -- "vista B" --> c2
  f -- "vista C" --> c3
  style Server fill:#F5F3FF,stroke:#8B5CF6
```

Anexo B — Referencias

- Starke, G. y Hruschka, P. *arc42 — Template for documentation of software and system architectures*. https://arc42.org
- Brown, S. *The C4 model for visualising software architecture*. https://c4model.com
- Working Software. *Software Architecture Documentation — The Ultimate Guide*. https://www.workingsoftware.dev/software-architecture-documentation-the-ultimate-guide/
- freeCodeCamp. *System Architecture Documentation Best Practices and Tools*. https://www.freecodecamp.org/news/system-architecture-documentation-best-practices-and-tools/
- Nygard, M. *Documenting Architecture Decisions* (formato ADR). https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions
- Documentación oficial: Socket.io (https://socket.io/docs/v4/), NestJS (https://docs.nestjs.com), Next.js (https://nextjs.org/docs), Supabase (https://supabase.com/docs), Render (https://render.com/docs), Playwright (https://playwright.dev), Jest (https://jestjs.io), Mermaid (https://mermaid.js.org).
- Ley 25.326 de Protección de los Datos Personales, República Argentina.
- Repositorio del proyecto: https://github.com/JuanCruzPalacios/AmongLies (incluye `docs/TESTING.md` con el detalle de las pruebas y `docs/PROGRESO.md` con las fases del desarrollo).
- Aplicación en producción: https://amonglies-web.onrender.com